

梳理论文补充要求

会话 ID: 019f4b03-e09c-7110-a425-04e6d87dec5f 开始时间: 2026-07-10 09:53:10 消息数: 303

用户 2026-07-10 09:53:10

Files mentioned by the user:

Research_Proposal.pdf: C:/Users/XiaoyanXiong/Documents/rearch topic/Research_Proposal.pdf

My request for Codex:

这是我的proposal。要完成最终的说是论文，我还需要什么？详细解析

助手 2026-07-10 09:53:23

我会先把 PDF 内容抽取出来，看看 proposal

里已经覆盖了哪些研究问题、方法、数据和实验设计，再按“最终论文还缺什么”拆成可执行清单。这里会用 PDF 处理流程来读内容，而不是只凭题目猜。

助手 2026-07-10 09:53:34

我已经确认会按 PDF 技能来处理。下一步我会定位可用的 PDF 文本/渲染工具，然后抽取 proposal 的正文和页数；如果外部路径读取被沙箱拦住，我会请求一次读取授权。

助手 2026-07-10 09:53:43

PDF 文件可读，大小约 326 KB，修改时间是 2026-04-24。现在我会用本地 PDF 工具抽取文本，同时查看页数，确保分析是基于 proposal 内容本身。

助手 2026-07-10 09:54:00

文本抽取库和 Poppler 渲染工具都在本地可用。为了避免只看纯文本漏掉结构，我会同时生成页面图片，快速检查 proposal 的版面和章节分布。

助手 2026-07-10 09:54:14

临时目录已准备好。我现在会提取 PDF

元数据、正文文本，并渲染页面图片；这些只是分析用的中间文件，不会改你的 proposal。

助手 2026-07-10 09:54:34

刚才有两个小问题：Poppler 的`.cmd`包装器没有直接跑起来，Python 的单行脚本转义也被 PowerShell 吃掉了。我会先看一下包装器实际指向哪里，然后用更稳的方式重新抽取文本。

助手 2026-07-10 09:54:53

找到了：包装器指向的是`Library\bin`下的真实可执行文件。我会绕过包装器直接调用真实 exe，并把 Python 脚本作为小段内联脚本运行，这样转义会稳很多。

助手 2026-07-10 09:55:18

抽取完成: proposal 是 18 页 A4, 由 LaTeX 生成, 正文约 4 万字符。接下来我会快速提炼目录、研究目标、方法、数据集、实验计划和参考文献部分, 再给你一份“从 proposal 到最终论文”的缺口分析。

助手 2026-07-10 09:55:29

目录结构很清楚: proposal 已经有 Introduction、Related Work、Problem Formulation、Viable Approach、Initial Results of Baselines、Risks、Planning 和 References。现在我重点读方法、数据、实验结果和计划, 因为这些决定最终论文还差什么。

助手 2026-07-10 09:55:36

我读到的关键点是: 你的 proposal 设计的是“包级序列 + 时间编码 + 跨流上下文”的分层 Transformer NIDS; 但目前第 5 章明确写的是 baseline design, 还没有 finalized empirical results。这会成为最终论文最大的缺口。

助手 2026-07-10 09:55:43

版面检查也确认了两个图: 整体框架图已经能支撑方法章节, 但目前更像 proposal 示意图; 最终论文里它还需要配套更具体的算法、输入特征表和实验设置表。

助手 2026-07-10 09:55:55

我还会顺手看一下当前项目目录的大致文件结构, 判断你已经有多少实现/数据/结果沉淀在 workspace 里。这样最后给你的不是泛泛的论文建议, 而是更贴近这个 NIDSTransformer 项目的完成路线。

助手 2026-07-10 09:56:12

从项目目录看, 你已经不只是 proposal 阶段: 有 `s1`/`s2`、baseline 结果、context analysis、dataset overview、label consistency 等输出。Git 状态因为 Windows sandbox 用户所有权不同没法读, 但这不影响我判断论文缺口。

助手 2026-07-10 09:56:25

目录里能看到一个很重要的数据事实: 当前数据集约 177,278 flows, 其中攻击类只有 6,556, 占 3.70%。所以最终论文必须把 class imbalance、threshold、PR-AUC/recall/FPR、per-attack 或 per-scenario 讲清楚, 不能只报 accuracy。

助手 2026-07-10 09:56:37

我会再读几个已有结果文件, 确认你已经有哪些数字可以进论文、哪些还只是试跑。之后我就收束成一份章节级和实验级清单。

助手 2026-07-10 09:56:51

已有数字显示: Stage-I 包级模型 test F1(label1) 约 0.782、AUC 0.986、PR-AUC 0.856; Stage-II 现有上下文实验反而更低, test F1(label1) 大约 0.596-0.649。这个结果对最终论文很有价值: 它可能说明跨流上下文并非无条件提升, 必须做更严谨的消融和解释。

助手 2026-07-10 09:57:01

一个醒目的发现：flow-level LGBM baseline 在已有结果里几乎满分，这很可能会成为论文答辩时最先被问的问题。最终论文需要证明它不是数据泄漏/标签泄漏/随机切分过易导致的，或者明确把研究贡献转为“为什么复杂时序模型未必优于强 flow baseline”的严谨实证分析。

助手 2026-07-10 09:57:10

我还注意到不同结果文件里的样本数和正类比例不一致，有的总数据是 177k 且攻击 3.7%，有的 LGBM test 里攻击接近 44%。这不是一定错，但最终论文必须解释每个实验到底用了哪个 dataset、哪些筛选条件、哪些 split。

助手 2026-07-10 09:58:22

我读了你的 [Research_Proposal.pdf](C:/Users/XiaoyanXiong/Documents/research topic/Research_Proposal.pdf)，也快速看了当前项目里的 `s1/s2`、dataset overview 和已有 metrics。结论是：你的 proposal 框架已经比较完整，真正缺的不是“再想一个新方向”，而是把研究问题变成可答辩的、可复现的最终实验链条。

****核心判断****

你的论文主题是：

> Hierarchical Time-Aware Transformer for Flow-Level Network Intrusion Detection

proposal 已经有 RQ1/RQ2、相关工作、问题定义、模型框架、预处理计划、baseline 计划、风险和时间表。最终论文还需要补齐的是：真实实验结果、严谨对照、数据泄漏排查、跨数据集/跨场景验证、结果解释和完整写作。

目前 workspace 里已经有一些结果：Stage-I packet model 的 test `f1\_label1` 约 0.782、AUC 约 0.986、PR-AUC 约 0.856；但 Stage-II context 模型目前看起来没有明显超过 Stage-I，test `f1\_label1` 约 0.596-0.649。另一方面，flow-level LGBM baseline 几乎满分，这个必须重点处理，因为答辩时一定会被问：你的 Transformer 为什么需要存在？

****你还需要完成的 8 件事****

1. ****先固定最终研究问题****

不要只写“我提出一个更复杂模型”。最终论文要回答两个问题：

- RQ1: packet-level + time-aware modeling 是否比 flow aggregation 更有信息量？
- RQ2: cross-flow context 是否真的提升 distributed/multi-flow attack detection？

如果实验发现 Stage-II 不提升，也没关系，但要把论文转成严谨结论：cross-flow context 的收益依赖 context construction、label quality 和 dataset structure。

2. ****做数据集与标签的“审计章节”****

你需要一个非常清楚的数据表：

- 每个数据集来源：CIC-IDS2017、Wednesday-workingHours、internal PCAP、firewall_test、bacore001 等
- flows 数量、packets 数量、attack/benign 比例
- attack 类型分布
- label 来源：CIC label、Suricata rule label、人工/内部标注
- 是否有 duplicate flow_id
- 是否删除了 Suricata rule id、signature、alert metadata
- 是否去掉 source/destination IP、port、timestamp 等可能泄漏的字段

你当前有的一个 dataset overview 显示 177,278 flows, 其中 attack 只有 6,556, 占 3.70%。但 LGBM 实验里的 Wednesday dataset attack ratio 接近

44%。这不是一定错误, 但论文必须解释: 这些是不是不同数据集、不同筛选条件、不同 split。

3. **做 leakage / shortcut 排查**

这是最重要的。因为 LGBM baseline 接近 0.998-0.999 F1, 太强了。你必须检查:

- random split 是否让同一 attack campaign 的相似 flows 同时进入 train/test
- duplicate flow_id 是否跨 split
- flow-level statistical features 是否直接编码了 label 规则
- ports/protocol/timestamp/IP 是否造成 shortcut
- Suricata label 是否和输入特征来自同一规则逻辑

最少要做三组对照: full features、no ports/no IP/no endpoint、time-based split 或 capture-based split。

4. **完成正式 baseline 表**

最终论文至少需要这些 baseline:

- Classical flow baselines: LogReg、Random Forest、ExtraTrees、XGBoost/LightGBM
- Neural flow baselines: MLP、LSTM、CNN+LSTM、flow Transformer
- Packet baselines: packet Transformer without time、PE only、IAT only、PE+IAT
- Proposed: Stage-I full time-aware packet model
- Proposed + Stage-II context: time-only、same source、same destination、same endpoint、source-or-destination

每个模型要用同一 split、同一 metric、同一 threshold-selection 规则。

5. **把消融实验补完整**

你的 proposal 已经设计好了, 但最终论文需要真实表格:

- 时间编码 ablation: none / positional / log-IAT / combined
- sequence length: 64 / 100 / 128, head vs head-tail vs sampling
- context window: K=5/10/32/64/128
- context method: time-only / same source / same destination / same endpoint
- loss: cross entropy vs focal loss
- threshold: fixed 0.5 vs validation-selected threshold
- flow fusion: packet only vs packet + flow features

6. **结果不能只报 accuracy**

因为类别不平衡, accuracy 很容易误导。最终结果表建议固定报:

- Precision(label1)
- Recall(label1)
- F1(label1)
- Macro-F1
- AUROC
- PR-AUC
- FPR
- confusion matrix
- inference latency / memory

对 NIDS 来说, Recall 和 FPR 特别关键。一个模型如果 recall 高但 false positives 爆炸, 实际部署价值不高。

7. **写出“为什么”的讨论**

你现在已有结果其实很适合写讨论：

- Stage-I packet model 有不错 AUC/PR-AUC，但不一定打败 LGBM。
- Stage-II context 当前没有稳定提升，可能因为 time-only context 信号弱。
- 你的 context statistical validation 里，source-based context 看起来比 time-only 更有信号；这可以解释为什么 context construction 比“加一个 Transformer”更关键。
- 如果 flow baseline 过强，说明 benchmark 可能被 flow statistics 或 split shortcut 简化了。

8. **整理可复现代码和论文材料**

最终提交前，你需要把 scattered notebooks 收敛成一条 canonical pipeline：

- 一个 README：如何从 raw data 到 final tables
- 固定 config 文件
- 固定 random seeds
- 保存 train/val/test split IDs
- 保存最终模型 metrics JSON/CSV
- 保存图：architecture、dataset distribution、PR/ROC、confusion matrix、ablation bar chart、context comparison

最终论文建议结构

1. Introduction：动机、gap、RQ、贡献
2. Related Work：rule-based、flow ML、deep learning、Transformer、SSL、encrypted traffic、graph/context
3. Dataset and Preprocessing：这是你论文的关键章节，必须写扎实
4. Methodology：packet representation、time encoding、Stage-I、Stage-II、loss
5. Experimental Setup：baselines、splits、metrics、hyperparameters、leakage controls
6. Results：主结果、消融、cross-dataset、efficiency
7. Discussion：回答 RQ1/RQ2，解释失败/成功，部署代价，label bias
8. Conclusion and Future Work

最优先下一步

先不要急着继续调大模型。你现在最该做的是：

1. 固定最终数据集和 split。
2. 做 leakage audit。
3. 重新跑一套统一 baseline + Stage-I + Stage-II。
4. 用同一张主表回答 RQ1/RQ2。
5. 再开始写 Results 和 Discussion。

一句话：你的 proposal 已经有“论文骨架”；现在缺的是“可信实验”和“诚实解释”。尤其是 LGBM 几乎满分、Stage-II 暂时不升这两点，不是坏消息，但必须被你主动讲清楚。

用户 2026-07-13 09:30:12

方案B，其他配置都一样，只有max_seq_len不一样。strategy = head

max_seq_len=8 结果：[TEST] Full metrics: {'accuracy': 0.982010731431799, 'macro_precision': 0.8836832121604303, 'macro_recall': 0.8792368878526773, 'macro_f1': 0.8814457544489398, 'weighted_precision': 0.9819082755428861, 'weighted_recall': 0.982010731431799, 'weighted_f1': 0.9819583104766979, 'precision_label1': 0.7769784172661871, 'recall_label1': 0.767590618336887,

'f1_label1': 0.7722559885591705, 'auc': 0.9830632758458266, 'pr_auc': 0.8368694251226644, 'per_class_f1': {'0': 0.9906355203387089, '1': 0.7722559885591705}, 'confusion_matrix': [[33693, 310], [327, 1080]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.06234634028821646, 'threshold': 0.43000000000000027}

max_seq_len=16 结果: [TEST] Full metrics: {'accuracy': 0.9819542502118046, 'macro_precision': 0.884991341352839, 'macro_recall': 0.8758008642617157, 'macro_f1': 0.8803348883599431, 'weighted_precision': 0.9817464683095168, 'weighted_recall': 0.9819542502118046, 'weighted_f1': 0.9818452533172927, 'precision_label1': 0.7798833819241983, 'recall_label1': 0.7604832977967306, 'f1_label1': 0.7700611730838431, 'auc': 0.9849720501061185, 'pr_auc': 0.8454853179059679, 'per_class_f1': {'0': 0.990608603636043, '1': 0.7700611730838431}, 'confusion_matrix': [[33701, 302], [337, 1070]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.06207364177607027, 'threshold': 0.47100000000000003}

max_seq_len=32 结果: [TEST] Full metrics: {'accuracy': 0.9819824908218018, 'macro_precision': 0.8824893708595296, 'macro_recall': 0.880925490478379, 'macro_f1': 0.8817056626340283, 'weighted_precision': 0.9819458765725516, 'weighted_recall': 0.9819824908218018, 'weighted_f1': 0.9819640360960449, 'precision_label1': 0.7744468236973591, 'recall_label1': 0.7711442786069652, 'f1_label1': 0.7727920227920227, 'auc': 0.9854155913873647, 'pr_auc': 0.8571311582683947, 'per_class_f1': {'0': 0.9906193024760337, '1': 0.7727920227920227}, 'confusion_matrix': [[33687, 316], [322, 1085]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.058221365120060174, 'threshold': 0.401000000000000025}

max_seq_len=64 结果: [TEST] Full metrics: {'accuracy': 0.9824060999717594, 'macro_precision': 0.8815947276477233, 'macro_recall': 0.8906845796310334, 'macro_f1': 0.8860805538653445, 'weighted_precision': 0.9826230585914077, 'weighted_recall': 0.9824060999717594, 'weighted_f1': 0.9825096408370192, 'precision_label1': 0.7718446601941747, 'recall_label1': 0.7910447761194029, 'f1_label1': 0.7813267813267813, 'auc': 0.9852139807639784, 'pr_auc': 0.8568879868326769, 'per_class_f1': {'0': 0.9908343264039076, '1': 0.7813267813267813}, 'confusion_matrix': [[33674, 329], [294, 1113]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05798349744683999, 'threshold': 0.391000000000000024}

max_seq_len=128 结果: [TEST] Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.45500000000000003}

max_seq_len=256 结果: [TEST] Full metrics: {'accuracy': 0.9817565659418244, 'macro_precision': 0.8771857046002475, 'macro_recall': 0.8869397597573909, 'macro_f1': 0.8819939067109874, 'weighted_precision': 0.982000950488625, 'weighted_recall': 0.9817565659418244, 'weighted_f1': 0.9818730037159743, 'precision_label1': 0.7633217993079585, 'recall_label1': 0.7839374555792467, 'f1_label1': 0.773492286115007, 'auc': 0.9855195790345936, 'pr_auc': 0.8492833724420306, 'per_class_f1': {'0': 0.990495527306968, '1': 0.773492286115007}, 'confusion_matrix': [[33661, 342], [304, 1103]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.059235590674816915, 'threshold': 0.419000000000000026}

详细解析结果

助手 2026-07-13 09:30:41

结论: **方案 B + head strategy 下, `max\_seq\_len=128` 是当前最稳的主结果; `64` 是偏 recall 的备选; `32/64/128` 差距很小, 但 `8/16` 信息不足, `256` 开始变差。 **

****核心排序****

按 `f1\_label1`:

max_seq_len	F1(label1)	Precision1	Recall1	PR-AUC	AUC	FP/FN
8	0.7723	0.7770	0.7676	0.8369	0.9831	310 / 327
16	0.7701	0.7799	0.7605	0.8455	0.9850	302 / 337
32	0.7728	0.7744	0.7711	**0.8571**	0.9854	316 / 322
64	0.7813	0.7718	**0.7910**	0.8569	0.9852	329 / **294**
128	**0.7853**	**0.7881**	0.7825	0.8564	**0.9860**	**296** / 306
256	0.7735	0.7633	0.7839	0.8493	0.9855	342 / 304

****怎么解读****

`max\_seq\_len=8/16` 明显偏短。它们 precision 还可以, 但 recall 较低, 说明只看前 8 或 16 个 packet 时, 很多 attack flow 的关键模式还没完全出现。特别是 `16` 的 FN=337, 是所有设置里漏报最多的。

`max\_seq\_len=32` 开始进入有效区间。PR-AUC

最高, 说明排序能力很好, 也就是模型能比较好地区分正负样本概率顺序。但最终 F1 没有最高, 说明 threshold 后的 hard classification 还不是最佳。

`max\_seq\_len=64` 偏向抓攻击, recall=0.7910 最高, FN=294 最少。代价是 FP=329, 比 128 多 33 个误报。对于 NIDS, 如果你更重视 “少漏报”, 64 很有吸引力。

`max\_seq\_len=128` 是综合最优。它的 F1(label1)、macro-F1、accuracy、precision1、AUC、loss 都是最好或接近最好。更重要的是它同时保持较低 FP=296 和较低 FN=306, precision/recall 最均衡。论文主表建议用 128 作为方案 B 的代表结果。

`max\_seq\_len=256` 反而下降。说明 head strategy 下继续保留更多 packet

没有带来稳定收益, 可能引入了噪声、padding/长序列稀释、训练难度增加, 或者后半部分 packet 对分类贡献不如前 64-128 个明显。

****论文里可以这样写****

> Under the head truncation strategy, performance improves when increasing the packet sequence length from very short sequences to medium-length sequences, indicating that the first 8-16 packets are insufficient to capture complete attack-related temporal patterns. The best overall performance is achieved at `max\_seq\_len=128`, with the highest attack-class F1-score and AUROC. Increasing the length to 256 does not further improve performance, suggesting that additional packets may introduce redundant or noisy information while increasing computational cost.

****最终选择建议****

主结果: 用 `max\_seq\_len=128`。

消融讨论: 强调 `64` recall 最高, 适合更重视漏报控制的 NIDS 场景。

不推荐: `8/16` 信息不足, `256` 性能下降且成本更高。

一句话总结: **方案 B 的有效 packet context 大概在 64-128 之间, 128 是最稳的论文主配置, 64 是 recall-oriented 配置。 **

用户 2026-07-13 09:46:48

方案B, 其他配置都一样, 只有max_seq_len不一样。strategy = head

max_seq_len=8 结果: [TEST] Full metrics: {'accuracy': 0.982010731431799, 'macro_precision': 0.8836832121604303, 'macro_recall': 0.8792368878526773, 'macro_f1': 0.8814457544489398, 'weighted_precision': 0.9819082755428861, 'weighted_recall': 0.982010731431799, 'weighted_f1': 0.9819583104766979, 'precision_label1': 0.7769784172661871, 'recall_label1': 0.767590618336887, 'f1_label1': 0.7722559885591705, 'auc': 0.9830632758458266, 'pr_auc': 0.8368694251226644, 'per_class_f1': {'0': 0.9906355203387089, '1': 0.7722559885591705}, 'confusion_matrix': [[33693, 310], [327, 1080]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.06234634028821646, 'threshold': 0.43000000000000027}

max_seq_len=16 结果: [TEST] Full metrics: {'accuracy': 0.9819542502118046, 'macro_precision': 0.884991341352839, 'macro_recall': 0.8758008642617157, 'macro_f1': 0.8803348883599431, 'weighted_precision': 0.9817464683095168, 'weighted_recall': 0.9819542502118046, 'weighted_f1': 0.9818452533172927, 'precision_label1': 0.7798833819241983, 'recall_label1': 0.7604832977967306, 'f1_label1': 0.7700611730838431, 'auc': 0.9849720501061185, 'pr_auc': 0.8454853179059679, 'per_class_f1': {'0': 0.990608603636043, '1': 0.7700611730838431}, 'confusion_matrix': [[33701, 302], [337, 1070]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.06207364177607027, 'threshold': 0.47100000000000003}

max_seq_len=32 结果: [TEST] Full metrics: {'accuracy': 0.9819824908218018, 'macro_precision': 0.8824893708595296, 'macro_recall': 0.880925490478379, 'macro_f1': 0.8817056626340283, 'weighted_precision': 0.9819458765725516, 'weighted_recall': 0.9819824908218018, 'weighted_f1': 0.9819640360960449, 'precision_label1': 0.7744468236973591, 'recall_label1': 0.7711442786069652, 'f1_label1': 0.7727920227920227, 'auc': 0.9854155913873647, 'pr_auc': 0.8571311582683947, 'per_class_f1': {'0': 0.9906193024760337, '1': 0.7727920227920227}, 'confusion_matrix': [[33687, 316], [322, 1085]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.058221365120060174, 'threshold': 0.40100000000000025}

max_seq_len=64 结果: [TEST] Full metrics: {'accuracy': 0.9824060999717594, 'macro_precision': 0.8815947276477233, 'macro_recall': 0.8906845796310334, 'macro_f1': 0.8860805538653445, 'weighted_precision': 0.9826230585914077, 'weighted_recall': 0.9824060999717594, 'weighted_f1': 0.9825096408370192, 'precision_label1': 0.7718446601941747, 'recall_label1': 0.7910447761194029, 'f1_label1': 0.7813267813267813, 'auc': 0.9852139807639784, 'pr_auc': 0.8568879868326769, 'per_class_f1': {'0': 0.9908343264039076, '1': 0.7813267813267813}, 'confusion_matrix': [[33674, 329], [294, 1113]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05798349744683999, 'threshold': 0.39100000000000024}

max_seq_len=128 结果: [TEST] Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.45500000000000003}

max_seq_len=256 结果: [TEST] Full metrics: {'accuracy': 0.9817565659418244, 'macro_precision': 0.8771857046002475, 'macro_recall': 0.8869397597573909, 'macro_f1': 0.8819939067109874, 'weighted_precision': 0.982000950488625, 'weighted_recall': 0.9817565659418244, 'weighted_f1': 0.9818730037159743, 'precision_label1': 0.7633217993079585, 'recall_label1': 0.7839374555792467, 'f1_label1': 0.773492286115007, 'auc': 0.9855195790345936, 'pr_auc': 0.8492833724420306, 'per_class_f1': {'0': 0.990495527306968, '1': 0.773492286115007}, 'confusion_matrix': [[33661, 342], [304, 1103]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.059235590674816915, 'threshold': 0.41900000000000026}

详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-13 09:47:35

下面按你指定的 metrics 来看：**macro_f1、macro_recall、macro_precision、PR-AUC、AUC、confusion matrix、FPR**。

这里 confusion matrix 默认是：

```
[[TN, FP],
 [FN, TP]]
```

所以：

$$FPR = FP / (FP + TN)$$

****总览表****

max_seq_len	Macro-F1	Macro-Precision	Macro-Recall	PR-AUC	AUC	TN	FP	FN	TP	FPR
8	0.8814	0.8837	0.8792	0.8369	0.9831	33693	310	327	1080	0.9117%
16	0.8803	0.8850	0.8758	0.8455	0.9850	33701	302	337	1070	0.8882%
32	0.8817	0.8825	0.8809	**0.8571**	0.9854	33687	316	322	1085	0.9293%
64	0.8861	0.8816	**0.8907**	0.8569	0.9852	33674	329	**294**	1113	0.9676%
128	**0.8882**	**0.8896**	0.8869	0.8564	**0.9860**	**33707**	**296**	306	1101	**0.8705%**
256	0.8820	0.8772	0.8869	0.8493	0.9855	33661	342	304	1103	1.0058%

****最重要结论****

`max\_seq\_len=128` 是综合最优。

它在这些关键指标上都很好：

- `macro\_f1 = 0.8882`，最高
- `macro\_precision = 0.8896`，最高
- `AUC = 0.9860`，最高
- `FPR = 0.8705%`，最低
- `FP = 296`，最低
- accuracy、weighted-F1、loss 也最好

这说明 `128` 不只是 attack F1 高，而是整体分类边界最稳：误报最低，同时漏报没有明显变多。

****Macro-F1 分析****

Macro-F1 排名：

128 > 64 > 256 > 32 > 8 > 16

`128` 的 Macro-F1 最高，说明它对 benign 和 attack 两类都比较均衡。

`64` 第二，主要靠 recall 提升。

`256` 没有继续提升，说明更长序列没有带来更多有效信息，反而可能引入噪声或训练难度。

所以从 Macro-F1 看，最佳选择是：

max_seq_len = 128

****Macro-Precision 分析****

Macro-Precision 排名：

128 > 16 > 8 > 32 > 64 > 256

`128` 最高，说明它整体误报控制最好。这个也和 confusion matrix 一致：`128` 的 FP=296，是所有设置中最低的。

`64` 的 Macro-Precision 比 128 低，因为它抓到了更多攻击，但也多报了一些 benign 为 attack，FP=329。

`256` 最差，FP=342，说明太长的 head sequence 反而让模型更容易误判 benign。

****Macro-Recall 分析****

Macro-Recall 排名：

64 > 256 ≈ 128 > 32 > 8 > 16

`64` 的 Macro-Recall 最高，原因很直接：它的 FN=294，是所有设置中最低的，也就是漏报最少。

这说明 `max\_seq\_len=64` 更偏向 detection-sensitive setting：宁愿多一点误报，也要少漏攻击。

但是 `64` 的 FP=329，比 `128` 多 33 个。因此如果论文强调实际部署中的误报成本，`128` 更好；如果强调少漏报，`64` 可以作为 recall-oriented 配置讨论。

****PR-AUC 分析****

PR-AUC 排名：

32 > 64 > 128 > 256 > 16 > 8

PR-AUC 最高的是 `32`，不是 `128`。这说明 `32` 的 probability ranking 很好，也就是在不同 threshold 下，它区分正类 attack 的排序能力很强。

但 `32` 最终 threshold 后的 Macro-F1 和 attack F1 不如 `64/128`。所以可以这样解释：

> `max\_seq\_len=32` has strong ranking ability, as reflected by the highest PR-AUC, but its final thresholded classification performance is slightly weaker than longer sequences.

也就是说，`32` 不是没用，它说明前 32 个 packet 已经包含大量攻击判别信息。

****AUC 分析****

AUC 整体差距很小：

```
128: 0.98597
256: 0.98552
32: 0.98542
64: 0.98521
16: 0.98497
8: 0.98306
```

AUC 最高是 `128`。

但从 `16` 开始，AUC 都已经在 0.985 左右，说明模型整体 ranking 能力比较稳定。真正拉开差距的是 threshold 后的 FP/FN 平衡，而不是 AUC 本身。

论文里可以强调：

> AUC remains relatively stable for sequence lengths above 16, suggesting that the model can generally rank benign and malicious flows well. However, the confusion matrix and macro-level metrics reveal that the best operational trade-off is achieved at 128 packets.

****Confusion Matrix 和 FPR 分析****

看误报 FP:

```
128: 296
16: 302
8: 310
32: 316
64: 329
256: 342
```

看漏报 FN:

```
64: 294
256: 304
128: 306
32: 322
8: 327
16: 337
```

所以:

- `128`: 误报最少, 整体最稳
- `64`: 漏报最少, recall 最强
- `256`: 漏报还可以, 但误报最多, 不划算
- `8/16/32`: 序列偏短, 攻击模式捕获不足

FPR 排名:

```
128    0.8705%
16 0.8882%
8 0.9117%
32 0.9293%
64 0.9676%
256    1.0058%
```

在 NIDS 里, FPR 很重要, 因为正常流数量通常远大于攻击流。`128` 的 FPR 最低, 说明它最适合写成主配置。

****最终解释****

从这些结果看, 方案 B 在 `head` strategy 下呈现出一个很清楚的趋势:

1. `8/16` 太短, 无法稳定捕捉攻击行为, Macro-Recall 和 PR-AUC 偏低。
2. `32` 已经捕捉到主要判别信号, PR-AUC 最高。
3. `64` 最擅长减少漏报, Macro-Recall 最高, FN 最低。
4. `128` 综合最优, Macro-F1、Macro-Precision、AUC、FPR 都最好。
5. `256` 没有继续提升, 反而增加误报, 说明更长 head 序列带来冗余或噪声。

论文主结论可以写:

> The best overall trade-off is achieved at `max\_seq\_len=128`, which obtains the highest Macro-F1, Macro-Precision, AUROC, and the lowest false positive rate. Although `max\_seq\_len=64` achieves the highest Macro-Recall and lowest number of false negatives, it also increases false positives. Therefore, 128 packets are selected as the main configuration, while 64 packets can be considered when recall is prioritized over false-alarm reduction.

最终建议:

```
max_seq_len = 128
recall-oriented    max_seq_len = 64
PR-AUC/ranking    max_seq_len = 32
8 16 256
```

用户 2026-07-13 10:31:26

```
!python /content/drive/MyDrive/s1/run_comparison_0712.py \
--config /content/drive/MyDrive/s1/0712/B/compare/stage1_config_B_Both.yaml \
--data_dir /content/drive/MyDrive/s1/data/ \
--tensor_dir /content/drive/MyDrive/s1/0712/B/seq128 \
--experiment_name 0712_seed2026 \
--output_dir /content/drive/MyDrive/s1/0712/B/compare \
--num_epochs 500 \
--seed 2026 \
--device cuda
```

运行之后的结果：Model Params Time (s) Accuracy F1 Macro F1 Class1 Precision Class1 Recall Class1
ROC-AUC PR-AUC threshold precision_macro recall_macro precision_weighted recall_weighted
f1_weighted confusion_matrix

Ours_BothEncoding	344963	4337.6	0.9822	0.8841	0.7775	0.7739	0.7811	0.9852	0.8572	0.4257	0.8824	0.8858	0.9823	0.9822	0.9823	[[33682, 321], [308, 1099]]
Ours_TimeOnly	344963	2250.5	0.9822	0.8818	0.7728	0.7818	0.764	0.9846	0.851	0.4785	0.886	0.8776	0.982	0.9822	0.9821	[[33703, 300], [332, 1075]]
Ours_PositionOnly	344963	1994.7	0.9812	0.8739	0.7575	0.7791	0.737	0.9846	0.8482	0.4703	0.8841	0.8642	0.9808	0.9812	0.981	[[33709, 294], [370, 1037]]
LSTM_with_time	217346	3556.3	0.9806	0.8677	0.7455	0.7786	0.715	0.9801	0.8077	0.4638	0.8834	0.8533	0.9799	0.9806	0.9802	[[33717, 286], [401, 1006]]
CNN1D_with_time	70274	1162.4	0.9803	0.867	0.7443	0.768	0.7221	0.9793	0.7821	0.4822	0.8782	0.8565	0.9798	0.9803	0.98	[[33696, 307], [391, 1016]]
GRU	162690	3684.4	0.9797	0.8595	0.7296	0.7748	0.6894	0.9799	0.8009	0.4565	0.881	0.8406	0.9788	0.9797	0.9791	[[33721, 282], [437, 970]]
LSTM	216834	3381.2	0.9792	0.8582	0.7273	0.7575	0.6994	0.9796	0.7976	0.4272	0.8726	0.845	0.9785	0.9792	0.9788	[[33688, 315], [423, 984]]
CNN1D	68354	1591	0.9779	0.8579	0.7274	0.712	0.7434	0.9809	0.7975	0.4203	0.8507	0.8655	0.9783	0.9779	0.9781	[[33580, 423], [361, 1046]]
BiLSTM	564738	2625.7	0.9778	0.8522	0.716	0.7273	0.705	0.9797	0.7903	0.4408	0.8575	0.8471	0.9775	0.9778	0.9776	[[33631, 372], [415, 992]]
Ours_NoEncoding	344963	2213.9	0.977	0.8507	0.7134	0.7069	0.72	0.9816	0.8003	0.4335	0.8477	0.8538	0.9772	0.977	0.9771	[[33583, 420], [394, 1013]]

方案B， max_seq_len=128 and strategy = head

不同模型的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-13 10:32:20

下面这组结果可以很明确地支持一个结论：

**在方案 B、`max\_seq\_len=128`、`strategy=head` 下，`Ours\_BothEncoding` 是综合最优模型；`Ours\_TimeOnly` 是 precision/FPR 更稳的次优版本；`Ours\_PositionOnly` 单独用位置信息不够；传统 LSTM/GRU/CNN 整体落后。 **

confusion matrix 按：

```
[[TN, FP],
 [FN, TP]]
```

所以：

$$FPR = FP / (FP + TN)$$

核心表

Model	Macro-F1	Macro-P	Macro-R	PR-AUC	AUC	FP	FN	TP	FPR
--- --- --- --- --- --- --- --- ---									
Ours_BothEncoding	0.8841	0.8824	0.8858	0.8572	0.9852	321	308	1099	0.944%
Ours_TimeOnly	0.8818	0.8860	0.8776	0.8510	0.9846	300	332	1075	0.882%
Ours_PositionOnly	0.8739	0.8841	0.8642	0.8482	0.9846	294	370	1037	0.865%
LSTM_with_time	0.8677	0.8834	0.8533	0.8077	0.9801	286	401	1006	0.841%
CNN1D_with_time	0.8670	0.8782	0.8565	0.7821	0.9793	307	391	1016	0.903%
GRU	0.8595	0.8810	0.8406	0.8009	0.9799	282	437	970	0.829%
LSTM	0.8582	0.8726	0.8450	0.7976	0.9796	315	423	984	0.926%
CNN1D	0.8579	0.8507	0.8655	0.7975	0.9809	423	361	1046	1.244%
BiLSTM	0.8522	0.8575	0.8471	0.7903	0.9797	372	415	992	1.094%
Ours_NoEncoding	0.8507	0.8477	0.8538	0.8003	0.9816	420	394	1013	1.235%

Macro-F1

`Ours\_BothEncoding` 最高, `0.8841`。

这说明它不是只对 attack 类好, 也不是只对 benign 类好, 而是在两类之间取得了最好的整体平衡。相比:

- `Ours\_TimeOnly`: +0.0023
- `Ours\_PositionOnly`: +0.0102
- `LSTM\_with\_time`: +0.0164
- `Ours\_NoEncoding`: +0.0334

这个差距可以支持论文里的核心结论: **同时使用 position encoding 和 time encoding, 比单独使用任一种 encoding 更有效。**

Macro-Recall

`Ours\_BothEncoding` 也是最高, `0.8858`。

这个结果很重要, 因为 NIDS 里 recall 代表模型抓攻击的能力。它的 FN=308, 是所有模型里最低的, 也就是说漏报攻击最少。

对比几个关键模型:

Ours_BothEncoding	FN = 308
Ours_TimeOnly	FN = 332
PositionOnly	FN = 370
LSTM_with_time	FN = 401
GRU	FN = 437

所以 BothEncoding 比 TimeOnly 多抓到 24 个 attack, 比 PositionOnly 多抓到 62 个 attack, 比 GRU 多抓到 129 个 attack。

Macro-Precision

Macro-Precision 最高的是 `Ours\_TimeOnly = 0.8860`, 不是 BothEncoding。

这说明 TimeOnly 更保守, 误报更少一些。它的 FP=300, FPR=0.882%, 比 BothEncoding 的 FP=321、FPR=0.944% 更低。

但是代价是: TimeOnly 的 FN=332, 比 BothEncoding 多漏 24 个攻击。

所以这里是一个典型 trade-off:

TimeOnly	
BothEncoding	Macro-F1

从 NIDS 角度, BothEncoding 更适合作为主模型, 因为漏报攻击通常比多一点误报更危险。

****PR-AUC****

PR-AUC 排名最清楚:

Ours_BothEncoding	0.8572
Ours_TimeOnly	0.8510
Ours_PositionOnly	0.8482
LSTM_with_time	0.8077
Ours_NoEncoding	0.8003
GRU	0.8009
CNN/LSTM/BiLSTM	0.79-0.80
CNN1D_with_time	0.7821

PR-AUC 对不平衡数据特别重要, 因为 attack 是 minority class。BothEncoding 的 PR-AUC 最高, 说明它不只是某一个 threshold 下表现好, 而是在不同 threshold 下, 对 attack 样本的排序能力也最好。

论文里可以重点写:

> Both positional and temporal encoding improve the precision-recall trade-off for the minority attack class, as shown by the highest PR-AUC.

****AUC****

AUC 最高也是 `Ours\_BothEncoding = 0.9852`。

不过 AUC 差距没有 PR-AUC 那么大。比如:

Ours_BothEncoding	0.9852
Ours_TimeOnly	0.9846
Ours_PositionOnly	0.9846
Ours_NoEncoding	0.9816
CNN1D	0.9809

这说明很多模型都有不错的整体 ranking ability, 但在 minority attack 检测上, PR-AUC 和 confusion matrix 更能拉开差距。

简单说: ****AUC 看起来大家都不差, 但 PR-AUC 和 FN/FP 显示 BothEncoding 更适合实际 NIDS。****

****Confusion Matrix 和 FPR****

从 FPR 看, 最低的是:

GRU:	0.829%
LSTM_with_time:	0.841%
PositionOnly:	0.865%
TimeOnly:	0.882%
BothEncoding:	0.944%

但低 FPR 不等于模型最好。比如 GRU 的 FP 最少, 只有 282, 但 FN=437, 漏掉了最多攻击之一。它是“保守型模型”: 少误报, 但漏报多。

BothEncoding 的 FP=321, 不是最低, 但 FN=308 是最低。它牺牲了少量 FPR, 换来了明显更高的 attack recall。

这可以这样解释:

> Models with the lowest FPR, such as GRU and LSTM_with_time, achieve this partly by missing more attacks. In contrast, BothEncoding accepts a slightly higher FPR but substantially reduces false negatives,

leading to the best Macro-F1 and Macro-Recall.

****Encoding 消融结论****

这组实验最有价值的部分是 encoding ablation:

NoEncoding	Macro-F1 0.8507, PR-AUC 0.8003
PositionOnly	Macro-F1 0.8739, PR-AUC 0.8482
TimeOnly	Macro-F1 0.8818, PR-AUC 0.8510
BothEncoding	Macro-F1 0.8841, PR-AUC 0.8572

可以得到三个结论:

1. 不加 encoding 最差, 说明 packet 顺序/时间信息非常重要。
2. TimeOnly 比 PositionOnly 更强, 说明真实时间间隔比纯 packet index 更有用。
3. BothEncoding 最强, 说明 packet order 和 inter-arrival timing 是互补信息。

****最终论文建议写法****

主模型选择:

Ours_BothEncoding

因为它在最关键的综合指标上最好:

- Macro-F1 最高
- Macro-Recall 最高
- PR-AUC 最高
- AUC 最高
- FN 最低

部署型备选:

Ours_TimeOnly

如果你想强调低误报, 可以说 TimeOnly 有更低 FPR 和更高 Macro-Precision, 但它漏报更多。

论文结论可以写成:

> The BothEncoding variant achieves the best overall performance, with the highest Macro-F1, Macro-Recall, AUROC, and PR-AUC. Although TimeOnly and several recurrent baselines produce slightly lower false positive rates, they do so at the cost of substantially higher false negatives. Therefore, BothEncoding provides the most favorable trade-off for intrusion detection, where reducing missed attacks is particularly important.

用户 2026-07-13 10:41:51

Model Params Time (s) Accuracy F1 Macro F1 Class1 Precision Class1 Recall Class1 ROC-AUC PR-AUC
threshold precision_macro recall_macro precision_weighted recall_weighted f1_weighted
confusion_matrix

Ours_BothEncoding 344963 4337.6 0.9830 0.8882 0.7853 0.7881 0.7825 0.9860 0.8564 0.4550 0.8896
0.8869 0.9830 0.9830 0.9830 [[33707, 296], [306, 1101]]

Ours_TimeOnly 344963 2250.5 0.9822 0.8818 0.7728 0.7818 0.764 0.9846 0.851 0.4785 0.886 0.8776 0.982
0.9822 0.9821 [[33703, 300], [332, 1075]]

Ours_PositionOnly 344963 1994.7 0.9812 0.8739 0.7575 0.7791 0.737 0.9846 0.8482 0.4703 0.8841 0.8642
0.9808 0.9812 0.981 [[33709, 294], [370, 1037]]

LSTM_with_time 217346 3556.3 0.9806 0.8677 0.7455 0.7786 0.715 0.9801 0.8077 0.4638 0.8834 0.8533
0.9799 0.9806 0.9802 [[33717, 286], [401, 1006]]

CNN1D_with_time 70274 1162.4 0.9803 0.867 0.7443 0.768 0.7221 0.9793 0.7821 0.4822 0.8782 0.8565
0.9798 0.9803 0.98 [[33696, 307], [391, 1016]]
GRU 162690 3684.4 0.9797 0.8595 0.7296 0.7748 0.6894 0.9799 0.8009 0.4565 0.881 0.8406 0.9788 0.9797
0.9791 [[33721, 282], [437, 970]]
LSTM 216834 3381.2 0.9792 0.8582 0.7273 0.7575 0.6994 0.9796 0.7976 0.4272 0.8726 0.845 0.9785
0.9792 0.9788 [[33688, 315], [423, 984]]
CNN1D 68354 1591 0.9779 0.8579 0.7274 0.712 0.7434 0.9809 0.7975 0.4203 0.8507 0.8655 0.9783 0.9779
0.9781 [[33580, 423], [361, 1046]]
BiLSTM 564738 2625.7 0.9778 0.8522 0.716 0.7273 0.705 0.9797 0.7903 0.4408 0.8575 0.8471 0.9775
0.9778 0.9776 [[33631, 372], [415, 992]]
Ours_NoEncoding 344963 2213.9 0.977 0.8507 0.7134 0.7069 0.72 0.9816 0.8003 0.4335 0.8477 0.8538
0.9772 0.977 0.9771 [[33583, 420], [394, 1013]]

方案B, max_seq_len=128 and strategy = head

不同模型的运行结果。详细解析结果 (主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics, 并从这些数据进行分析)

助手 2026-07-13 10:43:20

下面按 `max\_seq\_len=128, strategy=head` 的方案 B 来分析。样本结构固定为:

```
Benign = 34003
Attack = 1407
Confusion matrix = [[TN, FP], [FN, TP]]
FPR = FP / (FP + TN)
```

****核心表****

Model	Macro-F1	Macro-Precision	Macro-Recall	ROC-AUC	PR-AUC	FP	FN	TP	FPR
Ours_BothEncoding	0.8882	0.8896	0.8869	0.9860	0.8564	296	306	1101	0.8705%
Ours_TimeOnly	0.8818	0.8860	0.8776	0.9846	0.8510	300	332	1075	0.8823%
Ours_PositionOnly	0.8739	0.8841	0.8642	0.9846	0.8482	294	370	1037	0.8646%
LSTM_with_time	0.8677	0.8834	0.8533	0.9801	0.8077	286	401	1006	0.8411%
CNN1D_with_time	0.8670	0.8782	0.8565	0.9793	0.7821	307	391	1016	0.9029%
GRU	0.8595	0.8810	0.8406	0.9799	0.8009	282	437	970	0.8293%
LSTM	0.8582	0.8726	0.8450	0.9796	0.7976	315	423	984	0.9264%
CNN1D	0.8579	0.8507	0.8655	0.9809	0.7975	423	361	1046	1.2440%
BiLSTM	0.8522	0.8575	0.8471	0.9797	0.7903	372	415	992	1.0940%
Ours_NoEncoding	0.8507	0.8477	0.8538	0.9816	0.8003	420	394	1013	1.2352%

****总判断****

`Ours\_BothEncoding` 是综合最优模型。它在

`Macro-F1`、`Macro-Precision`、`Macro-Recall`、`ROC-AUC`、`PR-AUC` 上全部最高, 同时 FN 最少、TP 最多。虽然它的 FPR 不是最低, 但它在保持 FPR 低于 0.9% 的同时抓到最多攻击样本, 是最适合作为论文主结果的模型。

****Macro-F1****

`Ours\_BothEncoding` = 0.8882, 明显高于第二名 `Ours\_TimeOnly` = 0.8818。相比最强传统序列 baseline `LSTM\_with\_time` = 0.8677, 提升约 +0.0205。

这说明 BothEncoding 不只是提高 attack 类表现，也改善了两类的整体平衡。Macro-F1 对类别不平衡更敏感，因此比 accuracy 更能说明模型对 minority attack class 的实际提升。

****Macro-Recall****

`Ours\_BothEncoding = 0.8869`，最高。Macro-Recall 本质上接近：

$$(TNR + TPR) / 2$$

因为 benign 类数量大，所有模型的 TNR 都很高，真正拉开差距的是 attack recall。BothEncoding 的 attack recall 是 `0.7825`，对应：

```
FN = 306
TP = 1101
```

这是所有模型里漏报最少的。对 NIDS 来说，这点很重要，因为 FN 代表攻击没有被检测出来。

****Macro-Precision****

`Ours\_BothEncoding = 0.8896`，也是最高。注意它不是 FP 最少的模型，但它的 TP 很高，所以 attack precision 仍然最高：

```
Precision Class1 = 0.7881
FP = 296
TP = 1101
```

相比 `PositionOnly`，BothEncoding 只多了 2 个 FP，但多检测出 64 个 TP。因此 precision 和 recall 都更好。

****PR-AUC****

PR-AUC 排名很清楚：

```
BothEncoding 0.8564
TimeOnly     0.8510
PositionOnly 0.8482
LSTM_time    0.8077
GRU          0.8009
NoEncoding   0.8003
```

PR-AUC 在类别不平衡任务里非常重要，因为 attack 是 minority class。BothEncoding 的 PR-AUC 最高，说明它不仅在某个 threshold 下表现好，而且整体上对 attack 样本的排序能力最好。

从消融看：

```
BothEncoding - NoEncoding = +0.0561 PR-AUC
BothEncoding - TimeOnly   = +0.0054 PR-AUC
BothEncoding - PositionOnly = +0.0082 PR-AUC
```

这说明时间编码和位置编码都有贡献，组合后最好。

****ROC-AUC****

`Ours\_BothEncoding = 0.9860`，最高。`TimeOnly` 和 `PositionOnly` 都是 `0.9846`，说明单独使用时间或位置时，ranking 能力已经不错，但组合编码进一步提升。

不过 ROC-AUC 差距没有 PR-AUC 差距大。这是正常的，因为在不平衡数据里 ROC-AUC 有时不够敏感，PR-AUC 更能反映 attack detection 的真实质量。

****Confusion Matrix 和 FPR****

BothEncoding：

```
[[33707, 296],
 [306, 1101]]
```

含义是：

```
TN = 33707
FP = 296
FN = 306
TP = 1101
FPR = 296 / 34003 = 0.8705%
```

它不是 FPR 最低的模型。FPR 最低的是：

```
GRU: 0.8293%
LSTM_with_time: 0.8411%
PositionOnly: 0.8646%
```

但这些模型的 FN 明显更多：

```
GRU FN = 437
LSTM_with_time FN = 401
PositionOnly FN = 370
BothEncoding FN = 306
```

所以 GRU 和 LSTM_with_time 的低 FPR 是通过“少报 attack”换来的，模型更保守，漏报更多。对 NIDS 来说，这种低 FPR 不一定更好。

BothEncoding 的优势是：**FPR 仍然很低，同时 attack recall 最高**。它比 GRU 多 14 个 FP，但少漏报 131 个 attack；这个 trade-off 很划算。

****编码方式消融****

对比四个 ours：

```
BothEncoding > TimeOnly > PositionOnly > NoEncoding
```

BothEncoding 相比 NoEncoding：

```
Macro-F1: 0.8507 -> 0.8882
Macro-Recall: 0.8538 -> 0.8869
Macro-Precision: 0.8477 -> 0.8896
PR-AUC: 0.8003 -> 0.8564
FP: 420 -> 296
FN: 394 -> 306
```

这说明 encoding 不是装饰性模块，而是实质性提升。它同时减少误报和漏报。

TimeOnly 比 PositionOnly 更强：

```
TimeOnly FN = 332
PositionOnly FN = 370
```

说明在你的数据里，inter-arrival time / temporal dynamics 比单纯 packet order 更有信息量。但 BothEncoding 最好，说明 packet order 和 continuous time 是互补的。

****和传统序列模型比较****

BothEncoding 比 LSTM、GRU、CNN1D、BiLSTM 都强。特别是 PR-AUC 差距明显：

```
BothEncoding: 0.8564
LSTM_with_time: 0.8077
CNN1D_with_time: 0.7821
GRU: 0.8009
```

这说明 Transformer-based packet modeling 对 minority attack ranking 更有效。传统 RNN/CNN 即使加入 time feature，也没有达到 BothEncoding 的排序能力和最终分类平衡。

****最终结论****

`Ours\_BothEncoding` 是最好的主模型：

```
Macro-F1
Macro-Precision
```

```
Macro-Recall
ROC-AUC
PR-AUC
TP
FN
FPR    0.9%
```

论文里可以这样写：

> The combined temporal and positional encoding achieves the best overall trade-off. It obtains the highest Macro-F1, Macro-Recall, Macro-Precision, ROC-AUC, and PR-AUC, while also detecting the largest number of attack flows. Although some recurrent baselines produce slightly lower FPR, they do so at the cost of substantially more false negatives. Therefore, the proposed BothEncoding model provides the most balanced and operationally useful detection performance.

用户 2026-07-13 11:12:09

如果写论文，上面这些stage1的结果是否覆盖全面？可以输出完成的stage1的结果了吗？

助手 2026-07-13 11:12:45

可以，**Stage1 的核心结果已经可以输出成论文结果了**。覆盖度已经比较完整，尤其是你现有三组关键证据：

1. **不同模型对比**： Ours vs LSTM/GRU/BiLSTM/CNN1D
2. **encoding ablation**： NoEncoding / PositionOnly / TimeOnly / BothEncoding
3. **max_seq_len ablation**： 8 / 16 / 32 / 64 / 128 / 256, strategy=head

这已经足够支撑 Stage1 的主要结论：**packet-level Transformer + time/position combined encoding 在 intra-flow detection 上优于传统序列模型和单一编码方式**。

但我建议论文里不要说“所有实验彻底完成”，而是说：

> Stage-I core experiments are completed.

也就是 Stage1 主结果可以定稿，但还有几个增强项可以作为 robustness check。

Stage1 可以输出的最终结论

主模型应选：

```
Ours_BothEncoding
max_seq_len = 128
strategy = head
```

因为它在关键指标上最好：

Macro-F1:	0.8882	highest
Macro-Precision:	0.8896	highest
Macro-Recall:	0.8869	highest
ROC-AUC:	0.9860	highest
PR-AUC:	0.8564	highest
F1 Class1:	0.7853	highest
FN:	306	lowest
FPR:	0.8705%	still very low

虽然 GRU / LSTM_with_time 的 FPR 稍低，但它们漏报更多攻击：

```
Ours_BothEncoding FN = 306
GRU FN = 437
LSTM_with_time FN = 401
```

所以它们是“更保守但漏报更多”，不如 BothEncoding 适合 NIDS 主模型。

****Stage1 覆盖是否全面****

基本全面，已经覆盖：

实验目标	是否已覆盖	说明
验证 proposed Stage1 是否优于传统序列模型	已覆盖	CNN1D、LSTM、GRU、BiLSTM 都有
验证 time encoding 是否有效	已覆盖	TimeOnly > PositionOnly > NoEncoding
验证 position + time 是否互补	已覆盖	BothEncoding 最强
验证 sequence length 影响	已覆盖	128 最优，64 recall-oriented
报告 minority attack 指标	已覆盖	PR-AUC、F1 Class1、Recall Class1
报告误报/漏报	已覆盖	confusion matrix 可计算 FPR
报告计算成本	已覆盖	Params、Time 都有

所以从论文 Stage1 章节角度，**可以输出完成版结果表**。

****但如果想让论文更稳，建议补 3 个小项****

不是必须，但很加分：

1. ****multi-seed mean $\pm$ std****

现在看起来像单 seed 结果。论文可以先用当前结果作为 main table，但如果时间允许，最好对主模型和几个关键 baseline 跑 3 seeds：

```
seed = 42, 130, 3407
```

至少跑：

```
Ours_BothEncoding  
Ours_TimeOnly  
Ours_PositionOnly  
Ours_NoEncoding  
LSTM_with_time  
CNN1D_with_time
```

2. ****head vs head-tail strategy****

你的 proposal 里提到 sequence handling 会比较 head、head-tail、sampling。如果最终只报告 head，也可以，但要解释：

> This stage focuses on the head truncation strategy because early packets are expected to contain connection-establishment and attack-initiation patterns.

如果时间够，补一个 `max\_seq\_len=128` 下的 head-tail 对照会更完整。

3. ****和 flow-level baseline 分开放****

LGBM / flow statistical baseline 不应该混进 Stage1 model table 里，而应作为一个单独 baseline section。因为它回答的是另一个问题：

```
flow-level aggregation vs packet-level sequence modeling
```

Stage1 表主要证明 packet sequence encoder 内部设计有效。

****论文里 Stage1 建议输出三张表****

第一张：Stage1 main model comparison

放你刚才这张完整模型对比表，重点 metrics：

```
Model, Params, Time, Macro-F1, Macro-Precision, Macro-Recall, F1 Class1, ROC-AUC, PR-AUC, FPR, Confusion  
Matrix
```


第二张: Encoding ablation

只放:

```
Ours_NoEncoding
Ours_PositionOnly
Ours_TimeOnly
Ours_BothEncoding
```

结论: BothEncoding 最好, TimeOnly 比 PositionOnly 更强。

第三张: Sequence length ablation

只放:

```
8, 16, 32, 64, 128, 256
```

结论: 128 综合最优, 64 recall 最高, 256 开始退化。

****最终判断****

可以输出 Stage1 完成结果。

建议你在论文里这样定性:

> Stage-I experiments are completed and show that the proposed packet-level Transformer with combined positional and temporal encoding achieves the best overall intra-flow detection performance. It provides the highest Macro-F1, Macro-Precision, Macro-Recall, ROC-AUC, and PR-AUC, while maintaining a low false positive rate. These results support the effectiveness of time-aware packet-sequence modeling for flow-level intrusion detection.

换句话说: ****Stage1 已经够写 Results 章节了。接下来重点可以转到 Stage2 或把 Stage1 的 robustness 补漂亮。****

用户 2026-07-13 12:00:45

Full metrics:

```
{'accuracy': 0.9626659135837334, 'macro_precision': 0.74145487832394, 'macro_recall': 0.809354661189905, 'macro_f1': 0.7706508462917445, 'weighted_precision': 0.9680282703050128, 'weighted_recall': 0.9626659135837334, 'weighted_f1': 0.9649653532784647, 'precision_label1': 0.4967628016480283, 'recall_label1': 0.6437833714721587, 'f1_label1': 0.5607973421926911, 'auc': 0.9552327656163553, 'pr_auc': 0.5604476128764463, 'per_class_f1': {'0': 0.9805043503907979, '1': 0.5607973421926911}, 'confusion_matrix': [[33244, 855], [467, 844]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.031577753180528016, 'threshold': 0.42999999999999977}
```

这是stage1中, MLP的运行结果 (flow-level aggregation statistical features)

助手 2026-07-13 12:01:07

这个 MLP flow-level aggregation baseline 结果很关键, 因为它可以作为 ****Stage1 的 “flow-level statistical feature baseline” ****。它不是 packet sequence model, 而是回答:

> 如果只用聚合后的 flow statistical features, 不看 packet sequence, 效果怎样?

****MLP 结果概览****

```
Accuracy:      0.9627
Macro-Precision: 0.7415
Macro-Recall:   0.8094
Macro-F1:       0.7707
F1 Class1:     0.5608
Precision1:     0.4968
```

Recall11: 0.6438
ROC-AUC: 0.9552
PR-AUC: 0.5604
Confusion: [[33244, 855], [467, 844]]
FPR: 855 / (33244 + 855) = 2.51%

****和 Ours_BothEncoding 对比****

Model	Input	Macro-F1	Macro-Precision	Macro-Recall	F1 Class1	Precision1	Recall1	ROC-AUC	PR-AUC	FP	FN	FPR
MLP	flow-level statistical features	0.7707	0.7415	0.8094	0.5608	0.4968	0.6438	0.9552	0.5604	855	467	2.51%
Ours_BothEncoding	packet sequence + time + position	**0.8882**	**0.8896**	**0.8869**	**0.7853**	**0.7881**	**0.7825**	**0.9860**	**0.8564**	**296**	**306**	**0.87%**

****关键提升****

Ours_BothEncoding 相比 MLP:

Macro-F1:	+0.1176
F1 Class1:	+0.2245
Precision1:	+0.2914
Recall11:	+0.1387
ROC-AUC:	+0.0308
PR-AUC:	+0.2960
FP:	855 -> 296, 559
FN:	467 -> 306, 161
FPR:	2.51% -> 0.87%

这个提升非常强，尤其是 ****PR-AUC 和 Precision1****。说明 MLP 虽然能抓到一部分攻击，但误报很高，attack ranking 能力也弱很多。

****怎么解释****

MLP 使用的是 flow-level aggregation statistical features，这类特征已经压缩了 packet order、direction changes、inter-arrival timing 等序列信息。因此它能学习一些总体流量模式，但很难捕捉细粒度时序行为。

从 confusion matrix 看:

MLP FP = 855
Ours FP = 296

MLP 把大量 benign flow 错判成 attack，说明 flow-level aggregation 的判别边界不够干净。

同时:

MLP FN = 467
Ours FN = 306

MLP 也漏掉更多 attack，说明聚合统计特征对某些 attack pattern 不够敏感。

****论文里的定位****

这个 MLP baseline 应该放在 Stage1 结果表里，但最好单独标注 input type:

Flow-level baseline:
MLP on aggregated statistical features

Packet-level sequence models:
CNN1D, LSTM, GRU, BiLSTM, Ours_NoEncoding, Ours_PositionOnly, Ours_TimeOnly, Ours_BothEncoding

这样逻辑更清楚。

****论文结论可以写****

> Compared with the MLP baseline trained on aggregated flow-level statistical features, the proposed packet-sequence model achieves substantially higher Macro-F1, attack-class F1, and PR-AUC, while reducing the false positive rate from 2.51% to 0.87%. This indicates that packet-level sequential modeling provides more discriminative representations than purely aggregated flow statistics for Stage-I intrusion detection.

这个结果很漂亮。它可以直接支撑 RQ1: **packet-sequence modeling 确实比 flow-level aggregation MLP 更有信息量。**

用户 2026-07-13 12:05:15

方案B, 其他配置都一样, max_seq_len=128。只有strategy不一样。

```
strategy = head_tail, max_seq_len = 128 运行结果: [TEST] Full metrics: {'accuracy': 0.9828014685117199,
'macro_precision': 0.8891173273709824, 'macro_recall': 0.8840772149771223, 'macro_f1':
0.8865791606921322, 'weighted_precision': 0.9826923799119973, 'weighted_recall':
0.9828014685117199, 'weighted_f1': 0.9827454138176045, 'precision_label1': 0.787463976945245,
'recall_label1': 0.7768301350390903, 'f1_label1': 0.7821109123434705, 'auc': 0.9858470721917363,
'pr_auc': 0.8566055012828656, 'per_class_f1': {'0': 0.9910474090407938, '1': 0.7821109123434705},
'confusion_matrix': [[33708, 295], [314, 1093]], 'num_classes': 2, 'num_samples': 35410, 'loss':
0.057151494749455245, 'threshold': 0.4600000000000003}

strategy = random, max_seq_len = 128 运行结果: [TEST] Full metrics: {'accuracy': 0.9826885060717312,
'macro_precision': 0.89588888710326, 'macro_recall': 0.8707326003949524, 'macro_f1':
0.8828567888162089, 'weighted_precision': 0.9821960184617861, 'weighted_recall':
0.9826885060717312, 'weighted_f1': 0.982404485462206, 'precision_label1': 0.802130898021309,
'recall_label1': 0.7491115849324804, 'f1_label1': 0.774715178243293, 'auc': 0.9832800090948957, 'pr_auc':
0.8446620469839563, 'per_class_f1': {'0': 0.9909983993891247, '1': 0.774715178243293},
'confusion_matrix': [[33743, 260], [353, 1054]], 'num_classes': 2, 'num_samples': 35410, 'loss':
0.06069993708511337, 'threshold': 0.4740000000000003}

strategy = head, max_seq_len = 128 运行结果: [TEST] Full metrics: {'accuracy': 0.9829991527817001,
'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1':
0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall':
0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607,
'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804,
'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606},
'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss':
0.05759464621893551, 'threshold': 0.4550000000000003}
```

不同配置下的运行结果。详细解析结果 (主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics, 并从这些数据进行分析)

助手 2026-07-13 12:05:56

结论: **在`max\_seq\_len=128`下, `head`是综合最优; `head\_tail`几乎一样稳, 但略低; `random`误报最低、precision最高, 但漏报明显变多, 不适合作为主配置。**

这里总样本固定:

```
Benign = 34003
Attack = 1407
FPR = FP / (FP + TN)
```

	Strategy	Macro-F1	Macro-Precision	Macro-Recall	PR-AUC	AUC	TN	FP	FN	TP	FPR
head	0.8882	0.8896	0.8869	0.8564	0.9860	33707	296	306	1101	0.8705%	
head_tail	0.8866	0.8891	0.8841	0.8566	0.9858	33708	295	314	1093	0.8676%	
random	0.8829	0.8959	0.8707	0.8447	0.9833	33743	260	353	1054	0.7646%	

Macro-F1

`head` 最高:

```
head      0.8882
head_tail 0.8866
random    0.8829
```

这说明 `head` 在 benign 和 attack 两类之间最平衡。`head\_tail` 只低 `0.0016`，差距非常小，可以认为性能接近。`random` 明显低一些，主要是 attack recall 掉得比较多。

Macro-Recall

`head` 最高，说明它整体漏报最少:

```
head      0.8869
head_tail 0.8841
random    0.8707
```

从 confusion matrix 看更直观:

```
head FN = 306
head_tail FN = 314
random FN = 353
```

`random` 少了误报，但多漏了 47 个 attack，相比 `head` 明显更保守。对 NIDS 来说，漏报 attack 通常比多一点误报更危险，所以 `random` 不适合作为主策略。

Macro-Precision

`random` 最高:

```
random    0.8959
head      0.8896
head_tail 0.8891
```

原因是 `random` 的 FP 最少:

```
random FP = 260
head_tail FP = 295
head FP = 296
```

但这个高 precision 是用 recall 换来的。`random` 的 attack recall 只有 `0.7491`，比 `head` 的 `0.7825` 低很多。所以它更像是“少报警，但漏掉更多攻击”的策略。

PR-AUC

PR-AUC 排名:

```
head_tail 0.8566
head      0.8564
random    0.8447
```

`head\_tail` 比 `head` 高 `0.0002`，这个差距非常小，基本可以认为两者 ranking ability 接近。`random` 明显低，说明随机采样破坏了一部分稳定的 attack ranking 信息。

AUC

AUC 排名:

```
head      0.9860
head_tail 0.9858
random    0.9833
```

`head` 最高, `head\_tail` 非常接近, `random` 明显落后。说明前 128 个 packet 的顺序信息对模型区分 benign/attack 很有帮助, 随机抽样会削弱这种时序结构。

****Confusion Matrix 和 FPR****

`head`:

```
[[33707, 296],
 [306, 1101]]
FPR = 0.8705%
```

`head\_tail`:

```
[[33708, 295],
 [314, 1093]]
FPR = 0.8676%
```

`random`:

```
[[33743, 260],
 [353, 1054]]
FPR = 0.7646%
```

`head\_tail` 和 `head` 的误报几乎一样, `head\_tail` 只少 1 个 FP, 但多 8 个 FN。因此 `head` 更好。

`random` 的 FPR 最低, 但 FN 增加到 353, 说明它为了减少 false alarm, 牺牲了 attack detection。这个 trade-off 不如 `head`。

****为什么 head 最好****

在你的数据里, 攻击相关模式很可能集中在 flow 的早期 packet, 例如连接建立、初始 payload size、方向变化、早期 burst、inter-arrival timing 等。因此保留前 128 个 packet 比随机抽样更稳定。

`head\_tail` 加入尾部 packet

后没有明显提升, 说明尾部信息对当前任务贡献有限, 甚至略微稀释了早期攻击信号。

`random` 打乱了固定的早期结构采样, 虽然可能减少一些误报, 但会丢掉连续时间模式, 所以 recall、PR-AUC、AUC 都下降。

****论文结论可以这样写****

> For sequence truncation strategy, the head strategy achieves the best overall performance, obtaining the highest Macro-F1, Macro-Recall, and ROC-AUC. The head-tail strategy performs very similarly but does not improve over head-only truncation, suggesting that the tail packets add limited additional discriminative information. Random sampling achieves the lowest FPR and highest Macro-Precision, but at the cost of substantially higher false negatives and lower PR-AUC, indicating weaker attack detection capability.

最终建议:

```
strategy = head
head_tail
random
```

一句话: ****head 在误报仍然很低的情况下抓到最多 attack, 是最适合 Stage1 主结果的 strategy.****

用户 2026-07-13 12:09:38

我现在stage1的结果是否已经大部分都覆盖了?

助手 2026-07-13 12:09:59

是的，**Stage 1 现在已经大部分覆盖了，而且已经足够支撑论文里的 Stage-I Results 主章节**。你现在不是“结果还不够”，而是需要把这些结果组织成清楚的证据链。

你已经覆盖了这几类关键实验：

目的	当前是否覆盖	评价
Flow-level aggregation baseline	已覆盖	MLP flow statistical features, 能支撑 RQ1
Packet sequence baseline	已覆盖	CNN1D、LSTM、GRU、BiLSTM
Time-aware sequence baseline	已覆盖	CNN1D_with_time、LSTM_with_time
Proposed model 消融	已覆盖	NoEncoding、PositionOnly、TimeOnly、BothEncoding
Sequence length sensitivity	已覆盖	8、16、32、64、128、256
Truncation/sampling strategy	已覆盖	head、head_tail、random
Minority attack metrics	已覆盖	F1 Class1、Precision1、Recall1、PR-AUC
Operational metrics	已覆盖	confusion matrix、FPR、FP/FN
Efficiency	已覆盖	Params、Time

所以 Stage1 的覆盖度可以说是 **80%-90% 完成**。现在已经能写出一个很完整的 Stage1 结果章节。

Stage1 当前可以形成的主结论

1. **Packet-level sequence modeling 明显优于 flow-level MLP aggregation**

MLP flow-level baseline:

```
Macro-F1 = 0.7707
PR-AUC = 0.5604
FPR = 2.51%
```

Ours_BothEncoding:

```
Macro-F1 = 0.8882
PR-AUC = 0.8564
FPR = 0.87%
```

这可以支撑 RQ1: packet sequence representation 比简单 flow-level aggregation 更有判别力。

2. **BothEncoding 是 Stage1 最优配置**

BothEncoding 同时取得最高:

```
Macro-F1
Macro-Precision
Macro-Recall
ROC-AUC
PR-AUC
F1 Class1
```

并且 FN 最少，说明 position + time 是互补的。

3. **TimeOnly 比 PositionOnly 更强**

说明 continuous timing / inter-arrival time 对你的任务很重要。这正好呼应 proposal 里的 time-aware modeling。

4. **max_seq_len=128 是最佳主配置**

在 head strategy 下，128 的综合指标最好；64 更偏 recall，32 的 PR-AUC 也强，但 128 最均衡。

5. ****head strategy 最适合作为主策略****

`head` 的 Macro-F1、Macro-Recall、AUC 最好；`random` FPR 最低但漏报更多；`head\_tail` 接近 head 但略低。

****还缺什么？不是必须，但加分****

如果时间紧，Stage1 可以结束。

如果想让论文更稳，再补这 3 个小题：

1. ****Multi-seed mean $\pm$ std****

当前结果更像单次运行。论文答辩时可能会问稳定性。建议至少对主模型和关键 baseline 跑 3 个 seed。

最小集合：

```
Ours_BothEncoding
Ours_TimeOnly
Ours_NoEncoding
MLP_flow-level
LSTM_with_time
CNN1D_with_time
```

2. ****统计显著性或稳定性说明****

不一定要复杂，可以写：

```
We report the main results under a fixed split and seed. Multi-seed validation is left as robustness analysis.
```

但如果能补 seed，会更强。

3. ****和强 flow baseline 分开讨论****

你之前的 LGBM flow baseline 太强，不能和 MLP 混为一谈。论文里要分清：

- MLP flow baseline：证明简单 aggregation 不够
- LGBM / tree-based flow baseline：作为强 tabular baseline，可能很强，需要单独讨论是否存在 shortcut/leakage/split bias

****最终判断****

可以这样定：

```
Stage1 core experiments: completed
Stage1 robustness experiments: optional but recommended
```

你现在可以开始输出 Stage1 的正式结果表了。建议最终 Stage1 放三张表：

1. ****Main comparison table****：MLP、CNN1D、LSTM、GRU、BiLSTM、Ours variants
2. ****Encoding ablation table****：NoEncoding、PositionOnly、TimeOnly、BothEncoding
3. ****Sequence handling table****：seq_len ablation + strategy ablation

一句话：****Stage1 已经大部分覆盖，而且可以写论文结果了；接下来重点是整理表格、写清楚结论，并决定是否补 multi-seed。****

用户 2026-07-13 13:04:13

```
方案A运行结果: "test_metrics": {
"accuracy": 0.9817283253318272,
"macro_precision": 0.891903312957051,
"macro_recall": 0.8600128012451596,
```



```

"macro_f1": 0.8752142703953976,
"weighted_precision": 0.9810772212036308,
"weighted_recall": 0.9817283253318272,
"weighted_f1": 0.9813408831023472,
"precision_label1": 0.7950310559006211,
"recall_label1": 0.7277896233120114,
"f1_label1": 0.759925788497217,
"auc": 0.9800628507610464,
"pr_auc": 0.8326108220868012,
"per_class_f1": {
"0": 0.990502752293578,
"1": 0.759925788497217
},
"confusion_matrix": [
[
33739,
264
],
[
383,
1024
]
],
"num_classes": 2,
"num_samples": 35410,
"loss": 0.11201799211734469,
"threshold": 0.7150000000000005
}

```

方案B运行结果: Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.4550000000000003}

方案C运行结果: "test_metrics": {
"accuracy": 0.9834202417634511,
"macro_precision": 0.8989505209974933,
"macro_recall": 0.8918983147078002,
"macro_f1": 0.8953896222507791,
"weighted_precision": 0.9832784957495156,
"weighted_recall": 0.9834202417634511,
"weighted_f1": 0.9833463190159758,
"precision_label1": 0.8069464544138929,
"recall_label1": 0.7920454545454545,
"f1_label1": 0.7994265232974911,
"auc": 0.9890371580790593,

```

"pr_auc": 0.8762558340285889,
"per_class_f1": {
  "0": 0.9913527212040671,
  "1": 0.7994265232974911
},
"confusion_matrix": [
  [
    80193,
    667
  ],
  [
    732,
    2788
  ]
],
"num_classes": 2,
"num_samples": 84380,
"loss": 0.06821018314045486,
"threshold": 0.6840000000000005
}

```

不同配置下的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-13 13:04:55

先说最重要的一点：**方案 A 和方案 B 可以直接比较，因为 test samples 都是 35,410，confusion matrix 里的 benign/attack 数量一致；方案 C 的 test samples 是 84,380，测试集大小和类别数量不同，所以不能和 A/B 做完全严格的横向比较。**

但从指标上看，**C 是目前最强结果；A 偏 conservative，误报少但漏报多；B 比 A 更适合检测任务，因为用很小的 FPR 代价换来了明显更高的 recall 和 macro-F1。**

****核心指标表****

Config	Samples	Macro-F1	Macro-Precision	Macro-Recall	ROC-AUC	PR-AUC	TN	FP	FN	TP	FPR
A	35,410	0.8752	0.8919	0.8600	0.9801	0.8326	33,739	264	383	1,024	**0.7764%**b>
B	35,410	0.8882	0.8896	0.8869	0.9860	0.8564	33,707	296	306	1,101	0.8705%
C	84,380	**0.8954**	**0.8990**	**0.8919**	**0.9890**	**0.8763**	80,193	667	732	2,788	0.8249%

****A vs B: B 更适合作为检测模型****

A 的特点是 **precision 高、FPR 最低**：

```

FP = 264
FPR = 0.7764%
Precision1 = 0.7950

```

但代价是漏报较多：

```

FN = 383

```

```
Recall1 = 0.7278
```

B 相比 A:

```
Macro-F1:      +0.0130
Macro-Recall:  +0.0269
PR-AUC:        +0.0238
ROC-AUC:       +0.0059
FN:            383 -> 306,      77  attack
TP:            1024 -> 1101,     77  attack
FP:            264 -> 296,      32  benign
FPR:           0.7764% -> 0.8705%
```

这说明 B 是更好的 NIDS trade-off: **只增加 32 个误报, 但减少 77 个漏报**。在入侵检测里, 漏报攻击通常比多一点误报更严重, 所以 B 明显比 A 更合理。

A 的 threshold 是 `0.715`, B 是 `0.455`。A 的阈值高很多, 所以它更保守: 少报警, 但漏掉更多攻击。

C 的表现最强, 但要注意测试集不同

C 的所有主指标都最高:

```
Macro-F1 = 0.8954
Macro-Precision = 0.8990
Macro-Recall = 0.8919
ROC-AUC = 0.9890
PR-AUC = 0.8763
F1 Class1 = 0.7994
Precision1 = 0.8069
Recall1 = 0.7920
```

confusion matrix:

```
[[80193, 667],
 [732, 2788]]
```

FPR:

```
667 / (80193 + 667) = 0.8249%
```

这个 FPR 比 B 还低, 同时 recall 比 B 更高:

```
B Recall1 = 0.7825
C Recall1 = 0.7920
```

如果 C 和 A/B 是同一数据分布、同一实验协议下的结果, 那么 C 就是目前最佳配置: **更高 PR-AUC、更高 AUC、更高 macro-F1, 并且 FPR 仍然低于 B**。

但是 C 的测试集是:

```
C: 84380 samples
A/B: 35410 samples
```

所以论文里不能直接写 “C beats B” 除非你说明为什么 test set 不一样, 或者重新用同一个 split 跑 A/B/C。

PR-AUC 分析

PR-AUC 排名:

```
C: 0.8763
B: 0.8564
A: 0.8326
```

PR-AUC 是不平衡分类里非常关键的指标。C 最高, 说明它对 attack class 的排序能力最好。B 明显强于 A, 说明 B 不只是 threshold 调得更激进, 而是整体 attack ranking 更好。

AUC 分析

ROC-AUC:

C: 0.9890
B: 0.9860
A: 0.9801

AUC 越高, 说明模型区分 benign 和 attack 的总体能力越强。B 明显优于 A, C 最强。

Macro 指标分析

Macro-F1:

C > B > A

Macro-Recall:

C > B > A

Macro-Precision:

C > A > B

A 的 macro precision 比 B 稍高, 是因为 A 更保守, 误报少。但 B 的 macro recall 明显更好, 所以 B 的 macro-F1 更高。这个说明: **A 更像低误报模型, B 更像平衡检测模型。**

论文建议

如果写论文, 建议这样处理:

1. **A/B 放在同一张可比表里**

因为它们测试集一致。结论: B 优于 A, 尤其是 Macro-F1、Macro-Recall、PR-AUC、AUC。

2. **C 单独作为扩展实验或最终增强配置**

但要注明 test set 不同。最好补一句:

> Since Configuration C was evaluated on a larger test set, it is reported separately and should not be interpreted as a strictly paired comparison with A and B.

3. **如果要正式宣称 C 最优**

最稳做法是: 用 C 的同一 split 重新跑 A 和 B, 或者用 A/B 的 35,410 test set 重新跑 C。

最终结论

- **A**: FPR 最低, 误报少, 但漏报多, 偏保守。
- **B**: 比 A 更均衡, 明显提升 recall、macro-F1、PR-AUC, 是 A/B 中更好的主配置。
- **C**: 指标最强, 尤其 PR-AUC、AUC、macro-F1 都最好, 但由于 test set 不同, 需要谨慎比较。

如果只能选当前论文主结果: **B** 可以作为严格可比主结果; C

可以作为最强配置结果, 但需要说明测试集差异或补同 split 实验。

用户 2026-07-13 13:14:02

如果要完成论文, 我stage2需要提供那些结果? 详细解析

助手 2026-07-13 13:14:45

要完成论文, **Stage2** 需要回答 RQ2:

> 加入 cross-flow context 是否能提升 distributed / multi-flow attack detection, 相比只做 independent per-flow classification 是否更好?

所以 Stage2 不只是“再跑一个模型”, 而是要证明或反驳: **跨流上下文有没有带来额外信息**。

****最低必须提供的 Stage2 结果****

你至少需要 5 类结果。

****1. Stage1 vs Stage2 主结果对比****

这是最核心的一张表。必须用同一个 dataset、同一个 train/val/test split、同一个 Stage1 embedding。

表格建议:

Model	Context	Macro-F1	Macro-Precision	Macro-Recall	F1 Class1	Precision1	Recall1	ROC-AUC	PR-AUC	FPR	FP	FN
---	---	---	---	---	---	---	---	---	---	---	---	---

至少包含:

- Stage1 only / no context
- Stage2 time-only context
- Stage2 same source context
- Stage2 same destination context
- Stage2 same endpoint context
- Stage2 source-or-destination context

这张表回答:

Stage2	Stage1
FN	FP
PR-AUC	
FPR	

如果 Stage2 没有超过 Stage1, 也可以写论文, 但要诚实说明: **cross-flow context did not consistently improve overall detection**。

****2. Context construction ablation****

Stage2 最重要的变量不是模型, 而是 context 怎么构造。你需要比较:

- time-only
- same source
- same destination
- same endpoint
- same source or destination

原因:

- `time-only`: 只看时间邻近流, 可能引入很多无关流。
- `same source`: 适合扫描、DDoS source behavior、host-level malicious activity。
- `same destination`: 适合 victim-centered attacks。
- `same endpoint`: 更宽泛, 覆盖 source/destination 任意一端相关。
- `source-or-destination`: context 最多, 但可能噪声也最多。

你需要报告每种 context 的:

- average context length
- coverage
- Macro-F1
- Macro-Recall
- PR-AUC
- FPR

FN / FP

如果某种 context 性能差，可以解释为：context 太稀疏或噪声太多。

3. Window size / K ablation

Stage2 必须说明 context window 多大合适。建议至少跑：

K = 5, 10, 20/32, 64, 128

如果时间不够，最小集合：

K = 10, 32, 64

你要观察：

- K 太小：上下文不足，提升有限。
- K 适中：可能最好。
- K 太大：噪声增加，FPR 上升或 PR-AUC 下降。

论文里可以写：

> Performance does not monotonically improve with larger context windows, indicating that context relevance is more important than context quantity.

4. Stage2 model ablation

你 proposal 里提到 second-stage Transformer / LSTM。至少需要比较：

Stage2 Transformer
Stage2 LSTM GRU
Stage2 pooling / attention baseline
Stage1 only baseline

目的不是证明 Transformer 一定最好，而是说明：

cross-flow context

如果 Transformer 没有比 LSTM 好，也没关系，可以说：

> The benefit of Stage2 is limited by context quality rather than sequence modeling capacity.

5. Error analysis / case study

Stage2 很容易整体指标不明显，但对某些样本有帮助。所以你需要做一个小的错误分析：

比较 Stage1 和 Stage2：

Stage1 FN but Stage2 TP Stage2
Stage1 TP but Stage2 FN Stage2
Stage1 FP but Stage2 TN Stage2
Stage1 TN but Stage2 FP Stage2

最有价值的是：

Stage1 missed, Stage2 detected

你可以分析这些 flow 是否具有：

- same source repeated behavior
- same destination burst
- nearby attack cluster
- multi-flow pattern
- unusual temporal density

这部分可以支撑论文讨论，即使整体提升小，也能解释 context 的局部价值。

****强烈建议额外提供的结果****

这些不是绝对必须，但会让论文稳很多。

****6. Context availability/statistical validation****

你之前已经有一些 context analysis，这个可以放进论文。需要回答：

context signal

例如：

Context	K	Coverage	Attack context ratio	Benign context ratio	AUC as single feature

如果 same-source context 的 single-feature AUC 明显高于 time-only，就能解释为什么某些 Stage2 context 更有效。

****7. Split leakage / causality check****

Stage2 特别容易被质疑数据泄漏。你必须说明：

```
context      split
train context    val/test flows
test context     train labels
```

另外还要说明是否使用未来流：

- 如果 context 使用前后窗口，即 past + future flows，这是 offline detection。
- 如果你想贴近 real-time NIDS，应提供 past-only context 或至少声明 limitation。

论文里一定要写清楚：

> To avoid cross-split leakage, context windows are constructed within each split independently.

这个非常重要。

****8. Efficiency / cost****

Stage2 会增加计算成本，所以需要报告：

```
Params
training time
inference time
average context length
memory usage if available
```

尤其如果 Stage2 指标只提升一点点，就必须讨论 trade-off：

****Stage2 最终结果应该怎么组织****

论文里建议放 4 张表 + 1 个图。

****Table 1: Stage1 vs Stage2 main comparison****

包含 Stage1 only 和最好的 Stage2 配置。

****Table 2: Context type ablation****

比较 time/source/destination/endpoint/source-or-destination。

****Table 3: Context window size ablation****

比较 K=10/32/64/128。

****Table 4: Stage2 model ablation****

比较 Transformer / LSTM / pooling baseline。

****Figure: Stage1 vs Stage2 confusion matrix 或 FP/FN change****

重点展示：

FN
FPR
PR-AUC

****如果 Stage2 结果不如 Stage1，论文还能写吗？****

可以。你可以把结论写成更成熟的版本：

> Stage2 cross-flow modeling does not consistently improve overall detection over the strong Stage1 packet-sequence encoder. However, context-based analysis shows that certain relational contexts, especially source- or endpoint-based neighborhoods, contain useful attack-related signals. The limited improvement suggests that context quality and leakage-free construction are more important than simply adding a second-stage sequence model.

这不是失败，反而是很好的研究发现。

****最终最低完成标准****

如果你时间紧，Stage2 至少完成这些：

1. Stage1 only baseline
2. Stage2 best Transformer result
3. context type ablation: time/source/destination/endpoint
4. K ablation: 10/32/64
5. confusion matrix + FPR + PR-AUC + Macro-F1
6. error analysis
7. split leakage

一句话：****Stage2 不需要跑无穷多实验，但必须能清楚回答 “cross-flow context 有没有帮助、什么时候帮助、代价是什么、为什么” 。****

用户 2026-07-13 13:19:57

Params

training time

inference time

我现在的s2的代码中有没有统计这个信息？如果没有，我该加在哪里？

助手 2026-07-13 13:20:10

我先看一下`s2`的训练入口、trainer 和 metrics

保存逻辑，确认现在到底有没有记录参数量、训练耗时、推理耗时，以及结果 JSON 写在哪里。

助手 2026-07-13 13:20:29

初步看下来：主训练路径`run\_stage2.py` + `stage2/trainer.py` 主要保存 metrics 和 predictions；调参脚本` tune\_transformer\_hyperparams.py` 有记录` num\_params` 和` train\_time\_seconds`，但这不等于正式 Stage2 run 都会保存。我要再看具体写 JSON 的位置，给你精确到函数和插入点。

助手 2026-07-13 13:20:40

我看到正式输出位置了：`final\_evaluate\_and\_save()` 会生成 `stage2\_metrics.json`，`\_save\_summary()` 会生成 txt summary；现在这两个位置没有把 params/training time/inference time 写进去。训练时间目前只在另一个调参脚本里做了，不在主 Stage2 跑法里。

助手 2026-07-13 13:21:33

你现在的 **主 Stage2 代码里基本没有正式统计这三项**。

准确说：

- `Params`：调参脚本里有统计，但主运行结果 `stage2\_metrics.json` 里没有。
- `training time`：`tune\_transformer\_hyperparams.py` 里有统计 trial 的训练时间，但 `run\_stage2.py` 主流程没有保存。
- `inference time`：主代码里没有正式统计。

已有位置：

- [tune_transformer_hyperparams.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_transformer_hyperparams.py:393) 有 `n\_params = sum(p.numel() for p in model.parameters())`
- [tune_transformer_hyperparams.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_transformer_hyperparams.py:407) 有 `start\_time = time.time()` 包住 `trainer.fit()`
- 但正式主流程 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:465) 只 build model、train、evaluate，没有保存 runtime 信息
- 正式输出在 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:607) 的 `final\_evaluate\_and\_save()`，最后写入 [stage2_metrics.json](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:691)

****建议加在哪里****

最推荐加在 `s2/stage2/trainer.py`，因为这样所有 Stage2 入口都会受益，不只 `run\_stage2.py`。

你需要改 3 个地方。

****1. 在 trainer.py 顶部加 time****

位置：[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:3)

```
import time
```

****2. 在 Stage2Trainer.__init__ 里记录参数量****

位置：[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:314)

```
self.num_params = sum(p.numel() for p in self.model.parameters())
self.num_trainable_params = sum(p.numel() for p in self.model.parameters() if p.requires_grad)
self.train_time_seconds = None
```

****3. 在 fit() 里统计训练时间****

位置：[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:350)

在 `fit()` 开始处加：

```
fit_start = time.perf_counter()
```

在训练循环结束后加：

```
self.train_time_seconds = time.perf_counter() - fit_start
print(f"[INFO] Stage2 training time: {self.train_time_seconds:.2f}s")
```

注意要放在`for epoch ...`循环之后，这样 early stopping 之后也能记录实际训练耗时。

****4. 在 final_evaluate_and_save() 里统计 inference time****

位置: [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:671)

把现在的:

```
metrics, pred_df = evaluate(...)
```

外面包一层计时:

```
if self.device.type == "cuda":
    torch.cuda.synchronize()

infer_start = time.perf_counter()

metrics, pred_df = evaluate(
    model=self.model,
    loader=loader,
    loss_fn=self.loss_fn,
    device=self.device,
    threshold=threshold,
)

if self.device.type == "cuda":
    torch.cuda.synchronize()

infer_time = time.perf_counter() - infer_start
metrics["inference_time_seconds"] = float(infer_time)
metrics["inference_ms_per_sample"] = float(infer_time / max(metrics["num_samples"], 1) * 1000.0)
```

这样每个 split 都会有:

```
train.inference_time_seconds
val.inference_time_seconds
test.inference_time_seconds
test.inference_ms_per_sample
```

论文里通常用 test split 的 inference time。

****5. 在 final_metrics 里保存 params 和 training time****

位置: [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:640)

在`final\_metrics`里加:

```
"model_info": {
    "num_params": int(self.num_params),
    "num_trainable_params": int(self.num_trainable_params),
},
"runtime": {
    "train_time_seconds": None if self.train_time_seconds is None else float(self.train_time_seconds),
},
```

最后`stage2\_metrics.json`就会包含:

```
{
  "model_info": {
    "num_params": 123456,
    "num_trainable_params": 123456
  },
  "runtime": {
    "train_time_seconds": 1234.56
  },
  "splits": {
    "test": {
      "inference_time_seconds": 1.23,
      "inference_ms_per_sample": 0.16
    }
  }
}
```

```
}  
}  
}
```

****论文里怎么报****

建议 Stage2 表格里放:

```
Params  
Training Time (s)  
Test Inference Time (s)  
Inference ms/flow  
Macro-F1  
Macro-Recall  
PR-AUC  
FPR
```

最重要的是 `Inference ms/flow`，因为不同 split 样本数可能不同，单纯 `inference\_time\_seconds` 不太公平。

用户 2026-07-13 13:56:42

Params

training time

inference time

我现在的s2的代码中有没有统计这个信息？如果没有，我该加在哪里？请帮我修正代码

助手 2026-07-13 13:56:53

我先看一下 `s2` 的训练入口、trainer 和 metrics 保存逻辑，确认现在有没有已经记录 `Params / training time / inference time`。如果缺，我会把统计放在最接近现有 `stage2\_metrics.json` / summary 输出的位置，这样后续论文表格可以直接读。

助手 2026-07-13 13:57:05

我看到 `stage2\_metrics.json` 现在主要保存的是 train/val/test 的分类指标，`verify\_reproducibility.py` 里有参数统计的零散逻辑，但正式训练路径 `run\_stage2.py -> stage2/trainer.py` 没有把参数量、训练耗时、推理耗时写进结果。接下来我会改 `trainer.py`，让所有正常 Stage2 训练自动记录这些字段。

助手 2026-07-13 13:57:16

`trainer.py` 里 `fit()` 负责训练循环，`final\_evaluate\_and\_save()` 负责最终评估和写 `stage2\_metrics.json`，这是最合适的修改点。我会加一个轻量 profiling：参数量在 trainer 初始化时计算，训练耗时在 `fit()` 里计时，最终评估时分别统计 train/val/test 推理时间并写入 JSON 和 txt summary。

助手 2026-07-13 13:57:31

我会把字段命名做得论文友好一些：`params\_total`、`params\_trainable`、`training\_time\_seconds`、`inference\_time\_seconds\_test`、`test\_samples\_per\_second`。同时保留更完整的 split 级 profiling，方便你后面汇总表格。

助手 2026-07-13 13:58:10

训练部分已经会统计参数量和训练总耗时了。现在我把最终评估阶段的`evaluate()`包一层计时，按 train/val/test 分别记录推理时间，并写入`stage2\_metrics.json`和`stage2\_run\_summary.txt`。

助手 2026-07-13 13:58:27

现在 JSON 里会有`model\_profile`、`training\_profile`、`inference\_profile`和一个扁平的`efficiency`汇总块。接下来我会把 txt summary 也加上这些字段，方便你不用打开 JSON 就能抄论文表格。

助手 2026-07-13 13:58:48

代码已经改好，下一步我会做语法检查，确保`trainer.py`能正常 import/compile。之后我会告诉你具体输出字段在哪里，以及旧结果是否需要重跑才能生成这些新字段。

助手 2026-07-13 13:58:59

默认`python`命令在这个 PowerShell 环境里不可用，所以我会用 Codex bundled Python 路径来做同样的语法检查。刚才的失败只是找不到`python`命令，不是代码语法错误。

助手 2026-07-13 13:59:09

语法检查通过了。现在我再快速确认修改后的输出结构，确保`efficiency`块会在最终 JSON 里包含你要的三个论文字段。

助手 2026-07-13 13:59:23

还有一个小收尾：`fit()`之前虽然标注返回`Dict`，但实际没有返回值。我顺手让它返回`training\_profile`，这样如果以后别的脚本调用`trainer.fit()`，也能直接拿到训练耗时。

助手 2026-07-13 13:59:29

返回值也补上了。我会再跑一次语法检查，确认最终状态干净。

助手 2026-07-13 13:59:50

已帮你修好了。原来的正式`s2`训练路径里**没有稳定输出 Params / training time / inference time**，只有分类 metrics；现在我已经加到 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:21) 里了，并通过了语法检查。

新增输出位置：

`stage2\_metrics.json`里会多出：

```
"model_profile": {
  "params_total": ...,
  "params_trainable": ...,
  "params_non_trainable": ...,
  "estimated_fp32_size_mb": ...
},
"training_profile": {
  "training_time_seconds": ...,
  "epochs_completed": ...,
  "best_epoch": ...,
  "avg_seconds_per_epoch": ...
```

```

    },
    "inference_profile": {
      "splits": {
        "train": {...},
        "val": {...},
        "test": {
          "inference_time_seconds": ...,
          "num_samples": ...,
          "num_batches": ...,
          "seconds_per_sample": ...,
          "samples_per_second": ...
        }
      }
    },
    "efficiency": {
      "params_total": ...,
      "params_trainable": ...,
      "training_time_seconds": ...,
      "inference_time_seconds_test": ...,
      "test_seconds_per_sample": ...,
      "test_samples_per_second": ...
    }
  }
}

```

`stage2\_run\_summary.txt` 也会同步写出这些字段，方便你直接整理论文表格。

说明一下口径：`training\_time\_seconds` 是整个 `fit()` 的 wall-clock 时间，包含每个 epoch 的训练、validation evaluation、threshold search 和 early stopping 过程；`inference\_time\_seconds\_test` 是最终 test split 的评估耗时，包含 dataloader 遍历、model forward、loss/metrics/prediction dataframe 生成，不包含保存 CSV 文件的时间。

旧的 Stage2 结果文件不会自动出现这些字段，需要用新代码重新跑一次 Stage2。之后论文表里的 `Params / Training Time / Inference Time` 就可以直接从 `stage2\_metrics.json -> efficiency` 读取。

用户 2026-07-13 14:07:42

Stage2,其余配置都一样，只是context:下的method: 不一样

```

method = srouce_host 运行结果: test: loss=0.019559, acc=0.988552, macro_precision=0.932788,
macro_recall=0.922427, macro_f1=0.927539, weighted_precision=0.988425, weighted_recall=0.988552,
weighted_f1=0.988482, auc=0.993827886090999, pr_auc=0.934499456433569,
precision_label1=0.872086247086247, recall_label1=0.850284090909091,
f1_label1=0.8610471806674338, cm=[[80421, 439], [527, 2993]]
method = destination_host 运行结果: test: loss=0.020364, acc=0.987106, macro_precision=0.918267,
macro_recall=0.920994, macro_f1=0.919625, weighted_precision=0.987149, weighted_recall=0.987106,
weighted_f1=0.987127, auc=0.9935290144441571, pr_auc=0.9261714364906419,
precision_label1=0.8431151241534989, recall_label1=0.8488636363636364,
f1_label1=0.8459796149490374, cm=[[80304, 556], [532, 2988]]
method = time_only 运行结果: test: loss=0.020335, acc=0.987461, macro_precision=0.928200,
macro_recall=0.912348, macro_f1=0.920109, weighted_precision=0.987254, weighted_recall=0.987461,
weighted_f1=0.987343, auc=0.9922409629157017, pr_auc=0.9215432131808575,
precision_label1=0.8637706855791962, recall_label1=0.8303977272727273,
f1_label1=0.84675550405562, cm=[[80399, 461], [597, 2923]]
method = endpoint 运行结果: test: loss=0.018487, acc=0.986928, macro_precision=0.905650,
macro_recall=0.938699, macro_f1=0.921457, weighted_precision=0.987566, weighted_recall=0.986928,
weighted_f1=0.987184, auc=0.9935241080262182, pr_auc=0.9287117262713892,
precision_label1=0.8162784611358284, recall_label1=0.8860795454545455,
f1_label1=0.8497479907369568, cm=[[80158, 702], [401, 3119]]

```

不同配置下的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-13 14:08:25

结论: `**`source_host`` 是当前 Stage2 最好的主配置; ``endpoint`` 是 recall-oriented 配置; ``time_only`` 和 ``destination_host`` 整体弱一些。 **

这里四组结果的 test set 一样:

```
Benign = 80860
Attack = 3520
Total = 84380
FPR = FP / (FP + TN)
```

****核心表****

Method	Macro-F1	Macro-Precision	Macro-Recall	ROC-AUC	PR-AUC	TN	FP	FN	TP	FPR
source_host	**0.9275**	**0.9328**	0.9224	**0.9938**	**0.9345**	**80421**	**439**	527	2993	**0.5429%**
destination_host	0.9196	0.9183	0.9210	0.9935	0.9262	80304	556	532	2988	0.6876%
time_only	0.9201	0.9282	0.9123	0.9922	0.9215	80399	461	597	2923	0.5701%
endpoint	0.9215	0.9057	**0.9387**	0.9935	0.9287	80158	702	**401**	**3119**	0.8682%

****Macro-F1 分析****

Macro-F1 排名:

```
source_host > endpoint > time_only > destination_host
```

``source_host`` = 0.9275` 最高, 说明它在 benign 和 attack 两类之间最均衡。
``endpoint`` = 0.9215` 第二, 但它是靠高 recall 拉上来的, precision 明显下降。
``time_only`` 和 ``destination_host`` 很接近, 整体都不如 ``source_host``。

所以从综合分类质量看:

```
source_host      Stage2
```

****Macro-Precision 分析****

Macro-Precision 排名:

```
source_host > time_only > destination_host > endpoint
```

``source_host`` 最高, 而且 FP 最少:

```
source_host FP = 439
FPR = 0.5429%
```

这说明 source-based context 不只是能抓攻击, 还能很好地控制误报。

``endpoint`` 的 Macro-Precision 最低, 因为它 FP=702, 误报最多。它把更多 benign flow 判成 attack, 所以 precision 被拉低。

****Macro-Recall 分析****

Macro-Recall 排名:

```
endpoint > source_host > destination_host > time_only
```


`endpoint` 最高:

```
FN = 401
TP = 3119
Recall11 = 0.8861
```

这说明 endpoint context 覆盖更广, 能抓到最多 attack。
但代价是:

```
FP = 702
FPR = 0.8682%
```

所以 endpoint 是典型的 ****recall-oriented**** 方法: 漏报最少, 但误报更多。

`source\_host` 的 recall 也不错:

```
FN = 527
TP = 2993
Recall11 = 0.8503
```

虽然不如 endpoint 抓得多, 但误报少很多, 因此整体 Macro-F1 更高。

****PR-AUC 分析****

PR-AUC 排名:

```
source_host 0.9345
endpoint    0.9287
destination 0.9262
time_only   0.9215
```

PR-AUC 对 attack minority class 很重要。`source\_host` 最高, 说明它对 attack flow 的排序能力最好, 不只是 threshold 下表现好, 而是整体概率质量最好。

这点很关键: `source\_host` 的优势不是偶然 threshold 造成的, 而是模型本身更能区分 attack/benign。

****ROC-AUC 分析****

ROC-AUC 排名:

```
source_host > destination_host ≈ endpoint > time_only
```

四个都很高, 但 `source\_host` = 0.9938 最高。

`time\_only` = 0.9922 最低, 说明单纯时间邻近 context 的判别信息最弱。

这符合直觉: 时间接近的 flow 不一定相关; 同一 source 的 flow 更可能体现攻击行为模式, 例如扫描、重复连接、burst、异常访问行为。

****Confusion Matrix 和 FPR 分析****

`source\_host`:

```
[[80421, 439],
 [527, 2993]]
FPR = 439 / 80860 = 0.5429%
```

这是最好的误报控制, 同时 attack detection 也强。

`endpoint`:

```
[[80158, 702],
 [401, 3119]]
FPR = 702 / 80860 = 0.8682%
```

它抓到最多攻击, 但误报也最多。

`time\_only`:

```
[[80399, 461],  
 [597, 2923]]  
FPR = 0.5701%
```

误报低，但漏报最多，说明 time-only context 不够有针对性。

`destination\_host`:

```
[[80304, 556],  
 [532, 2988]]  
FPR = 0.6876%
```

表现中等，没有 source_host 稳。

****为什么 source_host 最好****

这些结果说明，在你的数据里，攻击行为更强地体现在 ****source-side behavior**** 上。也就是说，同一 source 发出的多个 flow 之间包含更有用的攻击上下文。

可能原因：

```
source  
source    packet/flow pattern  
source context    endpoint
```

相比之下，`endpoint` 虽然覆盖更多相关流，但也把 source 和 destination 两边都混进来，context 更宽，容易引入噪声，所以 recall 高但 precision 下降。

****论文结论可以这样写****

> Among different cross-flow context construction strategies, source-host context achieves the best overall performance, with the highest Macro-F1, Macro-Precision, ROC-AUC, PR-AUC, and the lowest false positive rate. Endpoint context obtains the highest Macro-Recall and detects the largest number of attack flows, but this comes at the cost of substantially more false positives. Time-only context performs worse, suggesting that temporal proximity alone is insufficient for constructing informative cross-flow neighborhoods.

****最终建议****

主配置：

```
method = source_host
```

原因：

```
Macro-F1  
Macro-Precision  
PR-AUC  
AUC  
FPR  
trade-off
```

补充讨论：

```
endpoint = recall-oriented configuration
```

如果你的论文强调“少漏报攻击”，可以单独说 endpoint 抓到最多 attack；但如果要选正式 Stage2 主结果，****source_host 更稳、更适合写论文主表****。

用户 2026-07-13 14:09:39

Stage2,其余配置都一样, 只是context:下的method: source_host, window_size: 不一样

window_size=16 运行结果: test: loss=0.019233, acc=0.986051, macro_precision=0.910546, macro_recall=0.916232, macro_f1=0.913367, weighted_precision=0.986151, weighted_recall=0.986051, weighted_f1=0.986099, auc=0.9935823350684684, pr_auc=0.9180648212098178, precision_label1=0.8280593671240549, recall_label1=0.8400568181818182, f1_label1=0.834014948526301, cm=[[80246, 614], [563, 2957]]

window_size=32 运行结果: test: loss=0.019280, acc=0.985459, macro_precision=0.896361, macro_recall=0.931003, macro_f1=0.912877, weighted_precision=0.986209, weighted_recall=0.985459, weighted_f1=0.985763, auc=0.9935286209469791, pr_auc=0.9207883345376225, precision_label1=0.7983346344002081, recall_label1=0.8715909090909091, f1_label1=0.833355969034361, cm=[[80085, 775], [452, 3068]]

window_size=64 运行结果: test: loss=0.019851, acc=0.985909, macro_precision=0.909244, macro_recall=0.916022, macro_f1=0.912602, weighted_precision=0.986030, weighted_recall=0.985909, weighted_f1=0.985967, auc=0.9935464495311761, pr_auc=0.9163385841053305, precision_label1=0.825467746439542, recall_label1=0.8397727272727272, f1_label1=0.8325587945359808, cm=[[80235, 625], [564, 2956]]

window_size=128 运行结果: test: loss=0.019559, acc=0.988552, macro_precision=0.932788, macro_recall=0.922427, macro_f1=0.927539, weighted_precision=0.988425, weighted_recall=0.988552, weighted_f1=0.988482, auc=0.993827886090999, pr_auc=0.934499456433569, precision_label1=0.872086247086247, recall_label1=0.8502840909090909, f1_label1=0.8610471806674338, cm=[[80421, 439], [527, 2993]]

window_size=256 运行结果: test: loss=0.019004, acc=0.986584, macro_precision=0.916646, macro_recall=0.915287, macro_f1=0.915965, weighted_precision=0.986563, weighted_recall=0.986584, weighted_f1=0.986574, auc=0.9937077904009175, pr_auc=0.9220614184225225, precision_label1=0.8403648802736602, recall_label1=0.8375, f1_label1=0.8389299943084804, cm=[[80300, 560], [572, 2948]]

不同配置下的运行结果。详细解析结果 (主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics, 并从这些数据进行分析)

助手 2026-07-13 14:10:14

结论很清楚: **在`method=source\_host`下, `window\_size=128`是最优主配置; `window\_size=32`是 recall-oriented 配置; `16/64`表现接近但不如 128; `256`没有继续提升, 说明 context 过大后收益下降. **

这里 test set 固定:

```
Benign = 80860
Attack = 3520
Total = 84380
FPR = FP / (FP + TN)
```

Window	Macro-F1	Macro-Precision	Macro-Recall	ROC-AUC	PR-AUC	TN	FP	FN	TP	FPR
16	0.9134	0.9105	0.9162	0.9936	0.9181	80246	614	563	2957	0.7593%
32	0.9129	0.8964	0.9310	0.9935	0.9208	80085	775	452	3068	0.9584%
64	0.9126	0.9092	0.9160	0.9935	0.9163	80235	625	564	2956	0.7729%
128	0.9275	0.9328	0.9224	0.9938	0.9345	80421	439	527	2993	0.5429%
256	0.9160	0.9166	0.9153	0.9937	0.9221	80300	560	572	2948	0.6925%

****Macro-F1 分析****

Macro-F1 排名:

128 > 256 > 16 > 32 > 64

`128 = 0.9275` 明显最高, 比其他窗口高出约 `0.011-0.015`。这说明 128 的上下文长度最能平衡 benign 和 attack 两类表现。

`16/32/64` 都在 `0.912-0.913` 左右, 说明较短上下文虽然有用, 但上下文信息还不够稳定。`256` 比它们稍好, 但仍明显不如 128, 说明继续扩大 context 没有带来线性收益。

****Macro-Precision 分析****

Macro-Precision 排名:

128 > 256 > 16 > 64 > 32

`128` 最高, 而且 FP 最少:

FP = 439
FPR = 0.5429%

这说明 `window\_size=128` 的 source-host context 最能帮助模型减少误报。它不是靠激进预测 attack 获得提升, 而是在保持较高 attack detection 的同时, 明显降低 false positives。

`32` 的 Macro-Precision 最低, 因为它 FP=775, 是所有配置里误报最多的。

****Macro-Recall 分析****

Macro-Recall 排名:

32 > 128 > 16 > 64 > 256

`32` 的 Macro-Recall 最高:

Recall11 = 0.8716
FN = 452
TP = 3068

这说明 `window\_size=32` 更偏向抓攻击, 漏报最少。但它的代价也很明显:

FP = 775
FPR = 0.9584%
Precision1 = 0.7983

也就是说, 32 是 ****recall-oriented****, 适合强调少漏报的场景, 但误报太多, 不如 128 稳。

`128` 的 recall 不是最高, 但仍然很强:

Recall11 = 0.8503
FN = 527

它比 32 多漏 75 个 attack, 但少误报 336 个 benign。对 NIDS 实际部署来说, 这个 trade-off 更合理。

****PR-AUC 分析****

PR-AUC 排名:

128 > 256 > 32 > 16 > 64

`128 = 0.9345` 最高, 说明它对 attack class 的整体 ranking 能力最好。这个很重要, 因为 PR-AUC 不依赖某一个 threshold, 能说明模型输出概率的质量更好。

`32` 虽然 recall 最高, 但 PR-AUC 低于 128, 说明它是 threshold 下比较激进, 并不代表整体排序能力最强。

****ROC-AUC 分析****

ROC-AUC 都很接近:

```
128: 0.9938
256: 0.9937
16: 0.9936
64: 0.9935
32: 0.9935
```

AUC 差距很小，说明不同 window size 的总体区分能力都很强。真正拉开差距的是：

```
PR-AUC
Macro-F1
FP/FN trade-off
FPR
```

这也说明论文里不要只强调 ROC-AUC，否则看不出 128 的优势；要重点报告 PR-AUC 和 confusion matrix。

****Confusion Matrix 和 FPR 分析****

`window\_size=128`:

```
[[80421, 439],
 [527, 2993]]
FPR = 439 / 80860 = 0.5429%
```

这是所有配置里最低 FPR，同时 Macro-F1 最高。

`window\_size=32`:

```
[[80085, 775],
 [452, 3068]]
FPR = 0.9584%
```

FN 最低，但误报最高。

`window\_size=256`:

```
[[80300, 560],
 [572, 2948]]
FPR = 0.6925%
```

误报比 128 多，漏报也比 128 多，所以 256 明显不如 128。

这点很关键：****128 同时比 256 少 FP、少 FN****，说明 256 加入的额外 source context 不是有效信息，反而可能带来噪声。

****为什么 128 最好****

结果说明 source-host context 的有效信息需要一定范围：

- `16` 太短，source 行为模式不够完整。
- `32` 抓 attack 更积极，但上下文可能偏局部，导致误报高。
- `64` 没有明显优势。
- `128` 最平衡，既有足够 source 行为上下文，又没有过多噪声。
- `256` 过大，加入更多弱相关 flow，稀释有效上下文。

所以 Stage2 的窗口不是越大越好，而是存在最佳上下文尺度。

****论文结论可以这样写****

> For source-host context modeling, `window\_size=128` achieves the best overall trade-off, obtaining the highest Macro-F1, Macro-Precision, ROC-AUC, PR-AUC, and the lowest FPR. Although `window\_size=32` achieves the highest Macro-Recall and detects the largest number of attack flows, it also introduces substantially more false positives. Increasing the window size to 256 does not further improve performance, suggesting that overly large context windows introduce noisy or weakly related flows.

****最终建议****

主配置:

```
method = source_host
window_size = 128
```

可作为补充讨论:

```
window_size = 32 recall-oriented
window_size = 256 context
```

一句话: ****Stage2 的最佳 context 不是越大越好, 而是 `source\_host + window\_size=128` 最稳。****

用户 2026-07-13 14:11:24

如果要完成论文, 现在stage2还需要什么对比结果吗?

助手 2026-07-13 14:11:49

现在 Stage2 已经覆盖了****最核心的两类对比****:

1. ****context method ablation****

```
source_host / destination_host / time_only / endpoint
```

2. ****window size ablation under source_host****

```
16 / 32 / 64 / 128 / 256
```

这已经能支撑一个很强的 Stage2 结果章节。尤其你已经得到很清楚的主结论:

```
source_host + window_size=128      Stage2
endpoint    recall-oriented
time_only
window_size
```

但如果要“完成论文”, 我建议 Stage2 还需要补 ****3 个关键对比****, 其中前两个非常重要。

****1. Stage1 vs Stage2 主对比, 必须补****

这是最重要的。Stage2 的意义是证明 cross-flow context 是否比 Stage1 independent flow detection 更好。所以你必须把:

```
Stage1 best model
Stage2 best model: source_host + window_size=128
```

放在同一张表里比较。

需要注意: 必须是****同一个 test set / 同一个 Stage1 embedding split****。你现在 Stage1 有 35,410 test samples 的结果, 也有方案 C 84,380 samples 的结果; Stage2 当前是 84,380 samples。论文里最好用和 Stage2 一致的 Stage1 baseline。

也就是要有:

```
Stage1-only on 84,380 test samples
Stage2 source_host window=128 on 84,380 test samples
```

如果没有这个, 读者会问:

> Stage2 到底比 Stage1 提升了多少?

这张表应报告:

```
Macro-F1
```

```
Macro-Precision
Macro-Recall
F1 Class1
Precision1
Recall1
ROC-AUC
PR-AUC
FPR
FP/FN
```

****2. No-context Stage2 / MLP-on-embedding baseline, 强烈建议补****

这个用于证明 Stage2 的提升来自 context, 而不是来自 “又加了一个分类器”。

你需要一个 baseline:

```
Stage2_NoContext
```

输入只用当前 flow 的 Stage1 embedding, 不看邻居 context。

如果你代码里已有 `stage2\_no\_context.yaml` 或 `Stage2NoContextMLP`, 就直接跑它。

这张对比非常关键:

```
Stage1 embedding + no context
Stage1 embedding + source_host context
```

如果 source_host 明显更好, 就能证明 cross-flow context 有价值。

否则别人可能说:

> 你的 Stage2 只是重新训练了一个 classifier, 不一定是 context 带来的提升。

****3. Stage2 model architecture ablation, 建议补****

你现在 Stage2 大概率主要是 Transformer context encoder。最好再补一个轻量 baseline:

```
LSTM context
GRU context
Mean/attention pooling context
```

最小可以只补:

```
Stage2 Transformer vs Stage2 LSTM
```

目的: 证明不是任意序列模型都一样, 或者说明 context quality 比 architecture 更重要。

如果时间紧, 这个可以作为 optional, 但论文里会更完整。

****4. Past-only vs past+current / leakage 说明, 至少要写清楚****

这个不一定要重新跑, 但论文必须说明:

```
context          split
test context      train/val flows
      future flows
include_target    true
```

如果你现在是 split-isolated context, 就要明确写:

> Context windows are constructed within each split independently to avoid cross-split leakage.

如果用了未来流, 必须说明是 offline detection; 如果想更严谨, 可以补一个:

```
past-only source_host window=128
```

这个是部署友好版本, 但不是绝对必须。

****5. Efficiency 对比，重新跑后可自动输出****

你刚才已经让我帮你加了：

```
Params
training_time_seconds
inference_time_seconds_test
samples_per_second
```

所以 Stage2 最终主表里可以加：

```
Params
Training Time
Inference Time
```

建议至少对这几个模型有 efficiency：

```
Stage1 best
Stage2 no-context
Stage2 source_host window=128
Stage2 endpoint window=128
```

****最终推荐的 Stage2 完成清单****

如果你想 “论文够完整” ， Stage2 最少补这 2 个：

1. Stage1-only vs Stage2 source_host window=128 test set
2. Stage2 no-context vs Stage2 source_host window=128

如果你想 “论文更稳” ， 再补这 2 个：

3. Stage2 Transformer vs Stage2 LSTM
4. past-only / leakage-free context

****你现在已经有的可以作为论文表****

Table A: Context method ablation

```
time_only
source_host
destination_host
endpoint
```

Table B: Window size ablation

```
16
32
64
128
256
```

****还需要补的论文表****

Table C: Stage1 vs Stage2 contribution

```
Stage1-only
Stage2 no-context
Stage2 source_host w=128
```

Table D optional: Architecture ablation

```
Transformer context
LSTM context
```

一句话：****Stage2 的 ablation 已经很不错，但要完成论文，还必须补一个 “Stage1/No-context vs Stage2 context” 的贡献对比；否则 Stage2 的提升来源还不够闭合。 ****

用户 2026-07-13 14:22:32

我现在的stage2,只有no-context和Stage2 LSTM, 我需要添加一个GRU在加一个CNN+LSTM。检查我现在s2下面的代码, 在我现在代码的基础上, 加上这两个模型, 尽量少的改动

助手 2026-07-13 14:22:44

我会先看 `s2/stage2/model.py` 里现有的 `no-context` 和 `LSTM` 是怎么接入的, 再看 `run\_stage2.py` 对 `model\_type` 有没有 choices 限制。目标是只在模型工厂和必要的配置入口上加 `gru`、`cnn\_lstm`, 不碰训练流程。

助手 2026-07-13 14:22:57

发现一个好消息: `Stage2CNNLSTM` 类其实已经有雏形, 但还没有接到 `build\_stage2\_model()`, 而且它现在返回 sigmoid 单 logit, 不兼容现有 trainer 的 CrossEntropy/FocalLoss 路径。GRU 还没有类。我会尽量复用 LSTM 的结构: 新增一个 `Stage2GRU`, 并把 CNN+LSTM 改成同样的 `cfg,input\_dim -> logits [B,2]` 风格。

助手 2026-07-13 14:23:12

我会做三处小改动: 1) 在 `model.py` 加 `Stage2GRU`; 2) 把现有 `Stage2CNNLSTM` 改成 trainer 兼容的 cfg 版本; 3) 把 `build\_stage2\_model()` 和 `run\_stage2.py --model\_type` choices 接上 `gru`、`cnn\_lstm`。训练器、dataset、context 都不动。

助手 2026-07-13 14:24:20

补丁第一次没套上, 是因为文件里已有 CNN+LSTM 注释有编码乱码, 精确匹配不稳。为了少动现有代码, 我换一个更安全的做法: 保留旧 `Stage2CNNLSTM` 不动, 新增一个兼容 trainer 的 `Stage2CNNLSTMCompat`, factory 里让 `cnn\_lstm` 指向新类。

助手 2026-07-13 14:25:03

新模型类已经加进 `model.py` 了。现在我把模型工厂和命令行 `--model\_type` 接上, 这样你可以直接在 YAML 写 `model\_type: gru` 或 `model\_type: cnn\_lstm`, 也可以命令行覆盖。

助手 2026-07-13 14:25:17

为了避免以后忘记支持哪些类型, 我再更新一下默认配置注释, 不改变实际默认值。然后会做 py_compile 和一个轻量实例化检查。

助手 2026-07-13 14:25:38

我会先编译 `model.py/run\_stage2.py/config.py`, 再用一个很小的假输入实例化 `gru` 和 `cnn\_lstm` 并做一次 forward, 确认输出是 `[batch, 2]` logits, 能直接喂给现有 loss。

助手 2026-07-13 14:25:45

编译检查通过了; 实例化测试在 bundled Python 里失败是因为这个 runtime 没有安装 `torch`, 不是代码问题。我会用你本机 Python 路径再做一次轻量检查; 如果本机 Python 也没有 torch, 我就只保留静态检查结果。

助手 2026-07-13 14:26:16

本机 Python 有 `torch 2.11.0+cpu`。我现在用同一个解释器做一次新增模型的 forward smoke test，确保不是只通过语法检查。

助手 2026-07-13 14:26:37

smoke test 通过: `gru` 和 `cnn\_lstm` 都输出了 `[batch, 2]` 的 float logits，和现有 Stage2 trainer/loss 兼容。最后我看一下 diff 范围，确认只改了必要文件。

助手 2026-07-13 14:27:12

已加好，尽量保持了最小改动。

改动位置：

- [s2/stage2/model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:378)
- 新增 `Stage2GRU`
- 新增 trainer 兼容版 `Stage2CNNLSTMCompat`
- 在 `build\_stage2\_model()` 里接入：
- `model\_type: gru`
- `model\_type: cnn\_lstm`
- [s2/run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:92)
- `--model\_type` choices 增加 `gru` 和 `cnn\_lstm`
- [s2/stage2/config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:46)
- 更新注释，标明支持 `gru / cnn\_lstm`

我保留了原来那个旧 `Stage2CNNLSTM` 类不动，但没有让 factory 使用它，因为它原本返回的是 sigmoid 单输出，不兼容你现在的 Stage2 trainer/loss。新的 `Stage2CNNLSTMCompat` 输出 `[B, 2]` logits，可以直接和现有训练流程一起用。

****怎么跑****

GRU：

```
model:
  model_type: gru
  d_model: null
  lstm_hidden_dim: 256
  lstm_num_layers: 2
  lstm_bidirectional: false
  dropout: 0.3
  pooling: last
```

也可以用更明确的 GRU 字段：

```
model:
  model_type: gru
  gru_hidden_dim: 256
  gru_num_layers: 2
  gru_bidirectional: false
```

CNN+LSTM：

```
model:
  model_type: cnn_lstm
  d_model: null
  cnn_kernel_sizes: [3, 5, 7]
  cnn_out_channels: 128
```

```
cnn_lstm_hidden_dim: 256
cnn_lstm_num_layers: 2
cnn_lstm_bidirectional: false
dropout: 0.3
pooling: last
```

命令行也可以直接覆盖：

```
--model_type gru
```

或：

```
--model_type cnn_lstm
```

我做了两步验证：

```
- `py_compile` 通过: `model.py / run_stage2.py / config.py`
- 用本机 `torch 2.11.0+cpu` 做了 forward smoke test:
- `gru -> (3, 2)`
- `cnn_lstm -> (3, 2)`
```

也就是两个新模型都能输出现有 trainer 需要的二分类 logits。

用户 2026-07-13 15:04:41

我稍后会补充stage2中缺少的和non-context、LSTM、GRU、CNN+LSTM 的对比结果。
我的整篇论文，现在还缺少什么必要的结果？

助手 2026-07-13 15:05:12

现在你的论文结果已经比较成形了。等你补上 Stage2 的 `no-context / LSTM / GRU / CNN+LSTM`，**核心模型实验基本就闭合了**。剩下真正“必要”的结果，主要不是再加很多模型，而是补齐论文可信度：baseline、公平比较、泄漏排查、鲁棒性和效率。

****必须补的结果****

1. ****最终总表：Stage1 vs Stage2****

你需要一张全论文最重要的总表：

```
Flow-level MLP baseline
Stage1 best: Ours_BothEncoding
Stage2 no-context
Stage2 best: source_host + window_size=128
Stage2 LSTM / GRU / CNN+LSTM
```

而且最好都在同一个 test set 上。

你现在 Stage1 有 35,410 test samples，也有方案 C 的 84,380 samples；Stage2 是 84,380 samples。为了论文严谨，最终总表必须避免混用不同 test set。

最理想口径：

```
84,380 test samples
```

这张表回答：

```
> packet sequence 是否优于 flow aggregation?
> Stage2 context 是否优于 Stage1/no-context?
> Transformer context 是否优于 LSTM/GRU/CNN+LSTM?
```

2. ****强 flow-level baseline****

你现在有 MLP flow-level baseline，但论文最好还需要一个强 tabular baseline：

```
Random Forest / XGBoost / LightGBM / ExtraTrees
```

原因是安全领域审稿/答辩很容易问：

> 你的 Transformer 是否真的比传统 flow-feature model 强？

如果 LGBM 特别强甚至超过深度模型，也可以写成重要发现，但必须报告。

最少补一个：

```
LightGBM on flow-level statistical features
```

但要放在单独 section 里，并讨论可能的 shortcut / flow statistics advantage。

3. **Leakage / split sanity check**

这是必要的，不一定是很复杂的新模型结果，但必须有证据。至少需要报告：

```
train/val/test split counts
label distribution per split
    flow_id IP timestamp Suricata alert/signature/rule id
duplicate flow_id      split
Stage2 context      split-isolated
```

如果你能做一个表最好：

```
| Check | Result |
|---|---|
| Cross-split context leakage | No |
| Rule/signature features used | No |
| Duplicate flow IDs across split | 0 / or reported |
| Normalization fitted on train only | Yes |
| Test threshold selected from test? | No, from val |
```

这个对论文非常关键。

4. **最终配置消融汇总**

你已经有 Stage1 和 Stage2 消融，但最终论文需要整理成清楚的表：

Stage1:

```
NoEncoding / PositionOnly / TimeOnly / BothEncoding
max_seq_len: 8/16/32/64/128/256
strategy: head / head_tail / random
```

Stage2:

```
context method: time_only/source_host/destination_host/endpoint
window_size: 16/32/64/128/256
model architecture: Transformer/LSTM/GRU/CNN+LSTM/no-context
```

这些你基本都有或即将补齐。

5. **Efficiency 结果**

论文需要报告：

```
Params
Training time
Inference time
Samples/sec
```

你现在 Stage1 已经有 Params/Time, Stage2 代码我也帮你加了统计。等重跑 Stage2 后, 把这些放进最终总表或单独 efficiency table。

****强烈建议补的结果****

6. ****Multi-seed 或稳定性结果****

如果时间允许, 至少对最终主模型跑 3 seeds:

```
Stage1 best
Stage2 best
LightGBM / MLP baseline
```

报告:

```
Macro-F1 mean ± std
PR-AUC mean ± std
FPR mean ± std
```

如果没时间跑所有模型, 最少跑:

```
Stage2 source_host window=128
Stage2 no-context
```

这样能说明 Stage2 提升不是偶然。

7. ****Error analysis****

需要一小节, 不一定大量实验。建议做:

```
Stage1 FN -> Stage2 TP
Stage1 TP -> Stage2 FN
Stage1 FP -> Stage2 TN
Stage1 TN -> Stage2 FP
```

重点分析 Stage2 修正了哪些攻击:

- 是否同一 source 有攻击簇?
- 是否周围 flows attack ratio 更高?
- endpoint/source context 是否提供了额外证据?

这会让论文讨论非常有说服力。

8. ****Context availability / context quality 分析****

你已经有类似 context analysis。论文里可以放一张简表:

```
context type
average context length
coverage
attack-flow context signal
benign-flow context signal
```

用来解释为什么:

```
source_host
endpoint recall
time_only    host-based context
```

****不是必须, 但可以作为 future work 的结果****

这些不一定要补:

```
cross-dataset evaluation
unseen attack evaluation
past-only online detection
full multiclass classification
interpretability/attention visualization
```

如果时间够当然好，但不是完成论文的最低要求。

****你现在的完成度判断****

部分	状态
Stage1 主结果	基本完成
Stage1 消融	基本完成
Stage2 context method	已完成
Stage2 window size	已完成
Stage2 architecture baseline	你准备补，补完就完整
Flow-level MLP baseline	已有
强 tabular baseline	建议补
Efficiency	代码已加，需重跑整理
Leakage/split sanity	必须整理成表
Multi-seed	建议补
Error analysis	建议补

****最低完成论文还缺这 5 个****

如果按“必须交论文”的最低标准，我建议只盯这 5 件：

1. test set
2. Stage2 no-context / LSTM / GRU / CNN+LSTM
3. flow-level baseline LightGBM
4. leakage/split sanity check
5. efficiency table Params / training time / inference time

一句话：****模型结果主体已经够了；现在论文缺的是把结果变成可信的实验闭环。接下来不要无限加模型，优先补同 test set 总表、强 baseline、leakage 检查和效率结果。****

用户 2026-07-13 15:23:48

我的整个idea，首先公司抓取网络上的PCAP文件，我拿到PCAP后用suricata自定义的插件（suricata-plugin/custom-out），从PCAP中抽取相关的packet和flow features的csv文件。然后对数据进行清洗、preprocessing，必要的转换等。然后传入到stage1-time-aware transformer生成相应的embedding在和flow的statistical feature进行late融合，生成一个最终的stage1 embedding。然后将这个s1 embedding传入到stage2的transformer（构建context的方法：time_only、source_host、destination_host、endpoint），得到最终的分类结果。

结合我的代码，详细解析。

（主要关注macro_f1 作为F1、macro_recall作为recall、macro_precision作为precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

帮我完成我的论文，我该包含那些模块？每个模块该写些什么？接下来一个模块一个模块的写。我需要latex模式，加入相关的引用。

助手 2026-07-13 15:24:09

我会先快速对照你现在的代码路径：Suricata 插件、Stage1 模型/融合、Stage2 context 构建和 trainer 输出指标。这样我给你的论文结构不是抽象模板，而是能直接对应你项目里的实现。

助手 2026-07-13 15:27:13

你的论文主线可以写成一句话：

> 本文提出一个两阶段层次化入侵检测框架：首先从 PCAP 中抽取 packet-level 与 flow-level 特征，用 Stage1 time-aware Transformer 学习单个 flow 内部的时序 packet 表征，并与 flow statistical features 进行 late fusion；随后将 Stage1 embedding 输入 Stage2 context-aware Transformer，通过 time-only/source-host/destination-host/endpoint 等上下文构建方式建模 flow 之间的关联，从而提升最终攻击检测性能。

下面是我建议你的论文模块结构。后面我们可以按这个顺序，一个模块一个模块写成 LaTeX。

****1. Introduction****

写什么：

- 网络入侵检测为什么重要。
- 传统基于规则的 IDS，如 Suricata/Snort，能提取高质量告警和流量信息，但难以直接捕获复杂时序行为。
- 传统 ML/DL 方法常使用 flow-level aggregate statistical features，容易丢失 packet 顺序、时间间隔、跨 flow 上下文。
- 你的核心 idea：
- Stage1 学习 flow 内 packet 序列表示；
- Stage1 embedding 与 flow statistical features 融合；
- Stage2 建模 flow 之间的上下文关系；
- 使用 macro-F1、macro-recall、macro-precision、ROC-AUC、PR-AUC、FPR 等指标评估。
- 明确 contribution。

LaTeX 可写：

```
\chapter{Introduction}
```

Network intrusion detection systems (NIDSs) are a critical component of modern cyber-security infrastructure. Traditional rule-based systems such as Snort and Suricata are effective at identifying known attack signatures, but they often rely on manually designed rules and may fail to capture subtle temporal and contextual patterns in network traffic~\cite{roesch1999snort,oisf2026suricata}. Meanwhile, machine learning based NIDS approaches commonly represent each network flow using aggregated statistical features, such as packet counts, byte counts, duration, and inter-arrival time statistics. Although such features are compact and efficient, they may discard fine-grained packet ordering and timing information that is useful for detecting complex attacks.

This thesis proposes a hierarchical two-stage intrusion detection framework. Given raw PCAP files, a custom Suricata output plugin is used to extract packet-level and flow-level CSV features. After preprocessing, the first stage applies a time-aware Transformer to model packet sequences within each flow. The resulting packet-sequence embedding is fused with flow-level statistical features to obtain a final flow representation. The second stage further models contextual relationships among flows by constructing flow neighborhoods according to temporal proximity, source host, destination host, or endpoint identity. A context-aware Transformer is then used to produce the final binary classification result.

The main contributions of this thesis are as follows:

```
\begin{itemize}
```

- \item A packet-to-flow feature extraction pipeline based on a custom Suricata plugin, enabling both packet-level temporal modeling and flow-level statistical modeling.
- \item A Stage1 time-aware Transformer architecture that combines positional and temporal encodings to learn flow-level embeddings from packet sequences.
- \item A late-fusion mechanism that integrates learned packet-sequence embeddings with flow-level statistical features.
- \item A Stage2 context-aware Transformer that models inter-flow relationships using different context construction strategies, including time-only, source-host, destination-host, and endpoint-based contexts.
- \item A comprehensive evaluation using macro-F1, macro-precision, macro-recall, ROC-AUC, PR-AUC, confusion matrix, and false positive rate.

```
\end{itemize}
```

****2. Related Work****

写什么：

- Rule-based IDS: Snort、Suricata。
- ML-based NIDS: Random Forest、MLP、XGBoost、flow-level statistical features。
- Deep learning NIDS: CNN、LSTM、GRU、BiLSTM。
- Transformer for network traffic: attention 适合建模长距离依赖。
- Context-aware intrusion detection: 跨 flow、host-level、session-level 上下文。
- 指出 gap: 很多方法只看单个 flow, 或者只看统计特征; 你的方法同时建模 packet 内时序和 flow 间上下文。

可以用这些引用 key:

```
\cite{roesch1999snort,oisf2026suricata}
\cite{sommer2010outside}
\cite{vaswani2017attention}
\cite{mirsky2018kitsune}
\cite{yin2017rnn}
\cite{lin2022etbert}
\cite{manocchio2024flowtransformer}
```

****3. Data Collection and Feature Extraction****

这一章要结合你的代码重点写。

对应代码:

- `suricata-plugin/custom-out-40f.c`
- `suricata-plugin/custom-out.c`
- packet CSV extraction
- flow CSV extraction

写什么:

- 输入是 PCAP。
- Suricata 解析 packet、flow、五元组、时间戳、协议、方向、payload/length 等。
- 自定义 plugin 输出两类 CSV:
- packet-level features: 每个 packet 一行;
- flow-level statistical features: 每个 flow 一行。
- label 来源: Suricata alert/rule 触发情况。
- 需要强调: alert metadata 或 rule id 不应该作为模型输入, 否则会造成 label leakage。
- flow label 可以定义为: 如果该 flow 中任意 packet 触发 alert, 则该 flow 为 malicious。

LaTeX 可写:

```
\chapter{Data Collection and Feature Extraction}
```

Raw network traffic is provided in PCAP format. To transform PCAP traces into structured learning samples, this work implements a custom Suricata output plugin. The plugin extracts two complementary feature views: packet-level features and flow-level statistical features. Packet-level features preserve fine-grained temporal information, while flow-level features summarize aggregate behavior over the lifetime of a flow.

For each packet, the extractor records timestamp, flow identifier, protocol-related fields, packet length, direction-related information, and other packet-level attributes. For each flow, the extractor computes statistical features such as flow duration, packet counts, byte counts, packet-rate features, byte-rate features, and inter-arrival-time statistics. The label of a packet is derived from Suricata alerts. A flow is labeled as malicious if at least one packet within the flow triggers an alert.

To avoid label leakage, alert-specific metadata, such as rule identifiers or alert messages, is not used as model input. These fields are used only for label generation and analysis.

****4. Preprocessing****

对应代码:

- `s1/stage1/preprocessing.py`
- `s1/stage1/data\_io.py`
- `s1/stage1/splits.py`

写什么：

- CSV cleaning: missing values、invalid values、type conversion。
- numerical features: log1p、clipping、standardization。
- categorical features: one-hot encoding。
- packet sequence construction: 按 flow_id grouping, 再按 timestamp 排序。
- sequence truncation/padding:
- `max\_seq\_len`
- `strategy = head / head\_tail / random`
- train/validation/test split:
- flow-level split;
- 防止同一个 flow 出现在多个 split;
- 如果是时间序列实验, 说明 chronological split。
- class imbalance:
- 为什么 accuracy 不够;
- 所以重点看 macro-F1、PR-AUC、FPR。

LaTeX:

```
\chapter{Preprocessing}
```

After feature extraction, packet-level and flow-level CSV files are cleaned and transformed before training. Numerical features are converted to floating-point values, invalid values are handled, and heavily skewed features are transformed using logarithmic scaling where appropriate. Feature clipping and normalization are applied to reduce the influence of extreme values. Categorical fields are encoded into numerical representations.

Packet-level samples are grouped by flow identifier and sorted by timestamp to construct packet sequences. Since flows may contain different numbers of packets, each sequence is either truncated or padded to a fixed maximum sequence length. This thesis evaluates several truncation strategies, including head, head-tail, and random selection. Unless otherwise specified, the main Stage1 configuration uses the head strategy with a maximum sequence length of 128.

The dataset is split at the flow level to avoid leakage between training, validation, and test sets. This is important because packets from the same flow are highly correlated. If packets from the same flow appeared in both training and test sets, the evaluation would overestimate the generalization ability of the model.

****5. Methodology****

这是论文核心, 可以分成 Stage1 和 Stage2。

****5.1 Stage1: Time-Aware Flow Representation Learning****

对应代码：

- `s1/stage1/model\_modify.py`
- `TimeAwareEncoding`
- positional encoding
- temporal encoding
- attention pooling
- flow feature encoder
- fusion module

写什么：

- 输入：一个 flow 的 packet sequence。
- 每个 packet 有 feature vector。
- 加入 positional encoding 表示 packet 顺序。
- 加入 time-aware encoding 表示 packet 时间间隔或时间戳信息。
- Transformer encoder 学习 packet sequence。
- attention pooling 或 pooling 得到 flow embedding。
- flow statistical features 经过 MLP encoder。
- late fusion: packet embedding + flow statistical embedding。
- 输出 Stage1 classification 和 Stage1 embedding。

公式：

$$\mathbf{h}^{\{p\}}_i = \mathrm{Transformer}(\mathbf{x}_{i,1:T} + \mathbf{e}^{\{pos\}}_{1:T} + \mathbf{e}^{\{time\}}_{1:T})$$

$$\mathbf{z}^{\{packet\}}_i = \mathrm{Pool}(\mathbf{h}^{\{p\}}_i)$$

$$\mathbf{z}^{\{flow\}}_i = \mathrm{MLP}(\mathbf{s}_i)$$

$$\mathbf{z}^{\{stage1\}}_i = \mathrm{Fusion}(\mathbf{z}^{\{packet\}}_i, \mathbf{z}^{\{flow\}}_i)$$

5.2 Stage2: Context-Aware Flow Classification

对应代码：

- `s2/stage2/context.py`
- `s2/stage2/model.py`
- `s2/stage2/trainer.py`

写什么：

- Stage2 输入是 Stage1 embedding。
- 对每个 target flow 构建 context window。
- context 方法：
 - `time\_only`：只按时间邻近；
 - `source\_host`：相同 source host 的邻近 flows；
 - `destination\_host`：相同 destination host 的邻近 flows；
 - `endpoint`：共享 source 或 destination endpoint 的 flows。
- window_size 控制上下文长度。
- Transformer 对 context sequence 编码。
- 使用 target flow 的上下文表示做最终分类。

LaTeX：

$$\text{\section{Stage2: Context-Aware Flow Classification}}$$

Although Stage1 models packet-level behavior within each flow, many attacks are distributed across multiple flows. Therefore, Stage2 models relationships among neighboring flows. For each target flow, a context sequence is constructed from Stage1 embeddings according to a selected context method. This thesis considers four context construction strategies: time-only, source-host, destination-host, and endpoint-based context.

Let $\mathbf{z}^{\{stage1\}}_i$ denote the Stage1 embedding of flow i . For each target flow i , a context builder selects a sequence of neighboring flow embeddings:

$$\mathcal{C}_i = [\mathbf{z}^{\{stage1\}}_{j_1}, \mathbf{z}^{\{stage1\}}_{j_2}, \ldots, \mathbf{z}^{\{stage1\}}_{j_K}],$$

where K is controlled by the context window size. A context-aware Transformer then encodes $\mathcal{C}_i$ and produces the final prediction:

$$\hat{y}_i = \mathrm{Classifier}(\mathrm{Transformer}(\mathcal{C}_i)).$$

****6. Evaluation Metrics****

这一章必须写清楚，因为你的结果主要围绕这些指标。

重点：

- accuracy 不作为主要指标，因为类别不平衡。
- macro-precision、macro-recall、macro-F1 更公平。
- label1 precision/recall/F1 也要报告，因为 label1 是 attack。
- ROC-AUC 衡量整体 ranking ability。
- PR-AUC 对不平衡数据更敏感。
- confusion matrix 用来解释 FP/FN trade-off。
- FPR 对 NIDS 非常重要，因为误报太多会增加安全分析负担。

LaTeX：

```
\section{Evaluation Metrics}
```

Because the dataset is highly imbalanced, accuracy alone is insufficient. This thesis mainly reports macro-precision, macro-recall, macro-F1, ROC-AUC, PR-AUC, confusion matrix, and false positive rate.

For binary classification, the confusion matrix is defined as:

```
\[
\begin{bmatrix}
TN & FP \\
FN & TP
\end{bmatrix}.
\]
```

The false positive rate is computed as:

```
\[
\mathrm{FPR} = \frac{FP}{FP + TN}.
\]
```

For each class c , precision, recall, and F1-score are defined as:

```
\[
P_c = \frac{TP_c}{TP_c + FP_c},
\]
\[
R_c = \frac{TP_c}{TP_c + FN_c},
\]
\[
F1_c = \frac{2P_cR_c}{P_c + R_c}.
\]
```

Macro-F1 is computed by averaging the F1-scores of all classes:

```
\[
\mathrm{Macro}\text{-}F1 = \frac{1}{C} \sum_{c=1}^C F1_c.
\]
```

PR-AUC is particularly important for imbalanced intrusion detection tasks, because it focuses on the trade-off between precision and recall for the positive class~\cite{saito2015precision}. ROC-AUC is also reported to measure the threshold-independent ranking ability of the model~\cite{fawcett2006roc}.

****7. Experimental Results****

建议分成这些小节。

****7.1 Stage1: Flow-Level MLP vs Packet-Sequence Transformer****

目的：回答 RQ1。

你已经有结果：

- Flow-level MLP macro-F1 = 0.7707, PR-AUC = 0.5604。
- Stage1 Transformer macro-F1 约 0.8882, PR-AUC 约 0.8564。
- 说明 packet sequence + time-aware modeling 明显优于纯 flow statistical features。

可写分析：

The flow-level MLP baseline achieves a macro-F1 of 0.7707 and a PR-AUC of 0.5604, indicating that aggregated statistical features alone are insufficient for detecting minority-class attacks. In contrast, the proposed Stage1 time-aware Transformer achieves a substantially higher macro-F1 of 0.8882 and PR-AUC of 0.8564. This improvement suggests that packet ordering and temporal dynamics provide important information beyond flow-level aggregation.

****7.2 Stage1 Encoding Ablation****

比较：

- NoEncoding
- PositionOnly
- TimeOnly
- BothEncoding

结论：

- BothEncoding 最好。
- TimeOnly 优于 PositionOnly, 说明时间信息很关键。
- NoEncoding 最差, 说明单纯 Transformer 不加时序信息不够。

****7.3 Stage1 max_seq_len Ablation****

比较 8, 16, 32, 64, 128, 256。

结论：

- `max\_seq\_len=128` 是主结果。
- 64 recall 稍高, 但综合 macro-F1、precision、PR-AUC、FPR, 128 更稳。
- 256 下降, 说明过长序列可能引入噪声或训练难度。

****7.4 Stage1 Sampling Strategy Ablation****

比较：

- head
- head_tail
- random

结论：

- head 最佳 macro-F1;
- head_tail 接近;
- random FPR 最低但 FN 增多, recall 降低。

****7.5 Stage2 Context Method Ablation****

你已有：

- source_host 最佳: macro-F1 = 0.9275, PR-AUC = 0.9345, FPR = 0.5429%。
- endpoint recall 最高, 但 FPR 也最高。
- time_only 低于 source_host, 说明仅时间邻近不如 host-level context。
- destination_host 稍弱于 source_host。

可写:

Among the evaluated context construction strategies, source-host context achieves the best overall performance, with a macro-F1 of 0.9275, ROC-AUC of 0.9938, PR-AUC of 0.9345, and FPR of 0.5429%. This suggests that flows originating from the same source host provide highly informative behavioral context for intrusion detection. Endpoint-based context achieves the highest recall but introduces more false positives, indicating a recall-oriented trade-off.

****7.6 Stage2 Window Size Ablation****

比较 16, 32, 64, 128, 256。

结论:

- window=128 最佳。
- window=32 recall 较高, 但 FPR 高。
- window=256 没有继续提升, 说明上下文过大可能引入无关 flow。

****7.7 Stage2 Architecture Comparison****

这里你现在还缺最终完整表格。

需要补:

- Stage2 No-context
- Stage2 LSTM
- Stage2 GRU
- Stage2 CNN+LSTM
- Stage2 Transformer

这一节的目的:

- 证明提升不是仅仅来自 Stage1 embedding;
- 证明 context Transformer 比传统序列模型更适合建模 flow context;
- 和 no-context 比较, 回答 RQ2。

****8. Discussion****

写什么:

- 为什么 Stage1 有效:
- packet-level temporal behavior;
- flow aggregation 会丢失细节。
- 为什么 Stage2 有效:
- 攻击常常不是单个 flow, 而是一组相关 flow;
- source_host context 捕获同一源主机行为模式。
- FPR 解释:
- NIDS 中低 FPR 重要;
- endpoint recall 高但误报高;
- source_host 是更平衡选择。
- PR-AUC 解释:
- 数据不平衡;
- PR-AUC 提升说明对攻击类排序更好。
- 限制:
- label 来自 Suricata alert, 可能继承规则系统偏差;
- 需要更多真实环境数据验证;
- 如果不同实验 test set size 不同, 最终论文中必须明确说明, 最好统一 split 后重新跑主表。

****9. Conclusion****

写什么：

- 总结两阶段模型。
- Stage1 证明 packet-level temporal modeling 有效。
- Stage2 证明 inter-flow context 有效。
- source_host + window=128 是当前最佳配置。
- 未来工作：
 - online deployment;
 - multi-class classification;
 - real-time inference;
 - cross-dataset generalization;
 - explainability.

****你现在还缺哪些必要结果****

你的 Stage1 已经基本覆盖充分。Stage2 还需要补齐这几类，论文才完整：

1. Stage2 no-context vs context-aware Transformer

这是证明 Stage2 价值的核心。

2. Stage2 Transformer vs LSTM/GRU/CNN+LSTM

这是证明 Transformer context encoder 有优势的核心。

3. Stage1-only final classifier vs Stage1+Stage2 final classifier

必须在同一个 test split 上比较。

4. Efficiency table

每个模型报告：

- Params
- training time
- inference time
- macro-F1
- PR-AUC
- FPR

5. Same-split consistency check

你的 Stage1 方案 B 是 35,410 samples，方案 C 和 Stage2 是 84,380 samples。最终论文主表最好统一 test set，否则要分表并明确说明不可直接横向比较。

****最终论文 LaTeX 目录建议****

```
\chapter{Introduction}

\chapter{Related Work}
\section{Rule-Based Network Intrusion Detection}
\section{Machine Learning for Flow-Based Intrusion Detection}
\section{Deep Learning for Network Traffic Modeling}
\section{Transformer-Based Traffic Representation Learning}
\section{Context-Aware Intrusion Detection}

\chapter{Data Collection and Feature Extraction}
\section{PCAP Collection}
\section{Custom Suricata Feature Extraction}
\section{Packet-Level Features}
\section{Flow-Level Statistical Features}
\section{Label Generation and Leakage Prevention}
```

```

\chapter{Preprocessing}
\section{Data Cleaning}
\section{Feature Transformation}
\section{Packet Sequence Construction}
\section{Train-Validation-Test Split}
\section{Class Imbalance}

\chapter{Methodology}
\section{Overview}
\section{Stage1: Time-Aware Packet-Sequence Transformer}
\section{Flow Statistical Feature Fusion}
\section{Stage2: Context-Aware Flow Transformer}
\section{Context Construction Strategies}
\section{Training Objective}

\chapter{Experimental Setup}
\section{Dataset}
\section{Baselines}
\section{Implementation Details}
\section{Evaluation Metrics}

\chapter{Results}
\section{Stage1 Main Results}
\section{Stage1 Encoding Ablation}
\section{Stage1 Sequence Length Ablation}
\section{Stage1 Sampling Strategy Ablation}
\section{Stage2 Context Method Ablation}
\section{Stage2 Window Size Ablation}
\section{Stage2 Model Architecture Comparison}
\section{Efficiency Analysis}

\chapter{Discussion}
\section{Effectiveness of Packet-Level Temporal Modeling}
\section{Effectiveness of Flow-Level Context}
\section{Precision-Recall and FPR Trade-Off}
\section{Limitations}

\chapter{Conclusion and Future Work}

```

****BibTeX 参考 key 可以先准备这些****

```

@inproceedings{vaswani2017attention,
  title={Attention Is All You Need},
  author={Vaswani, Ashish and others},
  booktitle={NeurIPS},
  year={2017}
}

@article{fawcett2006roc,
  title={An Introduction to ROC Analysis},
  author={Fawcett, Tom},
  journal={Pattern Recognition Letters},
  year={2006}
}

@article{saito2015precision,
  title={The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets},
  author={Saito, Takaya and Rehmsmeier, Marc},
  journal={PLOS ONE},
  year={2015}
}

@inproceedings{sommer2010outside,
  title={Outside the Closed World: On Using Machine Learning for Network Intrusion Detection},
  author={Sommer, Robin and Paxson, Vern},
  booktitle={IEEE Symposium on Security and Privacy},
  year={2010}
}

```

```
@misc{oisf2026suricata,
  title={Suricata: Open Source Threat Detection Engine},
  author={{Open Information Security Foundation}},
  year={2026},
  note={Available online}
}

@inproceedings{roesch1999snort,
  title={Snort: Lightweight Intrusion Detection for Networks},
  author={Roesch, Martin},
  booktitle={LISA},
  year={1999}
}
```

下一步建议先写 `Introduction` 完整版本，然后写 `Related

Work`。你的论文主线已经很清楚了，现在关键是把 “代码 pipeline + 实验表格 + 指标解释” 组织成论文语言。

用户 2026-07-13 16:06:28

Files mentioned by the user:

main.tex: C:/Users/XiaoyanXiong/Downloads/main.tex

references.bib: C:/Users/XiaoyanXiong/Downloads/references.bib

My request for Codex:

我提供的文件使我的proposal的latex版本，请参考proposal中内容。请给我提供Introduction的完整版的latex，引入必要的论文

助手 2026-07-13 16:06:39

我先看一下你提供的 `main.tex` 和 `references.bib`，这样 Introduction 会尽量贴合 proposal 里的结构、术语和已有引用 key，而不是重新发明一套写法。

助手 2026-07-13 16:07:05

我看到了：你的 `main.tex` 已经有 proposal 版 Introduction，而且 `references.bib` 里已经准备了 Snort、Suricata、Transformer、GTID、FlowTransformer、ET-BERT、DoLLM 等引用。下面我会沿用这些 BibTeX key，给你一个可以直接替换 `\section{Introduction}` 的完整版。

助手 2026-07-13 16:07:36

下面这版可以直接替换你 `main.tex` 里的 `\section{Introduction}`。我沿用了你 `references.bib` 中已经存在的引用 key，比如 `Roesch1999`、`SuricataGuide2026`、`SommerPaxson2010`、`Vaswani2017`、`Han2023GTID`、`Manocchio2024FlowTransformer`、`Li2024DoLLM` 等。

小提醒：你的 `references.bib` 里 `Hozouri2025Survey` 出现了重复 key，后面编译 BibTeX/Biber 时可能报错，之后需要删掉一个重复项。

```
\section{Introduction}
```

Modern computer networks generate large volumes of heterogeneous traffic across enterprise systems, cloud infrastructures, mobile networks, and Internet-of-Things (IoT) environments. Protecting such networks requires continuous monitoring and timely detection of malicious activities, including scanning, exploitation attempts, botnet communication, distributed denial-of-service (DDoS) attacks, and multi-stage intrusions. Network Intrusion Detection Systems (NIDSs) are therefore a fundamental component of cyber-security infrastructures \cite{Roesch1999,SommerPaxson2010}. However, building accurate and practical NIDSs remains challenging because modern attacks are increasingly distributed, adaptive, and temporally complex, while the widespread use of encryption limits direct payload inspection and shifts detection toward header, metadata,

and behavioral traffic analysis \cite{Sharma2025EncryptedSurvey,Dong2025PretrainingEncrypted}.

Signature-based systems such as Snort and Suricata are widely used in practice because they are interpretable, efficient, and effective against known threats \cite{Roesch1999,SuricataGuide2026}. These systems rely on manually designed rules to identify suspicious patterns in network traffic. Although rule-based detection remains valuable, it has inherent limitations: it depends on expert-maintained signatures, struggles with zero-day attacks and evolving variants, and may fail to capture attacks whose malicious behavior emerges only through temporal or contextual patterns across multiple flows \cite{SommerPaxson2010,Hozouri2025Survey}. As network traffic volume grows and attack behavior becomes more diverse, purely rule-based detection is insufficient as the sole detection mechanism.

Machine learning based NIDSs have been proposed to improve adaptability by learning decision boundaries from data rather than relying only on predefined rules. Classical approaches typically represent each flow using aggregated statistical features such as duration, packet counts, byte counts, packet rates, and inter-arrival-time statistics \cite{BuczakGuven2016,Ahmed2016Survey}. These flow-level features are compact and computationally efficient, making them attractive for large-scale detection. However, aggregation also removes important fine-grained information. In particular, packet ordering, burst patterns, direction changes, and irregular timing intervals may be lost when a complete flow is compressed into a single statistical vector. This loss is important because many attacks are not characterized only by total volume, but also by how packets evolve over time.

Deep learning approaches, including convolutional neural networks, recurrent neural networks, LSTMs, GRUs, and autoencoder-based models, have been explored to learn richer representations for intrusion detection \cite{Yin2017RNNIDS,Shone2018NNAE,Mirsky2018Kitsune}. These methods reduce the need for manual feature engineering and can model non-linear relationships in traffic data. Nevertheless, many existing deep learning NIDSs still operate primarily on flow-level aggregated inputs. Even when sequence models are used, they often model fixed-size flow feature sequences rather than the internal packet sequence of each flow. As a result, they only partially preserve the hierarchical structure of network traffic, where packets form flows and related flows form broader host-level or endpoint-level behaviors.

Transformer architectures provide a promising direction for this problem because self-attention can model long-range dependencies and process sequences efficiently in parallel \cite{Vaswani2017}. Recent Transformer-based NIDS studies have shown strong performance on network traffic classification and intrusion detection tasks \cite{Wu2022RTIDS,Han2023GTID,Manocchio2024FlowTransformer}. For example, time-aware Transformer designs demonstrate that temporal information can improve intrusion detection when traffic behavior depends on ordering and timing \cite{Han2023GTID}. FlowTransformer further shows the flexibility of Transformer-based architectures for flow-based NIDS evaluation \cite{Manocchio2024FlowTransformer}. However, many existing approaches still treat each flow as a flat representation and do not fully model the two-level structure of traffic: packet-level dynamics within a flow and contextual dependencies across flows.

This thesis addresses this gap by proposing a hierarchical two-stage framework for flow-level network intrusion detection. The starting point is raw PCAP traffic. A custom Suricata output plugin is used to extract two complementary views of the traffic: packet-level features and flow-level statistical features. Packet-level features preserve fine-grained temporal behavior, while flow-level statistical features summarize aggregate properties of each flow. After data cleaning and preprocessing, the first stage learns a flow representation using a time-aware packet-sequence Transformer. This Stage1 model processes packets within each flow as an ordered sequence and combines positional information with real-time temporal information. The learned packet-sequence representation is then fused with flow-level statistical features through a late-fusion mechanism to produce a final Stage1 embedding.

The second stage further models dependencies among flows. This is motivated by the observation that malicious behavior may not be fully visible from a single flow in isolation. For example, scanning, brute-force attempts, DDoS behavior, and multi-step intrusions often involve groups of related flows sharing temporal, source-host, destination-host, or endpoint-level context. Therefore, the Stage1 embeddings are used as inputs to a context-aware Stage2 Transformer. For each target flow, a context window is constructed using different strategies, including time-only context, source-host context, destination-host context, and endpoint-based context. The Stage2 model then learns whether the target flow is benign or malicious by considering both its own Stage1 representation and the behavior of related neighboring flows. This design is also related to recent work emphasizing cross-flow, graph-based, and contextual traffic modeling \cite{Deng2023FlowTopology,Zhong2024GNNSurvey,Li2024DoLLM}.

The central research problem of this thesis is therefore:

\begin{displayquote}
How can network intrusion detection benefit from packet-level temporal modeling and cross-flow contextual modeling while preserving the hierarchical structure of network traffic?
\end{displayquote}

To answer this question, the thesis studies two research questions:

```

\begin{itemize}
  \item \textbf{RQ1:} Does time-aware packet-sequence modeling produce more informative flow
representations than flow-level aggregation alone?
  \item \textbf{RQ2:} Does incorporating cross-flow context improve flow-level intrusion detection compared
with independent per-flow classification?
\end{itemize}

```

The main contributions of this thesis are as follows:

```

\begin{itemize}
  \item A PCAP-to-learning pipeline based on a custom Suricata plugin that extracts both packet-level and
flow-level CSV features for flow-level intrusion detection.
  \item A Stage1 time-aware Transformer that models packet sequences within each flow using both positional
and temporal information.
  \item A late-fusion design that combines learned packet-sequence embeddings with flow-level statistical
features.
  \item A Stage2 context-aware Transformer that models relationships among neighboring flows using time-
only, source-host, destination-host, and endpoint-based context construction strategies.
  \item A comprehensive empirical evaluation using macro-F1, macro-precision, macro-recall, ROC-AUC, PR-AUC,
confusion matrices, false positive rate, model parameters, training time, and inference time.
\end{itemize}

```

Because intrusion detection datasets are often highly imbalanced, this thesis does not rely on accuracy alone. Instead, macro-F1 is used as the primary F1 measure because it gives equal importance to benign and attack classes. Macro-precision and macro-recall are reported to analyze the trade-off between false alarms and missed attacks. PR-AUC is emphasized because it is especially informative under class imbalance, while ROC-AUC measures threshold-independent ranking performance. Confusion matrices and false positive rate are also analyzed because, in operational NIDS settings, excessive false positives can overwhelm analysts and reduce the practical usability of a detector.

The remainder of this thesis is organized as follows. The next section reviews related work on rule-based NIDSs, classical machine learning, deep learning, Transformer-based traffic modeling, encrypted traffic analysis, and context-aware intrusion detection. The following sections describe the data extraction pipeline, preprocessing procedure, Stage1 time-aware Transformer, Stage2 context-aware Transformer, experimental setup, evaluation metrics, empirical results, discussion, and conclusion.

用户 2026-07-14 09:32:24

请给我提供Related Work的完整版的latex，引入必要的论文。（尽量引用最新的论文，引用不少于25篇）

助手 2026-07-14 09:32:36

我会先核对你现有的 `references.bib` 引用键，再检索近几年与 Transformer NIDS、加密流量、跨流上下文和数据集评估直接相关的原始论文。最终我会给出可直接放进论文的完整 `Related Work` LaTeX，并补齐新增 BibTeX，确保正文中实际引用不少于 25 篇。

助手 2026-07-14 09:33:19

你现有文献库已经有足够的数量基础，但我发现几处需要顺手规避：`Hozouri2025Survey` 重复定义，`Dong2025PretrainingEncrypted` 与 `Dong2025EncryptedPretrain` 实际上是同一篇综述，`Kim2016LSTM` 的引用键与条目内容也不一致。正文会只使用可核对的条目，并把最新文献集中放在 Transformer、跨流上下文和评估可信度三个小节中。

助手 2026-07-14 09:34:25

我准备把结果整理成两个可直接使用的文件：`doc/related\_work.tex` 是完整正文，`doc/related\_work\_additions.bib` 只包含你原文献库缺少的新条目，避免直接破坏现有 proposal。正文会明确区分“流内 packet sequence 建模”和“跨 flow context

建模”，这是你的论文相对已有工作的关键定位。

助手 2026-07-14 09:36:31

正文已经完成，目前统计为约 2000 词、37 个不同引用键，并且所有正文引用都能在“现有`references.bib` + 新增 BibTeX”中找到。我正在做最后一轮学术准确性检查，重点看是否把某篇论文的模式类型或贡献描述得过头，以及新增条目能否直接合并编译。

助手 2026-07-14 09:37:37

已完成完整版 Related Work，正文约 2000 词，共引用 **36 个不同文献键，其中 35 篇为论文**，覆盖 2024–2026 年的最新研究。

- [related_work.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/related_work.tex)
- [related_work_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/related_work_additions.bib)

正文包含七个部分：

1. Signature-based 与统计流特征检测
2. CNN、LSTM、GRU 等深度序列模型
3. Time-aware 与 flow-based Transformer
4. 自监督学习和加密流量表示
5. 时间、主机、endpoint 与跨流上下文
6. 数据集、类别不平衡和评估可靠性
7. 研究缺口及你的两阶段模型定位

新增文献包括 [FlowTransformer](<https://www.sciencedirect.com/science/article/pii/S095741742303066X>)、[MFT](<https://www.sciencedirect.com/science/article/pii/S0167404824004796>)、[2026 flow-to-window Transformer](<https://www.sciencedirect.com/science/article/pii/S2542660526000090>)、[2026 contextualized NetFlow survey](<https://arxiv.org/abs/2602.05594>)、[最新 NIDS 数据集综述](<https://www.sciencedirect.com/science/article/pii/S0167404825001993>)以及 [USENIX Security 的 ML 评估规范](<https://www.usenix.org/conference/usenixsecurity22/presentation/arp>)。

使用时，将`related\_work.tex`的内容替换`main.tex`中原来的 Related Work，并把新增 BibTeX 条目合并进`references.bib`。目前已完成静态检查：

- 36 个正文引用键全部存在
- 新增 10 个 BibTeX 条目与原条目没有键名冲突
- LaTeX 花括号数量匹配
- 研究缺口与 Stage1/Stage2 实际设计一致

现有`references.bib`还需要删除一个重复的`Hozouri2025Survey`条目；另外两个`Dong2025...`键实际上指向同一篇论文，本文只使用`Dong2025EncryptedPretrain`。当前系统没有安装 LaTeX 编译器，因此尚未执行完整 PDF 编译。

用户 2026-07-14 09:57:26

上面中Datasets, Class Imbalance, and Evaluation Reliability这一部分使干嘛的？为什么要有他？详细解析

助手 2026-07-14 09:58:33

这一部分不是在介绍新模型，而是在回答一个更基础的问题：

> 你的实验结果为什么可信？0.98 的 Accuracy 是否真的说明模型能够检测攻击？

对 NIDS 论文来说，这一部分很重要，因为数据集、类别不平衡、数据泄漏和指标选择，往往比模型结构本身更容易导致“看起来非常好但实际不可靠”的结果。

****一、为什么要讨论 Datasets****

NIDS 模型高度依赖数据集，但公开数据集或自行生成的数据可能存在：

- 攻击类型覆盖不足
- 正常流量不够真实
- 标签错误或标签来源单一
- 重复或高度相似的流
- 同一个主机或攻击会话同时进入训练集和测试集
- 特征提取工具计算错误
- 数据采集时间过短
- 训练和测试来自同一个网络环境

这些问题会直接影响论文结论。

例如，如果来自同一条原始连接的相关 flow 被随机分到训练集和测试集，模型可能只是记住：

- IP 地址
- 端口组合
- 时间段
- 特定攻击脚本产生的固定特征
- 某个 PCAP 的环境特征

这时测试集 Accuracy 很高，但模型未必学到了可迁移的攻击行为。

你的数据处理流程是：

```
PCAP
-> Suricata custom plugin
-> packet CSV + flow CSV
-> preprocessing
-> Stage1
-> Stage2
```

因此你必须说明：

1. PCAP 的来源和攻击场景。
2. packet 与 flow 如何对应。
3. 标签如何产生。
4. Suricata alert 是否只用于生成标签。
5. alert、signature ID 等信息是否从输入特征中排除。
6. flow 如何划分为 train、validation 和 test。
7. 同一 flow 的 packet 是否严格位于同一个 split。
8. Stage2 的 context 是否跨越了数据集边界。

特别需要注意：你的 `packet\_label` 和 `flow\_label` 来源于 Suricata alert。规则告警信息可以用于生成监督标签，但不能再作为模型输入，否则模型可能只是重新学习 Suricata 的规则输出，构成严重的 label leakage。

****二、为什么要讨论 Class Imbalance****

你的测试数据明显不平衡。

以 Stage1 方案B为例：

```
Benign: 34,003
Attack:  1,407
Total:  35,410
```

攻击比例只有约：

$$\frac{1407}{35410} \approx 3.97\%$$

即使一个完全无用的模型把所有样本都预测成 benign，它仍然可以获得：

$$\frac{34003}{35410} \approx 96.03\%$$

因此，96% Accuracy 在这个数据集上几乎没有检测意义。

这正好可以解释你的 flow-level MLP 结果：

```
Accuracy      = 0.9627
Macro-F1      = 0.7707
Attack F1     = 0.5608
Attack Recall = 0.6438
PR-AUC        = 0.5604
FP            = 855
FN            = 467
```

Accuracy 看起来达到 96.27%，但它只比“全部预测为 benign”的 96.03% 高一点。Attack F1 只有 0.5608，而且产生了 855 个误报。这说明 Accuracy 掩盖了模型对少数攻击类别的不足。

相比之下，Stage1 Transformer 的结果是：

```
Accuracy      = 0.9830
Macro-F1      = 0.8882
Attack F1     = 0.7853
Attack Recall = 0.7825
PR-AUC        = 0.8564
FP            = 296
FN            = 306
```

真正重要的提升不是 Accuracy 从 96.27% 到 98.30%，而是：

- Macro-F1 从 0.7707 提升到 0.8882
- Attack F1 从 0.5608 提升到 0.7853
- PR-AUC 从 0.5604 提升到 0.8564
- FP 从 855 降到 296
- FN 从 467 降到 306

所以，Class Imbalance 这一部分是在解释为什么你的论文使用 Macro-F1、PR-AUC 和 confusion matrix，而不是只报告 Accuracy。

****三、为什么 Macro-F1 是主要指标****

你的定义是：

$$\begin{aligned} & \backslash[\\ & F1_{\mathrm{macro}} \\ & = \\ & \frac{F1_{\mathrm{benign}} + F1_{\mathrm{attack}}}{2} \\ & \backslash] \end{aligned}$$

它让 benign 和 attack 两个类别拥有相同权重，不会因为 benign 样本很多就主导结果。

而 weighted F1 是：

$$\begin{aligned} & \backslash[\\ & F1_{\mathrm{weighted}} \\ & = \\ & \frac{N_0 F1_0 + N_1 F1_1}{N_0 + N_1} \\ & \backslash] \end{aligned}$$

由于大约 96% 的样本是 benign，weighted F1 会非常接近 benign 类别表现。因此你可以报告 weighted F1，但不应该把它作为主要结论依据。

你的论文合理的指标优先级应该是：

1. Macro-F1：主要综合指标。
2. Macro-recall：两个类别的平均检测能力。
3. Macro-precision：两个类别的平均预测可靠性。
4. Attack-class F1、precision 和 recall：直接分析攻击检测。
5. PR-AUC：分析不平衡条件下的排序能力。
6. ROC-AUC：补充阈值无关性能。
7. Confusion matrix 和 FPR：分析实际误报与漏报。
8. Accuracy 和 weighted metrics：作为辅助指标。

****四、为什么 PR-AUC 很重要****

ROC-AUC 使用：

$$\begin{aligned} & \backslash[\\ & \mathrm{TPR} = \frac{TP}{TP + FN} \\ & \backslash] \end{aligned}$$

和：

$$\begin{aligned} & \backslash[\\ & \mathrm{FPR} = \frac{FP}{FP + TN} \\ & \backslash] \end{aligned}$$

当 benign 数量特别大时，即使 FP 数量不小，FPR 仍可能看起来很低。因此 ROC-AUC 在类别极不平衡时可能显得过于乐观。

PR-AUC 更直接关注：

$$\begin{aligned} & \backslash[\\ & \mathrm{Precision} \\ & = \\ & \frac{TP}{TP + FP} \\ & \backslash] \end{aligned}$$

和：

$$\begin{aligned} & \backslash[\\ & \backslashmathrm{Recall} \\ & = \\ & \backslashfrac{TP}{TP+FN} \\ & \backslash] \end{aligned}$$

也就是“检测出的攻击有多少是真的”以及“真实攻击有多少被找到”。

以 Stage2 数据为例，攻击比例约为：

$$\begin{aligned} & \backslash[\\ & \backslashfrac{3520}{84380}\backslashapprox4.17\% \\ & \backslash] \end{aligned}$$

随机排序模型的 PR-AUC 基准大约只有 0.0417，而你的 `source\_host` 模型达到 0.9345。这比单纯报告 ROC-AUC 0.9938 更能说明模型在少数攻击类别上的区分能力。

****五、为什么还需要 confusion matrix 和 FPR****

AUC 和 PR-AUC 是阈值无关指标，但模型最终部署时必须选择一个 threshold。选择阈值后，真正影响安全人员的是：

- FP：误报多少正常流量
- FN：漏掉多少攻击
- FPR：正常流量中有多少被错误告警
- Recall：攻击中有多少被检测出来

例如 Stage2：

```
source_host: CM = [[80421, 439], [527, 2993]]
endpoint:    CM = [[80158, 702], [401, 3119]]
```

对应的 FPR 为：

$$\begin{aligned} & \backslash[\\ & \backslashmathrm{FPR}_{\mathrm{source}} \\ & = \\ & \backslashfrac{439}{80421+439} \\ & \backslashapprox0.543\% \\ & \backslash] \end{aligned}$$

$$\begin{aligned} & \backslash[\\ & \backslashmathrm{FPR}_{\mathrm{endpoint}} \\ & = \\ & \backslashfrac{702}{80158+702} \\ & \backslashapprox0.868\% \\ & \backslash] \end{aligned}$$

`endpoint` 比 `source\_host` 少漏掉：

$$\begin{aligned} & \backslash[\\ & 527-401=126 \\ & \backslash] \end{aligned}$$

个攻击，但多产生：

\[
702-439=263
\]

个误报。

这反映了实际部署中的选择：

- `source\_host` 更均衡，误报更少，适合作为主模型。
- `endpoint` 更重视 Recall，适合漏报代价特别高的场景。

如果没有 confusion matrix 和 FPR，只看 AUC，两种方法几乎没有明显差别；但从运营角度看，它们的行为完全不同。

****六、Evaluation Reliability 主要防止哪些实验问题****

这一部分主要防止以下问题：

- 在整个数据集上先做标准化，再划分训练和测试。
- 使用测试集选择 threshold。
- 使用测试集选择最佳 epoch 或超参数。
- 同一 flow 的 packet 出现在不同 split。
- 相邻或重复 flow 同时出现在训练集和测试集。
- Stage2 的测试 flow 使用训练 flow 作为不合理上下文。
- 随机划分破坏时间顺序，使模型看到“未来信息”。
- 将 Suricata alert 或 rule ID 作为模型输入。
- 只报告最佳单次运行，没有随机种子或方差。
- 只报告 Accuracy，没有少数类指标。
- 不报告参数量、训练时间和推理时间。

这些问题一旦存在，评审会怀疑 Stage2 的提升来自数据泄漏，而不是 cross-flow context。

****七、它和论文其他章节是什么关系****

这部分放在 Related Work

中，是为了说明已有研究已经认识到数据集和评估偏差问题，并为你的实验设计提供文献依据。

但它不能代替 Methodology 和 Experimental Setup。相关内容应该分别出现：

- Related Work：说明已有研究发现了哪些数据和评估问题。
- Dataset and Preprocessing：说明你的数据、标签、特征和清洗方式。
- Experimental Setup：说明 split、scaler、threshold、随机种子和 context policy。
- Evaluation Metrics：给出 Macro-F1、PR-AUC、FPR 等公式。
- Results：用 confusion matrix 和指标比较模型。
- Threats to Validity：承认 Suricata 标签、单一数据来源和跨数据集泛化等剩余限制。

****结论****

这一小节的核心作用是证明：

> 论文的贡献不仅是模型取得了较高分数，而且这些分数是在合理的数据划分、类别不平衡处理、无测试集泄漏和完整指标报告条件下得到的。

对于你的论文，它尤其必要，因为：

- 攻击类别只有约 4%；
- Stage2 会主动访问相邻 flow，存在特殊的 context leakage 风险；

- 标签来源与 Suricata 告警相关；
- 不同 context method 的主要差异体现在 FP、FN 和 Recall，而不只是 Accuracy；
- 你的研究目标是实际 NIDS 检测，误报率具有直接运营意义。

因此建议保留这一部分，但把具体实现细节放到后面的 Dataset、Experimental Setup 和 Evaluation Metrics 章节中。

用户 2026-07-14 10:04:57

在上面的related work中加入建议加入对比表：

Method	Input Packet order	Explicit time Flow context	Payload Task
FlowTransformer	Flow statistics	No Limited	No No NIDS
GTID	Packets/session	Yes Yes No	Yes/derived NIDS
ET-BERT	Datagram tokens	Yes Limited No	Often payload-derived Traffic classification
Flow-sequence model	Flow records	Across flows	Temporal order Yes No NIDS
Proposed	Packet + flow stats	Yes Yes Yes	No direct payload NIDS

这个对比表是否可行？是否能让论文更好？还是冗余？详细解析

助手 2026-07-14 10:05:15

这个表的方向是对的，而且很适合你的论文，因为它能把研究缺口压缩成一眼可见的“两个层次”：流内 packet dynamics 和流间 flow context。不过当前版本有几处概念混在同一列里，我会先核对 FlowTransformer、GTID、ET-BERT 和 flow-window 模型的原始论文，再给你一个可直接放入 Related Work 的严谨版本。

助手 2026-07-14 10:06:17

核对后，原表需要做两处关键修正：FlowTransformer 实际上会把 flow records 组成序列并利用前序 flows，因此不能写成 `Flow context = No`；GTID 和 ET-BERT 都直接利用 payload 字节或其 n-gram/token 表示，因此 `Payload` 应明确写成 `Yes`。我会采用“流内 packet order”和“流间 context”分列的版本，并在表下注明这是结构性比较，不是跨数据集性能排名。

助手 2026-07-14 10:07:54

这个对比表非常可行，而且能明显增强论文，不属于冗余。不过原始版本需要修正，否则会错误描述 FlowTransformer、GTID 和 ET-BERT。我已经把修正版加入：

[related_work.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/related_work.tex)

位置在 `Temporal, Relational, and Cross-Flow Context` 小节之后、`Datasets, Class Imbalance, and Evaluation Reliability` 之前。

****为什么值得加入****

这张表不是比较谁的分数更高，而是在比较不同方法“到底建模了什么”。它能直观支持你的核心研究缺口：

- FlowTransformer：建模 flow 之间的序列关系，但不保留 flow 内部 packet order。
- GTID：建模 flow/session 内的 packet order 和 inter-packet time，但不建模跨 flow context，而且依赖 payload。
- ET-BERT：建模 raw datagram bytes 和 BURST 上下文，但主要目标是 encrypted traffic classification。

- Flow-to-window Transformer: 建模时间窗口内多个 flow 的关系, 但不恢复 flow 内部的 packet sequence。
- Proposed: 同时结合 flow 内 packet dynamics、真实时间、flow statistics 和跨 flow context。

因此, 它把你的两条研究问题直接可视化了:

- RQ1 对应 `Intra-flow packet order` 和 `Explicit elapsed time`。
- RQ2 对应 `Inter-flow context`。
- `Payload dependency` 说明你的模型适用于加密环境。
- `Primary input` 说明 late fusion 的位置。
- `Primary task` 防止把 traffic classification 和 NIDS 混为一谈。

****原表需要修正的地方****

1. FlowTransformer 的 `Flow context` 不能写 `No`

FlowTransformer 会把多个 flow records 组成序列, 使用 Transformer 捕获 flow 之间的关系, 并对最后一个 flow 进行分类。因此应写成:

Inter-flow context: Yes; ordered flow sequence

它不建模的是 flow 内部 packet sequence, 而不是完全没有 flow context。[FlowTransformer 原文](https://arxiv.org/abs/2304.14746)明确说明 flow records 被输入为序列, 并利用前序 flows 的关系进行分类。

2. `Packet order` 应改为 `Intra-flow packet order`

原来的 `Packet order` 含义不够明确。例如:

Flow-sequence model: Across flows

这把 packet order 和 flow order 混在了同一列。它们是两个不同层次:

Intra-flow packet order:
p1 -> p2 -> p3 within one flow

Inter-flow context:
f1 -> f2 -> f3 across related flows

你的创新点正是同时覆盖这两个层次, 因此必须分开。

3. `Explicit time` 不能用含糊的 `Limited`

需要区分:

- Positional order: 只知道谁先谁后。
- Temporal window: 只知道 flows 在同一时间窗口。
- Explicit elapsed time: 直接输入 timestamp、time difference 或 inter-arrival interval。

所以表中改成:

FlowTransformer:
Not explicitly encoded

GTID:
Yes; inter-packet intervals

ET-BERT:
No; order only

Flow-to-window Transformer:
Window-based order, not packet intervals

Proposed:

Yes; packet timestamps or intervals

这样能够突出你的 time-aware encoding 不是普通 positional encoding。

4. GTID 的 Payload 应明确写 `Yes`

GTID 分别处理 packet header 和 packet payload，并使用 n-gram 表示 payload。因此：

Payload dependency: Yes; payload n-grams

不能写成 `Yes/derived`，因为 derived 容易让读者误以为它没有使用 payload。[GTID 原文](https://www.sciencedirect.com/science/article/pii/S0167404823000810)明确说明其分别处理 packet header 和 packet payload。

5. ET-BERT 也属于 Payload-dependent

ET-BERT 使用 datagram bytes，将原始流量转换成 token，并在 BURST 层进行预训练。虽然 payload 是加密的，但模型仍然读取加密后的 payload bytes，因此应写成：

Payload dependency: Yes; encrypted bytes

“加密 payload” 依然是 payload。你的方法则是：

No direct payload

即只使用 packet header、metadata、timing 和 flow statistics。这是隐私、存储和部署上的实际差异。[ET-BERT 原文](https://arxiv.org/abs/2202.06335)说明其学习 byte-level 和 BURST-level representation。

6. `Flow-sequence model` 应替换为具体论文

泛称 `Flow-sequence model` 无法被引用，也容易让评审问 “具体是哪一个模型”。我替换为：

Flow-to-window Transformer

对应 2026 年的 flow-to-window aggregation 模型。它把 flow 当作 token，在固定时间窗口内建模 inter-flow context，与 Stage2 非常相关。[该论文](https://www.sciencedirect.com/science/article/pii/S2542660526000090)明确说明每个 flow 是一个 token，Transformer 在时间窗口内捕获多个 flows 的上下文关系。

****最终表格表达的核心结论****

修正后的比较关系是：

Method	Flow内packet顺序	显式真实时间	跨flow context	Payload
--- ---	--- ---	--- ---	--- ---	
FlowTransformer	No	No explicit encoding	Yes	No
GTID	Yes	Yes	No	Yes
ET-BERT	Byte/BURST order	No explicit elapsed time	No cross-flow	Yes
Flow-to-window	No	Window-level only	Yes	No
Proposed	Yes	Yes	Yes	No direct payload

这里最有价值的不是 Proposed 一行有很多 `Yes`，而是它显示：

> 现有方法通常分别解决 flow 内部序列建模或 flow 之间上下文建模，而你的方法通过 Stage1 和 Stage2 将两个层次组合起来。

这比笼统地写 “existing methods fail to model hierarchical traffic” 更有说服力。

****为什么不冗余****

只要正文和表格承担不同职责，就不会冗余：

- 正文解释每篇工作的思想、贡献和局限。
- 表格统一比较维度。
- 表后段落归纳 research gap。
- Methodology 再详细介绍你的 Stage1 和 Stage2。

我还在表前明确说明：

The comparison is qualitative rather than performance-based because the studies use different datasets, labels, preprocessing pipelines, and evaluation protocols.

这句话很重要。不能在这张表中加入 Accuracy、F1 或 AUC，因为不同论文的数据集、split、标签和任务不同，数值不可公平比较。

****是否需要加入更多模型****

目前五行已经足够，每一行代表一个不同研究方向：

- Flow-level Transformer
- Time-aware packet/session Transformer
- Pretrained encrypted-traffic Transformer
- Cross-flow temporal-window Transformer
- Proposed hierarchical model

继续加入十几个模型反而会降低可读性。LSTM、GRU、CNN-LSTM 等属于实验 baseline，更适合放在 Results 的模型性能表，而不是这张研究定位表。

因此结论是：这张表应该保留。经过修正后，它不是装饰性总结，而是 Related Work 中证明研究缺口、连接 RQ1/RQ2，并解释 Stage1/Stage2 设计动机的核心表格。

用户 2026-07-14 10:26:16

请给我提供下一章节的完整版的latex，引入必要的论文。

助手 2026-07-14 10:26:24

我先确认 proposal 中 Related Work 后面的章节顺序，并对照你当前代码里的数据表示、Stage1/Stage2 输入输出和 context policy。这样下一章会直接承接前文，而不是写成泛泛的 Transformer 背景。

助手 2026-07-14 10:28:40

proposal 原来的 Problem Formulation 有三个需要修正的地方：它把 flow statistics 重复拼到每个 packet 上，与现在的 late fusion 代码不一致；context 写成了前后双向邻居，会引入未来 flow，而你的实现只扫描历史 flow；损失函数被固定写成 focal loss，但代码实际支持 class-weighted cross-entropy、focal 及采样策略。新版会把这些改成与实现一致的正式数学定义，并加入无 payload、因果 context 和验证集选阈值等约束。

用户 2026-07-14 10:31:22

<turn_aborted>

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

</turn_aborted>

用户 2026-07-14 10:32:05

我现在想完成我的论文，我已经有了introduction和related work。请给我提供论文中下一章节的完整版的latex，引入必要的论文。

助手 2026-07-14 10:32:18

在你当前结构中，Introduction 和 Related Work 之后最合适的下一章是`Problem Formulation and System Overview`。这一章先精确定义 flow、packet sequence、Stage1 embedding、Stage2 context 和预测目标，再给出完整系统流程与研究问题映射，为后面的 Methodology 章节建立符号和边界；我会先核对 proposal 的原章节结构和代码里的实际数据流，避免公式与实现不一致。

用户 2026-07-14 10:33:10

<turn_aborted>

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

</turn_aborted>

用户 2026-07-14 10:34:07

我的说是论文应该包含那些模块？每个模块应该些什么？详细解析

助手 2026-07-14 10:36:06

你的硕士论文建议采用“研究问题驱动”的结构，而不是按代码目录或实验运行顺序组织。整篇论文应围绕两条主线展开：

- **RQ1: ** packet-level time-aware modeling 是否比 flow statistical aggregation 提供更好的表示？
- **RQ2: ** cross-flow context 是否比 independent flow classification 提高检测性能？

推荐结构如下。

前置部分

Abstract

建议最后写，约 250–350 词，包含：

1. 问题：flow aggregation 丢失 packet order 和 timing，独立 flow 分类忽略跨流攻击行为。
2. 方法：Suricata 特征提取、Stage1 time-aware Transformer、flow-statistics late fusion、Stage2 context Transformer。
3. 数据：数据来源、flow 数量、攻击比例和划分方式。
4. 主要结果：Stage1 和 Stage2 最重要的 Macro-F1、PR-AUC、FPR 提升。
5. 结论：RQ1 和 RQ2 是否得到支持。
6. 效率：参数量、训练时间和推理时间。

不要在 Abstract 中列出所有消融结果，只保留最核心的 3–5 个数字。

其他前置内容

- Keywords

- Acknowledgements

- Table of Contents
- List of Figures
- List of Tables
- List of Abbreviations

缩写表建议包括：NIDS、PCAP、DDoS、MLP、CNN、RNN、LSTM、GRU、FPR、ROC-AUC、PR-AUC、IAT
。

1. Introduction

你已经完成。最终应包含：

- 研究背景和实际问题
- signature-based NIDS 的作用与不足
- flow aggregation 的信息损失
- packet-level temporal modeling 的必要性
- cross-flow context 的必要性
- 研究问题 RQ1、RQ2
- 贡献列表
- 评价指标选择理由
- 论文结构

Introduction 主要回答：**为什么要做这项研究？**

2. Related Work

你已经完成。当前结构合理，包括：

- Signature-based 与 flow-statistical NIDS
- CNN、LSTM、GRU 和 autoencoder
- Transformer-based NIDS
- time-aware packet modeling
- encrypted traffic 与 self-supervised learning
- graph、temporal window 和 cross-flow context
- dataset reliability 和 evaluation pitfalls
- 对比表及 research gap

Related Work 主要回答：**其他人已经做了什么，还缺少什么？**

不要在这一章详细介绍自己的层数、学习率或数据处理代码。

3. Problem Formulation and System Overview

这是你下一步最应该完成的章节。

3.1 Problem Definition

定义任务为 binary flow-level intrusion detection:

$y_i \in \{0, 1\}$,

\]

其中 0 表示 benign flow, 1 表示 malicious flow。

定义每个 flow 的输入:

\[

$$\mathcal{F}_i = \left(X_i, \boldsymbol{\tau}_i, \mathbf{s}_i, \mathbf{m}_i \right)$$

 \]

其中:

- (X_i) : packet feature sequence
- $(\boldsymbol{\tau}_i)$: packet timing sequence
- $(\mathbf{s}_i)$: flow statistical features
- $(\mathbf{m}_i)$: source、destination、timestamp 等上下文元数据

需要明确: flow statistics 不应在公式中重复拼接到每个 packet, 因为你的方案B采用 late fusion。

3.2 Hierarchical Traffic Structure

解释网络流量的两层结构:

```

Packet level:
p1 -> p2 -> ... -> pT
      one flow

Flow level:
f1 -> f2 -> ... -> fi
      related flows
  
```

分别定义:

- intra-flow dependency
- inter-flow dependency
- target flow
- historical context flow

3.3 Stage1 Objective

定义 Stage1:

\[

$$\mathbf{z}^{(p)}_i = f_{\{\mathrm{packet}\}}(X_i, \boldsymbol{\tau}_i),$$

 \]

\[

$$\mathbf{z}^{(s)}_i = f_{\{\mathrm{stat}\}}(\mathbf{s}_i),$$

 \]

\[

$$\mathbf{z}^{(1)}_i =$$

 \]

$$\mathrm{Fuse} \left(\mathbf{z}^{(p)}_i, \mathbf{z}^{(s)}_i \right).$$

这里需要说明 $\mathbf{z}^{(1)}_i$ 是最终导出到 Stage2 的 embedding。

3.4 Stage2 Context Definition

定义不同 context relation:

$$\mathcal{C}^{(r)}_i = \left\{ j < i: R_r(i, j) = 1 \right\},$$

其中 $\mathcal{C}^{(r)}$ 可以是:

- `time\_only`
- `source\_host`
- `destination\_host`
- `endpoint`

必须使用 $j < i$, 表示只使用 target flow 之前的 flow, 符合当前 `online` context policy, 避免 future leakage.

3.5 Final Detection Function

$$\hat{y}_i = f_{\mathrm{Stage2}} \left(\mathbf{z}^{(1)}_i, \left\{ \mathbf{z}^{(1)}_j: j \in \mathcal{C}^{(r)}_i \right\} \right).$$

3.6 Research Questions Mapping

明确对应关系:

Research question	实验模块
RQ1	MLP、CNN、LSTM、GRU、encoding、sequence length、fusion
RQ2	no-context、context methods、window size、Stage2 architectures
Efficiency	Params、training time、inference time

这一章回答: **研究问题在数学上是什么, 整个系统如何分解? **

4. Dataset Construction and Preprocessing

这一章对你的论文尤其重要, 因为你不是直接使用别人提供的 CSV, 而是从 PCAP 自己抽取特征。

4.1 Data Source

写清楚:

- PCAP 从哪里获得
- 采集网络环境
- 采集时间范围
- 攻击类型
- 是否包含企业敏感流量
- 数据是否匿名化
- 数据使用许可

如果公司数据不能公开, 需要明确说明保密限制和复现限制。

4.2 Custom Suricata Feature Extraction

介绍:

```
PCAP
-> Suricata
-> custom-out plugin
-> packet CSV
-> flow CSV
```

需要写:

- 插件如何注册 packet logger 和 flow logger
- packet 与 flow 如何通过 `flow\_id` 对齐
- packet features
- flow statistical features
- timestamp 单位
- source/destination 标识
- label 生成方式

不要把所有 40 个特征都挤在正文。正文按类别介绍, 完整 feature list 放 Appendix。

4.3 Label Construction

你需要明确说明:

- `packet\_label` 来自 packet 是否触发 Suricata alert
- 如果任意 packet 被标为 malicious, 则对应 `flow\_label=1`
- alert count、signature ID、rule message 等不作为模型输入

这能证明模型没有直接读取 Suricata 的检测结论。

同时必须承认: Suricata-derived labels 可能继承规则覆盖不足和误报偏差, 这应在 Limitations 再讨论。

4.4 Data Cleaning

包括:

- missing values
- infinite values
- duplicated flows
- invalid timestamps
- empty flows
- packet/flow mismatch
- categorical encoding
- numerical clipping
- `log1p`
- scaling
- padding 和 mask

4.5 Sequence Construction

写清楚：

- packet 按 timestamp 排序
- `max\_seq\_len`
- `head`、`head\_tail`、`random` 的定义
- 不足长度如何 padding
- padding 是否进入 attention
- 时间序列如何处理

4.6 Dataset Splitting

必须说明：

- chronological split 或其他 split
- train/validation/test 比例
- scaler 只在 training set 上拟合
- threshold 只在 validation set 上选择
- 同一 flow 的 packet 不得跨 split
- Stage2 context 如何防止 future leakage

4.7 Dataset Statistics

至少提供：

- benign/attack 数量
- class ratio
- packet count per flow
- flow duration distribution
- sequence truncation ratio
- 各 split 的数量
- 每个 context method 的可用 context 比例

推荐一张 dataset summary table 和两到三张分布图。

这一章回答：**数据是怎样从 PCAP 变成可信模型输入的？**

5. Proposed Methodology

这是论文的技术核心。

5.1 Overall Architecture

提供完整架构图：

```
PCAP
-> packet/flow extraction
-> Stage1 packet Transformer
-> attention pooling
-> flow-statistics late fusion
-> Stage1 embedding
-> context construction
-> Stage2 Transformer
-> classification
```

说明 Stage1 与 Stage2 是顺序训练还是端到端训练。按照当前实现，应写成先训练 Stage1、导出 embedding，再训练 Stage2。

5.2 Packet Feature Projection

```
\[
\mathbf{e}_{i,t}
=
W_x\mathbf{x}_{i,t}+\mathbf{b}_x.
\]
```

解释输入维度、投影维度和 padding mask。

5.3 Positional and Time-Aware Encoding

分别定义：

- packet position
- cumulative elapsed time
- local inter-arrival time
- `log1p` 时间转换
- position/time fusion

必须说明 position 和 elapsed time 的差别：

- position 表示第几个 packet
- time 表示 packets 实际间隔多久

5.4 Intra-Flow Transformer

介绍：

- multi-head self-attention
- feed-forward network
- residual connection
- layer normalization
- padding mask
- encoder layers

基础 Transformer 可引用 `Vaswani2017`，time-aware motivation 可引用 `Han2023GTID`。

5.5 Attention Pooling

给出：

$$\begin{aligned} & \backslash[\\ & a_{i,t} \\ & = \\ & \frac{\exp(q(\mathbf{h}_{i,t}))}{\sum_{k:m_{i,k}=1}\exp(q(\mathbf{h}_{i,k}))}, \\ & \backslash] \\ & \backslash[\\ & \mathbf{z}^{p}_i \\ & = \\ & \sum_t a_{i,t} \mathbf{h}_{i,t}. \\ & \backslash] \end{aligned}$$

说明 padding packet 不参与 pooling。

5.6 Flow Statistical Encoder and Late Fusion

定义 flow encoder:

$$\begin{aligned} & \backslash[\\ & \mathbf{z}^s_i = f_s(\mathbf{s}_i). \\ & \backslash] \end{aligned}$$

然后介绍你的实际 fusion method。若最终方案使用 gated fusion:

$$\begin{aligned} & \backslash[\\ & \mathbf{g}_i \\ & = \\ & \sigma \\ & \left(W_g[\mathbf{z}^p_i; \mathbf{z}^s_i] \right), \\ & \backslash] \\ & \backslash[\\ & \mathbf{z}^{(1)}_i \\ & = \\ & \mathbf{g}_i \odot \mathbf{z}^p_i + \\ & (1 - \mathbf{g}_i) \odot \mathbf{z}^s_i. \\ & \backslash] \end{aligned}$$

需要明确方案A/B/C各自代表什么，但主方法只详细介绍最终选择的方案，其他方案放到 ablation。

5.7 Context Construction

逐一解释:

- `time\_only`: 最近的历史 flows
- `source\_host`: 相同 source 的历史 flows
- `destination\_host`: 相同 destination 的历史 flows
- `endpoint`: 共享 source 或 destination endpoint

说明:

- context 按时间顺序排列
- target 位于最后一个有效位置

- context window size
- deduplication
- padding 和 mask
- `online`、`split\_isolated` 等 policy

5.8 Stage2 Context Transformer

定义：

$$\begin{aligned} & \backslash[\\ & H_i^{\{2\}} \\ & = \\ & \mathrm{Transformer}_{\{2\}} \\ & \left(\right. \\ & \mathbf{z}^{\{1\}}_{j_1}, \\ & \dots, \\ & \mathbf{z}^{\{1\}}_i \\ & \left. \right). \\ & \backslash] \end{aligned}$$

说明如何选取 target token，以及最终 classifier 如何输出 logits。

5.9 Loss Function

当前实验使用 focal loss，因此可以给出：

$$\begin{aligned} & \backslash[\\ & \mathcal{L}_{\{\mathrm{focal}\}} \\ & = \\ & -\sum_i \\ & \alpha_{y_i} \\ & (1-p_{i,y_i})^{\gamma} \\ & \log p_{i,y_i}. \\ & \backslash] \end{aligned}$$

引用 `Lin2017FocalLoss`。

同时说明：

- γ
- 是否使用 class alpha
- label smoothing
- weight decay
- Stage1 和 Stage2 是否使用相同 loss

5.10 Decision Threshold

定义：

$$\begin{aligned} & \backslash[\\ & \hat{y}_i = \\ & \mathbb{I}[p(y_i=1) \geq \delta]. \\ & \backslash] \end{aligned}$$

明确 threshold δ 在 validation set 上按照 attack F1 或指定指标选择，不能使用 test set。

这一章回答: **模型具体如何工作? **

6. Experimental Methodology

6.1 Experimental Environment

报告:

- CPU、GPU、RAM
- operating system
- Python、PyTorch、CUDA 版本
- batch size
- epoch
- optimizer
- learning rate
- weight decay
- early stopping
- random seeds

6.2 Baselines

Stage1 baselines:

- flow-statistics MLP
- CNN1D
- LSTM
- GRU
- BiLSTM
- CNN1D with time
- LSTM with time
- no encoding
- position only
- time only
- both encoding

Stage2 baselines:

- Stage1-only/no-context
- no-context MLP
- LSTM
- GRU
- CNN+LSTM
- Transformer

6.3 Ablation Studies

Stage1:

- scheme A/B/C
- position/time encoding
- max sequence length
- truncation strategy
- flow-feature fusion

Stage2:

- context method
- window size
- architecture
- include/exclude context
- context policy

6.4 Metrics

完整给出:

```
\[
\mathrm{FPR}=\frac{FP}{FP+TN}.
\]
```

将 Macro-F1 设为主要指标, 同时报告:

- Macro-precision
- Macro-recall
- Attack F1/precision/recall
- ROC-AUC
- PR-AUC
- Confusion matrix
- FPR
- Accuracy 和 weighted metrics

解释为什么不以 Accuracy 为主。

6.5 Efficiency Metrics

报告:

- total/trainable parameters
- training time
- test inference time
- milliseconds per flow
- flows per second
- peak GPU memory, 能够统计时建议加入

6.6 Statistical Reliability

理想情况下每个主要模型运行 3–5 个随机种子, 报告:

```
\[
\text{mean}\pm\text{standard deviation}.
\]
```

至少对以下比较进行重复运行:

- MLP vs Stage1 best
- Stage1-only/no-context vs Stage2 best
- Stage2 Transformer vs LSTM/GRU/CNN+LSTM

这一章回答: **实验怎样公平、可重复地进行? **

7. Results

结果章节必须围绕 RQ 组织。

7.1 Stage1 Results: RQ1

建议顺序：

1. Flow MLP vs packet-sequence models
2. Different Stage1 architectures
3. Position/time encoding ablation
4. `max\_seq\_len`
5. truncation strategy
6. flow fusion schemes
7. Stage1 efficiency

最重要结论应是：

- packet modeling 是否优于 flow-only MLP
- BothEncoding 是否优于 NoEncoding、TimeOnly、PositionOnly
- `max\_seq\_len=128` 为什么最好
- `head` 为什么作为最终策略

方案A/B/C如果 test set 不相同，不能直接声称 C 一定优于 B。应分别报告，或统一数据划分后再比较。

7.2 Stage2 Results: RQ2

建议顺序：

1. Stage1-only/no-context vs Stage2
2. context method comparison
3. window size
4. Transformer vs LSTM/GRU/CNN+LSTM
5. Stage2 efficiency

需要重点解释：

- `source\_host` 为什么综合表现最好
- `endpoint` 为什么 recall 高但 FPR 高
- `time\_only` 为什么不如 host-aware context
- window 128 为什么优于 16、32、64、256

7.3 End-to-End Improvement

必须提供同一 test set 上：

```
Flow MLP
Stage1-only
Stage2 no-context
Stage2 best context
```

这是整篇论文最重要的结果表，因为它展示完整贡献链：

```
flow statistics
-> packet temporal modeling
-> late fusion
-> cross-flow context
```

7.4 Error Analysis

建议分析：

- FP 最多的正常 flow 类型
- FN 最多的攻击类型
- 短 flow 和长 flow 的差异
- context 不足的 flows
- source/destination 新主机
- 模型置信度接近 threshold 的样本

7.5 Efficiency Results

比较性能收益与代价。例如：

- Stage2 提高多少 Macro-F1
- 参数量增加多少
- inference time 增加多少
- 是否适合在线部署

Results 主要回答：**实验发生了什么？**

8. Discussion

这一章不能只是重复结果数字。

8.1 Answer to RQ1

综合解释 packet order、time encoding 和 flow fusion 为什么有效，并用最核心结果回答 RQ1。

8.2 Answer to RQ2

解释为什么 host-aware context 有效，以及 source、destination、endpoint 对应什么攻击行为。

8.3 Precision-Recall Trade-Off

结合 confusion matrix 和 FPR 讨论：

- 哪个配置适合低误报部署
- 哪个配置适合高召回安全场景
- threshold 如何改变结果

8.4 Operational Implications

讨论：

- PCAP 离线处理与在线部署差异
- Suricata 集成方式
- context history 的内存开销
- 新主机 cold start
- 处理吞吐量
- 是否依赖 payload
- 加密环境的适用性

8.5 Comparison With Literature

回到 Related Work 的对比表，说明你的结果如何支持或补充 GTID、FlowTransformer 和 flow-window modeling，但不要跨数据集直接比较 F1 数字。

Discussion 回答：**为什么会得到这些结果，它们意味着什么？**

9. Threats to Validity and Limitations

建议分为：

Internal Validity

- threshold selection
- preprocessing leakage
- context leakage
- random seed variability
- hyperparameter search bias

Construct Validity

- Suricata alert 是否等同于 ground truth
- binary label 是否掩盖攻击类型差异
- Macro-F1 是否完全代表运营价值

External Validity

- 单一公司或数据来源
- 单一网络环境
- 跨数据集泛化未验证
- 未知攻击和 zero-day 未充分测试

Operational Limitations

- context storage
- online latency
- 长 flow 截断
- no-payload 模型可能遗漏依赖内容的攻击
- Stage1/Stage2 分开训练可能不是全局最优

这一章不是削弱论文，而是证明你理解结论的适用边界。

10. Conclusion and Future Work

Conclusion

简洁总结：

- 研究问题
- 两阶段方法
- RQ1 的答案
- RQ2 的答案
- 最重要的性能和效率结果
- 实际意义

不要引入新实验或新引用。

Future Work

可以包括：

- multiclass attack classification
- unseen/zero-day detection
- cross-dataset evaluation
- self-supervised pretraining
- end-to-end Stage1/Stage2 training
- graph-based context
- online concept drift adaptation
- model calibration
- attention/SHAP explainability
- pruning、quantization 和 distillation

Appendices

建议包含：

- 完整 packet feature list
- 完整 flow feature list
- Suricata CSV schema
- 配置文件和超参数
- 额外 confusion matrices
- 所有随机种子结果
- 未放入正文的消融实验
- 模型参数明细
- 数据清洗规则
- context construction pseudocode

你目前最缺少的关键内容

在已有 Introduction、Related Work 和现有实验基础上，完成论文前最重要的是：

1. `Problem Formulation and System Overview`
2. PCAP-Suricata-CSV 的完整数据章节
3. 与代码一致的 Stage1/Stage2 Methodology
4. 同一 test set 上的 MLP → Stage1 → Stage2 end-to-end 对比
5. Stage2 no-context、LSTM、GRU、CNN+LSTM、Transformer 对比
6. 多随机种子或置信区间
7. error analysis
8. Params、training time、inference time
9. leakage control 和 threshold selection 说明
10. Discussion 与 Threats to Validity

推荐最终正文约 65–90 页。下一章应正式写 `Problem Formulation and System Overview`，然后再进入 `Dataset Construction and Preprocessing`。

用户 2026-07-14 10:41:44

请给我提供Problem Formulation and System Overview的完整latex，并引入必要的论文。

助手 2026-07-14 10:43:08

从 proposal 原有的 `Problem Formulation` 看，最需要修正的是三点：flow statistics 不能再被写成每个 packet 的组成部分，因为当前主方案是 late fusion；Stage2 context 不能写成对称的 $\{(i-K, \dots, i+K)\}$ ，因为代码采用只访问历史 flow 的 online policy；Stage1 和 Stage2 也应写成分阶段优化，而不是默认端到端联合训练。接下来我会按这些实际实现重写完整章节，并加入系统流程图、符号表和 RQ 映射。

助手 2026-07-14 10:45:36

章节正文已经生成，包含完整的系统流程图、符号表、Stage1/Stage2 数学定义、四种 context relation、分阶段训练目标、validation-only threshold 规则以及 RQ1/RQ2 对应表。现在我在做 LaTeX 结构和引用键检查，同时检查是否需要在现有 preamble 中补充宏包。

助手 2026-07-14 10:46:21

已完成完整版章节：

[problem_formulation_system_overview.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/problem_formulation_system_overview.tex)

章节约 2380 词，引用 11 篇相关论文，所有引用键均可在现有

[references.bib](C:/Users/XiaoyanXiong/Downloads/references.bib) 与 [related_work_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/related_work_additions.bib) 中找到。

内容包括：

- Detection setting 与六项设计要求
- PCAP 到最终预测的完整 TikZ 系统流程图
- packet、flow、metadata、context 和 embedding 符号表
- Stage1 packet projection、time encoding、Transformer、attention pooling 与 late fusion
- Stage1 独立分类目标
- `time\_only`、`source\_host`、`destination\_host`、`endpoint` 的数学定义
- 只使用历史 flow 的 causal online context
- target flow 作为最后一个有效 token
- Stage2 context-aware 分类与 no-context control
- focal loss 与分阶段优化
- validation-only threshold selection
- RQ1、RQ2、效率实验的对应表

相较 proposal 中原来的 Problem Formulation，新版本修正了几个重要问题：

1. Flow statistics 被定义为独立向量 $\{\mathbf{s}_i\}$ ，在 packet Transformer 之后 late fusion，而不是重复拼接到每个 packet。
2. Stage2 context 定义为 $\{j < i\}$ 的历史 flows，不再使用可能包含未来信息的 $\{(i-K, \dots, i+K)\}$ 。
3. 明确 Stage1 先训练并导出 embedding，Stage2 随后独立训练，不假设端到端反向传播。
4. source、destination 和 timestamp 作为 context construction metadata，不与标签或 Suricata alert 混淆。
5. 明确 threshold 只能在 validation set 上选择。

静态检查已经通过：LaTeX 花括号匹配、所有环境成对、没有缺失引用键。你的 `main.tex` 现有宏包基本满足要求，但需要额外加入：

```
\usepackage{amssymb}
```

因为章节使用了 $\mathbb{R}$ 、 $\mathbb{N}$ 和 $\mathbb{I}$ 。下一章应接续编写 `Dataset Construction and Preprocessing`，详细描述 PCAP、Suricata 插件、标签生成、特征定义、清洗、划分与防止数据泄漏。

用户 2026-07-14 11:44:09

我是用的PCAP是ar002_et12_20260511_001.pcap(8,62 GB),公司同事从公司网络上使用工具download下来的。
生成的CSV文件中的相关信息: flows shape = (421898, 77)

[INFO]flows 原始列名:

```
['flow_id', 'flow_start_timestamp_us', 'flow_end_timestamp_us', 'source_ip', 'source_port', 'destination_ip', 'destination_port', 'protocol', 'flow_duration', 'total_fwd_packets', 'total_backward_packets', 'total_length_of_fwd_packets', 'total_length_of_bwd_packets', 'fwd_packet_length_max', 'fwd_packet_length_min', 'fwd_packet_length_mean', 'fwd_packet_length_std', 'bwd_packet_length_max', 'bwd_packet_length_min', 'bwd_packet_length_mean', 'bwd_packet_length_std', 'flow_bytes_per_s', 'flow_packets_per_s', 'flow_iat_mean', 'flow_iat_std', 'flow_iat_max', 'flow_iat_min', 'fwd_iat_total', 'fwd_iat_mean', 'fwd_iat_std', 'fwd_iat_max', 'fwd_iat_min', 'bwd_iat_total', 'bwd_iat_mean', 'bwd_iat_std', 'bwd_iat_max', 'bwd_iat_min', 'fwd_psh_flags', 'bwd_psh_flags', 'fwd_urg_flags', 'bwd_urg_flags', 'fwd_header_length', 'bwd_header_length', 'fwd_packets_per_s', 'bwd_packets_per_s', 'min_packet_length', 'max_packet_length', 'packet_length_mean', 'packet_length_std', 'packet_length_variance', 'fin_flag_count', 'syn_flag_count', 'rst_flag_count', 'psh_flag_count', 'ack_flag_count', 'urg_flag_count', 'cwe_flag_count', 'ece_flag_count', 'down_up_ratio', 'average_packet_size', 'avg_fwd_segment_size', 'avg_bwd_segment_size', 'has_init_win_bytes_forward', 'init_win_bytes_forward', 'has_init_win_bytes_backward', 'init_win_bytes_backward', 'act_data_pkt_fwd', 'min_seg_size_forward', 'active_mean', 'active_std', 'active_max', 'active_min', 'idle_mean', 'idle_std', 'idle_max', 'idle_min', 'label']
```

packets shape = (11750727, 40)

[INFO]packets 原始列名:

```
['record_type', 'packet_id', 'timestamp_us', 'ts_sec', 'ts_usec', 'flow_id', 'direction', 'ip_version', 'src_ip', 'dst_ip', 'src_port', 'dst_port', 'protocol', 'pkt_len', 'ip_len', 'payload_len', 'ttl_or_hop_limit', 'has_tcp', 'has_udp', 'has_icmp', 'tcp_seq', 'tcp_ack', 'tcp_flags', 'tcp_fin', 'tcp_syn', 'tcp_rst', 'tcp_psh', 'tcp_ack_flag', 'tcp_urg', 'tcp_ece', 'tcp_cwr', 'tcp_window', 'tcp_header_len', 'udp_len', 'icmp_type', 'icmp_code', 'l7_proto', 'flow_iat_us', 'direction_iat_us', 'packet_label']
```

标签分布

label

0 405254

1 16644

Name: count, dtype: int64

结合上面我提供的信息

请给我提供Dataset Construction and Preprocessing的完整latex，并引入必要的论文。

助手 2026-07-14 11:44:23

我会把这一章严格写成“可复现的数据构建过程”，并区分三类信息：你已经确认的事实、代码中能够核对的预处理步骤，以及目前仍需在最终论文中标注为数据来源限制的内容。重点会检查 Suricata 标签生成、packet/flow 对齐、实际使用和排除的特征、chronological split、scaler 拟合范围、序列截断与 Stage2 metadata，避免把 77/40 个原始字段误写成全部模型输入。

助手 2026-07-14 11:49:18

我会把这一章写成可直接并入论文的英文 LaTeX，并严格区分三件事：原始 CSV 中“可用的字段”、实际进入 Stage 1 的“模型特征”，以及仅用于 Stage 2 构建上下文的“主机与时间标识”。我也会把 Suricata 告警派生标签明确写成弱标签，避免把“未触发告警”误表述为经过人工确认的 benign ground truth。

助手 2026-07-14 11:49:42

代码与数据格式已经对上：当前配置实际使用 23 个原始 packet predictors，`flow\_iat\_us` 另外进入时间编码；IP、端口、绝对时间戳和标签都不会作为 Stage 1 的普通输入。原始 flow 表则保留完整统计字段，最终 fusion 配置从中选择数值和二元特征；`source\_ip`、`destination\_ip` 只作为 Stage 2 的关系键。

助手 2026-07-14 11:51:46

接下来我会生成两个文件：章节正文和本章新增的 BibTeX。正文会包含数据来源、双层 CSV 构建、告警标签公式、完整字段分组、清洗与对齐、特征选择、数值/类别变换、128-packet 序列化、70/10/20 时间切分、隐私与有效性限制，并把已知数字与尚需记录的 Suricata/ruleset 元数据分开写清楚。

助手 2026-07-14 11:55:01

正文已经写入。现在我在做最后一轮技术校验：检查所有 `cite{}` 是否能在三份 `bib` 中解析、表格字段是否与实际 40/77 列 CSV 头完全一致，以及 LaTeX 的括号、环境和新增宏包要求是否完整。

助手 2026-07-14 11:56:53

校验结果是干净的：本章共覆盖实际 CSV 的 40 个 packet 列和 77 个 flow 列，12 个引用键全部可解析，LaTeX 环境全部成对，主配置的切分数量也与 $421 \times 70/10/20\%$ 一致。唯一的既有问题在原 `references.bib`：`Hozouri2025Survey` 已经重复定义两次；这不是本章新增的，但正式编译前应删除其中一个。

助手 2026-07-14 11:57:38

已完成英文版 LaTeX 章节：

- [dataset_construction_preprocessing.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/dataset_construction_preprocessing.tex:12)
- [dataset_preprocessing_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/dataset_preprocessing_additions.bib:1)

****章节内容****

- PCAP 来源、规模及专有数据范围
- Suricata 自定义 packet/flow logger 数据生成流程
- 40 个 packet 字段与 77 个 flow 字段的完整分组表
- Suricata 告警派生标签的数学定义
- 弱标签的适用范围及潜在噪声
- 数据清洗、去重和 packet-flow 对齐
- Stage 1 实际使用的 23 个 packet predictors
- flow statistical features 与 Stage 1 late fusion 的关系
- IP 和时间信息在 Stage 1、Stage 2 中的不同用途
- `log1p`、0.1%/99.9% clipping、RobustScaler 和 one-hot encoding
- `max\_seq\_len=128`、`head/head\_tail/random` 序列构建
- 70%/10%/20% chronological split

- 数据不平衡、隐私、单一 PCAP 和标签有效性限制

完整数据切分为：

```
| Split | Flows |  
|---|---:|  
| Train | 295,329 |  
| Validation | 42,189 |  
| Test | 84,380 |  
| Total | 421,898 |
```

****并入主论文****

在 `main.tex` 的 preamble 中补充：

```
\usepackage{amssymb}  
\usepackage{longtable}
```

正文中引入该章节，并让 bibliography 同时加载：

```
\bibliography{  
  references,  
  doc/related_work_additions,  
  doc/dataset_preprocessing_additions  
}
```

需要注意，原始 `references.bib` 中的 `Hozouri2025Survey` 在第 94 和 254 行重复定义，正式编译前应删除其中一份。

本章新增或使用的核心文献已经根据[Suricata

官方插件接口](<https://docs.suricata.io/en/suricata-8.0.5/devguide/extending/output/>)、[Shiravi 等人的数据集构建方法](<https://www.sciencedirect.com/science/article/pii/S0167404811001672>)、[CICIDS2017 数据集论文](<https://www.scitepress.org/PublishedPapers/2018/66398/>)、[安全机器学习实验规范](<https://www.usenix.org/conference/usenixsecurity22/presentation/arp>)和[弱监督研究](<https://pmc.ncbi.nlm.nih.gov/articles/PMC5951191/>)核对。

最后有一个重要的一致性問題：完整数据集的测试集是 ****84,380 flows****。此前测试集为 ****35,410 flows**** 的 Stage 1 结果不能直接与该完整数据集结果放在同一主对比表中，除非确认它们来自完全相同的数据范围和 split。否则应将 35,410-flow 实验标记为早期子集实验或重新运行。当前环境没有 LaTeX 编译器，因此完成了字段、引用和环境静态检查，但未生成最终 PDF。

用户 2026-07-14 13:43:31

请给我提供

5. Proposed Methodology

的完整latex，并引入必要的论文。

助手 2026-07-14 13:43:51

我会把第 5 章写成真正对应当前实现的 Methodology，而不是泛泛介绍 Transformer。重点会逐项核对 Stage 1 的 packet/time encoding、统计特征融合、Stage 2 的四种 context 构建方法、因果掩码、训练目标和两阶段接口，并明确哪些配置属于主模型、哪些只用于消融实验。

助手 2026-07-14 13:44:22

目前主模型的关键实现已经确认：Stage 1 不是简单的 attention pooling，而是 attention、mean、max 三路汇聚后再投影；方案 C 使用带残差上限的 flow-conditioned token FiLM，在 Transformer 前用统计特征轻量调制 packet tokens。Stage 2 则把目标 flow 固定放在最后一个有效位置，并采用左侧 padding，这些实现细节会直接写进公式和算法中。

助手 2026-07-14 13:47:14

章节的主线已经定下来：以方案 C 的 `token\_film` 作为 Stage 1 主模型，以 `target\_query\_gated` 作为 Stage 2 主模型；普通 Transformer、no-context、LSTM、GRU 和 CNN+LSTM 会被明确标为对照模型。这样论文不会把当前代码里的主模型和实验基线混在一起，也不会把 `time\_only` 误写成连续时间编码，它实际上是按时间排序的全局近期上下文。

助手 2026-07-14 13:48:06

现在开始写正文。除了主模型公式，我会专门加入一段“术语与实现一致性”：当前方案 C 虽然保持 packet/flow 两条独立分支，但 `token\_film` 在 self-attention 前调制 token，因此严格来说属于 conditional intermediate fusion，而不是 post-pooling late fusion。这一点写清楚，能避免答辩时被评审抓住架构表述和代码不一致。

助手 2026-07-14 13:51:42

正文已写完，正在做一致性校验。我会同时检查章节内部的 equation/figure/table 标签是否重复，以及它与前两章之间是否出现同名 `\label{}`；这种问题单独编译章节时看不出来，但合并整篇论文后会造成交叉引用错乱。

助手 2026-07-14 13:52:36

校验发现并修复了本章与旧版 `problem\_formulation.tex` 的一个 equation label 冲突。当前本章自己的 59 个交叉引用标签均唯一；项目里仍有两个旧 Problem Formulation 文件都使用 `sec:problem\_formulation`，最终主文档只能输入其中一个，建议保留较完整的 `problem\_formulation\_system\_overview.tex`。

助手 2026-07-14 13:53:26

已完成完整英文 LaTeX 章节：

- [proposed_methodology.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/proposed_methodology.tex:12)
- [proposed_methodology_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/proposed_methodology_additions.bib:1)

本章约 3,700 词，包含 13 个文献引用，主要包括：

- 两阶段总体架构与数学定义
- Stage 1 packet-record projection
- cumulative time-aware sinusoidal encoding
- flow statistical feature bottleneck encoder
- 带 residual cap 的 token-FiLM
- 两层 packet Transformer
- attention、mean、max 三路混合池化
- 方案 A、B、C 的准确区别

- 四种 Stage 2 context relation
- online causal context policy
- context-age positional encoding
- `target\_query\_gated` 多头注意力
- no-context、Transformer、LSTM、GRU、CNN+LSTM 对照模型
- focal loss、controlled weighted sampling、AdamW
- 两阶段顺序训练和 embedding export
- validation-only threshold selection
- 主配置表和计算复杂度分析

****需要统一的术语****

代码中的主方案 C 使用：

```
method: token_film
```

统计特征在 Transformer 之前调制 packet tokens。因此严格来说，它属于：

> separate-branch conditional intermediate fusion

而不是严格的 post-pooling late fusion。本章已经按代码准确描述。Introduction 和 System Overview 中的“late fusion”后续也应改成该术语，或者把代码改成真正的 pooling 后 fusion。

Stage 2 主模型同样不是普通的全序列 Transformer，而是：

```
model_type: target_query_gated
```

即目标 flow 作为 query，历史 flows 作为 keys/values，再通过 gate 注入上下文。普通 Transformer、LSTM、GRU 和 CNN+LSTM 被准确归为对照模型。

此外，本章明确指出：

- `time\_only` 表示全局时间顺序下的近期 flows；
- 它目前没有显式编码 flow 之间的真实微秒时间差；
- Stage 2 使用的是 `age` positional encoding；
- Stage 1 才使用真实 packet inter-arrival time。

引用已根据 [Transformer 原论文](https://arxiv.org/abs/1706.03762)、[GTID](https://www.sciencedirect.com/science/article/pii/S0167404823000810)、[FiLM](https://ojs.aaai.org/index.php/AAAI/article/download/1671/11530)、[AdamW](https://openreview.net/forum?id=Bkg6RiCqY7)及[原始 CNN 序列分类论文](https://aclanthology.org/D14-1181/)核对。

并入主文档时需要：

```
\usepackage{amssymb}

\input{doc/proposed_methodology}

\bibliography{
  references,
  doc/related_work_additions,
  doc/proposed_methodology_additions
}
```

所有本章引用、环境和交叉引用已通过静态检查。当前系统没有 LaTeX 编译器，因此尚未执行最终 PDF 编译。原 `references.bib` 中重复的 `Hozouri2025Survey` 仍需删除一份。下一章应自然衔接 ****Experimental Setup and Evaluation Protocol****。

用户 2026-07-14 15:14:10

请给我提供Experimental Methodology的完整latex，并引入必要的论文。

助手 2026-07-14 15:14:19

我会把这一章写成可直接接在`Proposed Methodology`后面的完整实验方法章节，并严格区分“实验设置/评价协议”和下一章的“结果分析”，避免提前泄露测试集结论。先核对现有章节、配置文件、训练与阈值选择代码，再补齐可复现性、统计检验、效率测量和引用。

助手 2026-07-14 15:14:50

现有章节已经固定了主实验的关键协议：按时间顺序`70/10/20`划分、Stage 1的`L=128/head`、Stage 2的`source\_host/W=128`、两阶段均以验证集 PR-AUC 选模型。接下来我在核对代码实际输出的 metrics、推理计时口径和显著性检验，确保论文中的“如何测量”与实现完全一致。

助手 2026-07-14 15:18:21

核对后有两个需要在实验章里明确写出的关键点。第一，代码中的`pr\_auc`实际由`average\_precision\_score`计算，因此论文应准确称为“PR-AUC (average precision, AP)”；第二，主数据划分的测试集是`84,380` flows，而早期部分 Stage 1 结果使用`35,410` flows，这两组结果不能放进同一张排名表，必须重跑到同一 split manifest 或标成探索性结果。

助手 2026-07-14 15:19:07

我现在开始写入两个新文件：实验方法章节和专用 BibTeX 补充库。章节会把现有实验分成 Stage 1 编码/融合/序列敏感性、Stage 2 上下文/窗口/模型对比、统计可靠性与效率四个层次，并保留少量必须由真实运行环境补齐的硬件版本宏，绝不虚构 GPU 或软件版本。

助手 2026-07-14 15:22:50

初稿已经写入，内容没有混入测试结果。现在我在做交叉检查，重点包括：所有`cite`是否能在现有及新增`.bib`中解析、所有`ref`是否有对应标签、表格/公式环境是否成对，以及章节中是否出现与代码不一致的描述。

助手 2026-07-14 15:24:36

代码复核又发现一个会影响论文可信度的细节：独立的 Stage 1 主训练器按验证集 PR-AUC 保存 checkpoint，但`comparison.py`中模型家族对比统一按验证集 class-1 F1 保存 checkpoint；二者阈值搜索实现也不同。它们各自在内部仍是公平对比，但不能假装是完全相同的实验协议。我正在把这一区别明确写入章节，并会在交付说明中指出最稳妥的重跑方案。

助手 2026-07-14 15:25:45

检查完成：新增章节共有 43 个标签、17 个唯一引用键，引用均能在现有或新增 BibTeX 中解析；公式、表格、列表和花括号全部成对。当前机器没有 LaTeX 编译器，所以我无法做 PDF 级排版检查，但已经完成静态语法与交叉引用检查。

助手 2026-07-14 15:26:12

已完成完整的**Experimental Methodology**章节及参考文献：

- [experimental_methodology.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/experimental_methodology.tex)
- [experimental_methodology_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/experimental_methodology_additions.bib)

章节涵盖：

- 实验问题与研究问题的对应关系
- 时间顺序 `70/10/20` 数据划分与防泄漏协议
- Stage 1 模型对比、编码消融、A/B/C 融合、序列长度和截断策略
- Stage 2 no-context、Transformer、LSTM、GRU、CNN+LSTM 对比
- context method 与 window size 消融
- 训练、checkpoint 选择和验证集阈值选择
- Macro-F1、Macro-Precision、Macro-Recall、ROC-AUC、PR-AUC/AP、FPR及混淆矩阵的完整公式
- 参数量、训练时间、推理时间和吞吐量测量口径
- paired bootstrap、McNemar 检验、Holm 多重比较校正
- 可复现性、结果报告规范与有效性边界

新增的8篇方法学文献包括机器学习可复现性、随机种子方差、ROC、bootstrap和配对检验，信息已通过 [JMLR reproducibility report](https://www.jmlr.org/papers/v22/20-303.html)、[ACL score-distribution study](https://aclanthology.org/D17-1035/) 和 [MLSys benchmark variance study](https://bouthilx.github.io/publication/2021-04-07-accounting-for-variance) 等来源核验。

在主文件中加入：

```
\usepackage{amssymb}

\input{doc/experimental_methodology}

\bibliography{
  references,
  doc/related_work_additions,
  doc/dataset_preprocessing_additions,
  doc/proposed_methodology_additions,
  doc/experimental_methodology_additions
}
```

提交前需要处理四点：

1. 替换章节开头的 `\ExpGPU`、`\ExpCPU`、`\ExpOS`、`\ExpSoftware` 和 `\ExpSuricata`，填写实际运行环境。
2. 章节定义了主模型使用5个种子 `130--134` 的确认性实验。未完成这些运行前，不能报告 mean±SD 或声称训练稳定性。
3. `35,410` test flows 的早期 Stage 1 结果不能与 `84,380` test flows 的完整数据结果放在同一排名表。
4. `comparison.py` 按 validation class-1 F1 选择 checkpoint，而独立 Stage 1 主训练器按 validation PR-AUC 选择。章节已如实区分；若要合并成一张主表，应统一协议后重跑。

另外，代码中的 `pr\_auc` 实际是 `average\_precision\_score`，章节已准确写成 **PR-AUC (AP)**。静态检查已通过；当前环境没有 LaTeX 编译器，因此尚未执行 PDF 排版编译。

用户 2026-07-14 16:10:33

结合我上面已有的模块，我不希望总是重复和冗余的内容。请给我提供Experimental Methodology的完整latex，并引入必要的论文。

助手 2026-07-14 16:10:39

我会先对照你已经完成的前五章、实际配置文件和现有参考文献，划清本章只负责“实验如何被公平、可复现地执行”，避免再次讲模型结构或提前重复结果分析。随后我会生成可直接`\input`的完整 LaTeX，并单独补齐新增 BibTeX 条目。

用户 2026-07-14 16:10:46

<turn_aborted>

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

</turn_aborted>

用户 2026-07-14 16:13:25

结合我上面已有的模块，我不希望总是重复和冗余的内容。

我使用的是 Colab 跑的我的代码，公司给我购买了 Colab Pro+ (600 个计算单元)，我会使用 T4 GPU 或者 L4 GPU。请给我提供 Experimental Methodology 的完整 latex，并引入必要的论文。

助手 2026-07-14 16:13:31

我会先对照你现有的四个章节、配置文件和 BibTeX，划清这一章只负责“如何实验、如何保证公平与可复现”，避免再次讲数据构建、模型结构或提前解释结果。然后我会把 Colab

Pro+、T4/L4、阈值选择、时间划分、消融与统计报告规范都落实成可直接纳入论文的 LaTeX。

助手 2026-07-14 16:13:47

工作区里已经出现了`experimental\_methodology.tex`和对应的 BibTeX，时间晚于上一章，可能是你刚添加的草稿或一次未完成生成。我会先完整审阅它并保留其中有效内容，再根据真实代码配置修正；不会覆盖你已有的章节安排。

助手 2026-07-14 16:14:27

这份草稿的骨架是对的，但有两处需要收紧：它仍重复了数据规模和模型内部结构，而且把 T4/L4 混在同一运行环境会让时间对比失去公平性。我会把模型细节改为章节交叉引用，把本章重点压到实验矩阵、控制变量、阈值、统计可靠性和资源测量，并明确“预测指标可跨 GPU 比较，运行时间只能在同型号 GPU 内比较”。

助手 2026-07-14 16:16:56

我现在会直接修订这两个文件：正文中删掉与 Dataset Construction、Problem Formulation 和 Proposed Methodology 重复的定义，仅保留实验所需的控制变量；参考文献中补充 PyTorch、Google Colab 以及 T4/L4 的一手资料。统计部分会把你现有的`seed=130`

明确为探索性结果，并把最终主对比设为多随机种子确认实验，避免论文把单次最优结果写成稳定结论。

助手 2026-07-14 16:19:12

正文已经收紧：研究问题不再复制第二张映射表，数据规模和特征定义改为指向已有章节，模型超参数表也只保留训练与选择设置。Colab 部分现在把 600 计算单元定位为“资源额度”，并明确 T4 16 GB 与 L4 24 GB 必须逐次记录，尤其不能把两种 GPU 的训练或推理时间放在同一排名中。接下来我正在做交叉引用和 BibTeX 完整性检查。

助手 2026-07-14 16:22:03

已完成并整理为可直接纳入论文的 LaTeX:

- [experimental_methodology.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/experimental_methodology.tex)
- [experimental_methodology_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/experimental_methodology_additions.bib)

章节包含:

- 实验逻辑与完整实验矩阵
- 时间顺序数据划分和数据泄漏控制
- Stage 1/Stage 2 基线、消融和敏感性实验
- 训练、checkpoint 与验证集阈值选择
- Macro-F1、Macro-Precision、Macro-Recall、PR-AUC、ROC-AUC、FPR 和混淆矩阵定义
- 五随机种子确认实验及 paired bootstrap、McNemar 检验
- Params、训练时间、推理时间和吞吐量测量
- Colab Pro+、600 compute units、T4/L4 环境规范
- 可复现性、结果报告规则和有效性边界

为减少冗余, 数据规模、特征和模型内部结构均改为引用前面章节, 不再重复推导。

特别明确了: 600 compute units 是实验资源额度, 不是计算效率指标; Colab 的 GPU 分配会动态变化, 因此 T4 和 L4 的运行时间不能直接排名。[Google Colab FAQ](https://research.google.com/colaboratory/faq.html) 也说明硬件可用性并非固定。T4 为 16 GB, L4 为 24 GB, 分别依据 [NVIDIA T4 datasheet](https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet.pdf) 和 [NVIDIA L4 product brief](https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/l4/PB-11316-001_v01.pdf)。

最终主文件中加入:

```
\input{doc/experimental_methodology}

\bibliography{
  references,
  doc/related_work_additions,
  doc/dataset_preprocessing_additions,
  doc/proposed_methodology_additions,
  doc/experimental_methodology_additions
}
```

需要注意:

1. 当前已有敏感性实验是 `seed=130`, 章节将其如实定义为 single-seed exploratory analysis; 最终核心模型对比建议补跑 `{130,131,132,133,134}`。
2. `35,410` 和 `84,380` 测试样本的结果不能放在同一个排名表。
3. CPU、RAM、Python、PyTorch、CUDA 和 Suricata 版本必须从最终 Colab 运行环境填写, 文件顶部已预留宏, 未虚构这些信息。
4. `references.bib` 中仍有重复的 `Hozouri2025Survey`; 两个 Problem Formulation 文件也有重复 label, 最终只能引入其中一个。
5. 当前环境没有 LaTeX 编译器, 因此完成了引用、括号和环境检查, 但未生成 PDF。你目前的 proposal 使用 `article` 文档类, 而本章使用 `chapter`; 正式论文模板应使用支持章节的 thesis、`report` 或 `book` 文档类。

用户 2026-07-14 16:59:08

结合我上面已有的模块，我不希望总是重复和冗余的内容。
请给我提供Results的完整latex，并引入必要的论文。

助手 2026-07-14 16:59:18

我会把 Results 严格限定为“报告观察到什么、差异有多大、这些差异回答哪一个 RQ”，不再重复模型公式、数据清洗步骤或指标定义。先核对工作区中的 Stage 1/Stage 2 结果文件，尤其确认 no-context、LSTM、GRU、CNN+LSTM 是否已经有可用输出，并按 `35,410` 与 `84,380` 两个测试清单分开组织。

助手 2026-07-14 17:01:02

核对结果后有两个关键边界：Stage 2 的 context method 和 window-size 结果完整，但 `no-context/LSTM/GRU/CNN+LSTM` 目前只有配置文件，没有保存的测试指标；Stage 1 的 MLP 虽然也是 35,410 个样本，类别计数却与序列模型不同，因此也不能当成完全相同测试清单的直接基线。我会在正文中保留这些结果，但明确区分“可直接比较”“仅作参考”“尚待补跑”，不制造不存在的显著性或效率数字。

助手 2026-07-14 17:04:14

我现在开始写正文。表格会以同一测试清单为边界：模型性能与 confusion matrix/效率分开，避免一张表塞满重复列；Stage 1 的 encoding、fusion、sequence length、selection strategy 各回答一个独立问题，Stage 2 则先给出最关键的 Stage1-to-Stage2 直接增益，再分析 context relation 和 window size。缺失的架构对比和统计显著性会作为“证据边界”写清楚，而不是用占位数字填充。

助手 2026-07-14 17:09:24

Results 正文已经完成，数字检查也发现并处理了最容易误写的地方：例如 Stage 2 相对 Stage 1 的 FPR 是从 `0.8249%` 降至 `0.5429%`，相对下降约 `34.2%`，同时 FP 减少 228、FN 减少 205；这说明增益不是简单移动阈值造成的 precision-recall 交换。现在只剩最终的 LaTeX 一致性检查和接入说明。

助手 2026-07-14 17:09:44

已完成完整 Results 章节：

[results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex)

章节已避免重复前面的模型公式、预处理和指标定义，主要包含：

- 不同测试 manifest 的可比性边界
- Stage 1 模型与编码消融结果
- Schemes A/B/C 融合结果
- `max\_seq\_len` 敏感性结果
- packet selection strategy 结果
- Stage 1 到 Stage 2 的直接增益
- Stage 2 context method 对比
- Stage 2 window size 对比
- 参数量、训练时间和当前统计证据边界
- 对 RQ1、RQ2 的直接回答

核心结果已经准确写入：

- Stage 2 source-host 将 Macro-F1 从 `0.8954` 提高到 `0.9275`
- PR-AUC 从 `0.8763` 提高到 `0.9345`
- FP 从 667 降至 439
- FN 从 732 降至 527
- FPR 从 `0.8249%` 降至 `0.5429%`, 相对下降约 `34.2%`

章节也明确说明：

- MLP 与其他 35,410-flow 结果类别数量不同，不能直接计算提升幅度。
- Scheme C 与 Schemes A/B 使用不同测试 manifest，不能直接排名。
- Stage 2 的 no-context、LSTM、GRU、CNN+LSTM 和 vanilla Transformer 尚无完整测试指标，因此暂时不能声称 target-query encoder 优于所有架构。
- 当前结果主要是 `seed=130`，不能声称统计显著。
- Stage 2 阈值、参数量、训练时间及推理时间仍需从最终 `stage2\_metrics.json` 补入。

引用使用现有 BibTeX 中的：

```
SaitoRehmsmeier2015
SommerPaxson2010
Arp2022DosDonts
ReimersGurevych2017
Bouthillier2021Variance
```

因此无需新增 Results 专用 BibTeX 文件。主文件加入：

```
\input{doc/results}
```

已验证 10 张表、26 个 label、全部引用和交叉引用，未发现列数、括号或引用错误。当前环境没有 LaTeX 编译器，因此尚未进行最终 PDF 渲染。

用户 2026-07-15 08:59:19

S2 各种模型对比

non-context:

params_total: 514

test_seconds_per_sample: 0.0001811978512206705

test: loss=0.024987, acc=0.982543, macro_precision=0.881983, macro_recall=0.907744,
macro_f1=0.894396, weighted_precision=0.983195, weighted_recall=0.982543, weighted_f1=0.982828,
auc=0.9894142355333573, pr_auc=0.8786064196071548, precision_label1=0.7715574422923852,
recall_label1=0.8261363636363637, f1_label1=0.7979146659349705, cm=[[79999, 861], [612, 2908]]

LSTM:

params_total: 922626

test_seconds_per_sample: 0.0003463923777079853

test: loss=0.021092, acc=0.984878, macro_precision=0.894613, macro_recall=0.924315,
macro_f1=0.908866, weighted_precision=0.985532, weighted_recall=0.984878, weighted_f1=0.985152,
auc=0.9933858798456366, pr_auc=0.9132591902711918, precision_label1=0.7954186413902053,
recall_label1=0.8582386363636364, f1_label1=0.8256354195135283, cm=[[80083, 777], [499, 3021]]

GRU:

params_total: 692226

test_seconds_per_sample: 0.00034020485910168734

test: loss=0.020575, acc=0.985672, macro_precision=0.913832, macro_recall=0.905165,
macro_f1=0.909448, weighted_precision=0.985529, weighted_recall=0.985672, weighted_f1=0.985596,
auc=0.9933378134626627, pr_auc=0.9140912641219764, precision_label1=0.8356084809758931,

recall_label1=0.8173295454545455, f1_label1=0.8263679448513572, cm=[[80294, 566], [643, 2877]]

CNN-LSTM:

params_total: 1218306

test_seconds_per_sample: 0.0003584734180255668

test: loss=0.021654, acc=0.985612, macro_precision=0.919175, macro_recall=0.900603,
macro_f1=0.909657, weighted_precision=0.985328, weighted_recall=0.985612, weighted_f1=0.985449,
auc=0.990027257311198, pr_auc=0.9050362596932977, precision_label1=0.8468731735827002,
recall_label1=0.8076923076923077, f1_label1=0.8268188302425107, cm=[[80346, 514], [677, 2843]]

vanilla Transformer:

params_total: 132994

test_seconds_per_sample: 0.00019674967046693202

test: loss=0.020623, acc=0.985068, macro_precision=0.907818, macro_recall=0.904714,
macro_f1=0.906259, weighted_precision=0.985012, weighted_recall=0.985068, weighted_f1=0.985039,
auc=0.9931356173970725, pr_auc=0.9072855920250074, precision_label1=0.8235967926689576,
recall_label1=0.8170454545454545, f1_label1=0.8203080433542499, cm=[[80244, 616], [644, 2876]]

我的模型:

params_total: 283522

test_seconds_per_sample: 0.00019650210508413672

test: loss=0.019559, acc=0.988552, macro_precision=0.932788, macro_recall=0.922427,
macro_f1=0.927539, weighted_precision=0.988425, weighted_recall=0.988552, weighted_f1=0.988482,
auc=0.993827886090999, pr_auc=0.934499456433569, precision_label1=0.872086247086247,
recall_label1=0.850284090909091, f1_label1=0.8610471806674338, cm=[[80421, 439], [527, 2993]]

结果补充，上面是stage2中模型的对比我自己的模型和non-context、vanilla

transformer、LSTM、CNN+LSTM、GRU。

结合代码，详细解析结果

助手 2026-07-15 08:59:30

这组结果正好补上了 Results 里最大的证据缺口。我会先核对 `s2/0712`

六份配置和各模型实现，确认它们是否共享相同的 Stage 1

embeddings、`source\_host`、`W=128`、训练策略与阈值规则；随后计算每个模型的

FPR、FNR、吞吐量及相对增益，并把 Results 章节中的“尚待补跑”替换为正式对比表和结论。

助手 2026-07-15 09:02:47

代码核对显示这是一组基本公平的 architecture ablation: `non\_context`、LSTM、GRU、CNN+LSTM

和你的模型都读取同一 Stage 1 embedding 目录，使用 `source\_host`、`W=128`、online causal context、相同 batch/loss/sampler/checkpoint/threshold 搜索；区别集中在 encoder。代码中 no-context 只取当前 flow，LSTM/GRU 使用 packed valid sequence，CNN+LSTM 先做 3/5/7

多尺度卷积，而你的模型用当前 flow 作为单一 query，对历史做 gated cross-attention。唯一需要补档的是 vanilla Transformer 的 resolved YAML，目前 `0712` 目录没有它。

助手 2026-07-15 09:03:18

我会把新结果写成两张互补表：一张只放检测质量与完整 confusion

matrix，另一张只放参数数量和推理效率，避免重复。正文会突出三个层次：context 相对 no-context

的净贡献、你的 gated target-query 相对 vanilla self-attention 的架构贡献，以及相对 LSTM/GRU/CNN+LSTM

的性能与计算折中。

助手 2026-07-15 09:06:05

我已经把这组 Stage 2 架构对比整理进 Results 章节，并按代码中的真实结构核对了各模型的差异。现在做最后一轮交叉检查，重点确认表格、引用标签和“同条件对比”的论述边界。

助手 2026-07-15 09:06:25

核心结论已经稳定：所提模型在 Macro-F1、PR-AUC、少数类 F1 和 FPR 上均为最佳；LSTM 只在召回率上略高。我要特别把这个权衡写清楚，避免把“总体最佳”误写成“每一个指标都最佳”。

助手 2026-07-15 09:07:22

最后检查发现一个论文里必须主动交代的限制：`non-context` 只有 514 个参数，而所提模型有 283,522 个，因此它是很好的“当前流下界”，但不是严格的等容量 context 消融。我要把这一点补进 Results，防止把全部增益都因果归因于历史上下文；最强的补充实验应是同一 target-query 模型将历史 mask 掉。

助手 2026-07-15 09:09:35

已结合 Stage 2 代码完成分析，并把完整表格、效率比较和论文式讨论写入 [results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:487)。

****总体结果****

测试集包含 84,380 个 flow，其中正常类 80,860，攻击类 3,520，正类比例仅约 4.17%。因此，Macro-F1、少数类 F1 和 PR-AUC 比 Accuracy 更具有解释力。

Model	Macro-P	Macro-R	Macro-F1	Class-1 F1	ROC-AUC	PR-AUC	FPR
--- ---	--- ---	--- ---	--- ---	--- ---	--- ---	--- ---	--- ---
No-context	0.8820	0.9077	0.8944	0.7979	0.9894	0.8786	1.0648%
Vanilla Transformer	0.9078	0.9047	0.9063	0.8203	0.9931	0.9073	0.7618%
LSTM	0.8946	**0.9243**	0.9089	0.8256	0.9934	0.9133	0.9609%
GRU	0.9138	0.9052	0.9094	0.8264	0.9933	0.9141	0.7000%
CNN+LSTM	0.9192	0.9006	0.9097	0.8268	0.9900	0.9050	0.6357%
Proposed	**0.9328**	0.9224	**0.9275**	**0.8610**	**0.9938**	**0.9345**	**0.5429%**

其中：

$$\mathrm{FPR} = \frac{FP}{TN + FP}.$$

****1. Macro-F1****

所提模型的 Macro-F1 为 ****0.9275****，比最强基线 CNN+LSTM 的 0.9097 高 ****0.0179****，比 Vanilla Transformer 高 ****0.0213****，比 no-context 高 ****0.0331****。

这说明提升并不局限于占绝大多数的正常类，而是在两个类别之间取得了更好的平衡。

****2. Precision 与 Recall 的权衡****

所提模型取得最高 Macro-Precision 0.9328，说明它对两个类别的预测具有最高的平均可靠性。

LSTM 的 Macro-Recall 最高，为 0.9243；所提模型为 0.9224，仅低 0.0019。少数类召回率方面，LSTM 为 0.8582，所提模型为 0.8503，相差 28 个 TP：

- LSTM: 499 FN、777 FP
- Proposed: 527 FN、439 FP

LSTM 多识别出 28 个攻击 flow，但产生了额外 338 个误报。所提模型因此在实际告警负担与攻击覆盖率之间更均衡。

****3. PR-AUC 与 ROC-AUC****

所有上下文模型的 ROC-AUC 都已经接近饱和，约为 0.99。因此，ROC-AUC 的差异较小。所提模型仍然最高，为 0.9938，但只比 LSTM 高约 0.0004。

PR-AUC 的区分更加明显：

- Proposed: 0.9345
- 最强基线 GRU: 0.9141
- Vanilla Transformer: 0.9073
- No-context: 0.8786

所提模型比最强基线提高 ****0.0204****，比 no-context 提高 ****0.0559****。在正类只有 4.17% 的不平衡数据上，这是比 ROC-AUC 更重要的结果。

****4. Confusion Matrix 与 FPR****

所提模型的混淆矩阵为：

```
\[
\begin{bmatrix}
80421 & 439 \\
527 & 2993
\end{bmatrix}.
\]
```

与其他模型比较：

- 相比 no-context: 减少 422 FP 和 85 FN。
- 相比 Vanilla Transformer: 减少 177 FP 和 117 FN。
- 相比 GRU: 减少 127 FP 和 116 FN。
- 相比 CNN+LSTM: 减少 75 FP 和 150 FN。
- 相比 LSTM: 减少 338 FP，但增加 28 FN。

FPR 从 no-context 的 1.0648% 降至 0.5429%，相对降低约 ****49.0%****。在 80,860 个正常样本上，即使只有约 0.5 个百分点的下降，也对应数百个误报，因此具有直接的实际意义。

****5. 代码层面的模型差异****

模型实现集中在 [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:211):

- No-context 只分类当前 flow embedding，不使用历史 flow。
- LSTM 和 GRU 对压缩后的有效上下文序列进行单向递归编码。
- CNN+LSTM 先用多个一维卷积核提取局部模式，再通过 LSTM 聚合。
- Vanilla Transformer 对整个上下文窗口执行 self-attention，最后分类当前 token。
- 所提模型在

[target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:29) 中仅让当前 flow 作为 query，对历史 flow 执行 attention，再通过特征级门控融合历史摘要与当前表示。

因此，Vanilla Transformer 与所提模型几乎相同的推理时间是合理的：所提模型虽然参数更多，但只计算当前 flow 到历史上下文的目标查询，而不是更新整个上下文序列。

****6. 参数量与效率****

Model	Params	ms/flow	flows/s
--- ---:	---:	---:	---:
No-context	514	0.1812	5,518.8
Vanilla Transformer	132,994	0.1967	5,082.6
LSTM	922,626	0.3464	2,886.9
GRU	692,226	0.3402	2,939.4
CNN+LSTM	1,218,306	0.3585	2,789.6
Proposed	283,522	0.1965	5,089.0

所提模型比 GRU、LSTM 和 CNN+LSTM 分别减少约 59.0%、69.3% 和 76.7% 的参数，推理延迟分别降低约 42.2%、43.3% 和 45.2%。

它的延迟与 Vanilla Transformer 基本相同，0.0002 ms 的差异不足以形成有意义的速度结论。计时代码位于 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:727)。

****需要谨慎表述****

No-context 只有 514 个参数，而所提模型有 283,522 个参数，因此该结果可以证明“上下文模型显著优于简单当前流分类器”，但不能把全部增益严格归因于历史上下文。

论文最好再加入一个严格消融：

> 保留相同的 target-query gated 架构和参数量，但 mask 掉所有历史 token，只保留当前 flow。

此外，最终论文还应补充：

- 3至5个随机种子的均值和标准差；
- 每个模型的最终验证集选择阈值；
- 相同 T4 或 L4 GPU 上的训练时间；
- paired bootstrap 或 McNemar 显著性检验；
- Vanilla Transformer 的完整 YAML 配置存档。

目前这组结果已经足以构成完整的 Stage 2 主模型对比表，但在加入等容量 no-context 消融和多随机种子结果后，关于“上下文和目标查询机制有效”的结论会更扎实。

用户 2026-07-15 10:06:49

结合我上面已有的模块，我不希望总是重复和冗余的内容。

请给我提供Results and discussion(将result和discussion合并到一起)的完整latex

助手 2026-07-15 10:07:01

我会先核对现有 Results、Experimental Methodology 和前文标签，只保留本章真正需要的结果解释，把方法定义、数据处理和指标公式留在前章引用，避免再讲一遍。随后我会将本章整理为可直接插入论文的合并版 `Results and Discussion` LaTeX。

助手 2026-07-15 10:07:31

现有 `results.tex` 已经接近合并章的形态：每张结果表后面紧跟解释，而不是另设一章重复复述。接下来我会把章名和研究问题结论改成严格的 `Results and Discussion` 表述，并修正 no-context 不是等容量控制这一处潜在过度结论。

助手 2026-07-15 10:09:52

我现在会做四处很小但关键的编辑：改为合并章标题；在 Stage 1 和 Stage 2 各加入一句与已有文献的对照；修正 RQ2 对 no-context 的表述；保留统计局限与效率边界。这样讨论与数据放在一起，同时不复述方法公式、预处理过程或评价指标定义。

助手 2026-07-15 10:10:44

合并章已经完成，结构检查也通过：12 个表格环境完整、括号和标签配对正常、9 个引用键均能在现有 bibliography 文件中找到。现在只剩下确认最终模板层级，因为你提供的 proposal `main.tex` 是 `article`，而硕士论文各章目前按 `chapter` 组织。

助手 2026-07-15 10:11:17

完整的合并版已经写入：

[results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:1)

章标题已修改为：

```
\chapter{Results and Discussion}
\label{sec:results}
```

本章采用“结果表格后立即讨论”的结构，没有单独设置重复性的 Discussion 章节，完整覆盖：

- 实验可比性与测试集边界
- Stage 1 模型与时间编码消融
- packet 与 flow statistical feature 融合
- `max\_seq\_len` 敏感性
- packet selection strategy
- Stage 1 到 Stage 2 的层次化增益
- Stage 2 context method 对比
- Stage 2 window size 敏感性
- no-context、Transformer、LSTM、GRU、CNN+LSTM 与所提模型对比
- 参数量、推理延迟与吞吐量
- 统计可靠性和当前实验局限
- RQ1、RQ2 的证据化回答
- 本章总结

讨论部分已经特别避免过度结论：no-context 仅有 514 个参数，因此被定义为“current-flow-only lower bound”，而不是严格的等容量消融；论文中也明确提出需要补充 history-masked、capacity-matched 实验。

本章使用9个已有引用键，均可在原始 `references.bib`、[related_work_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/related_work_additions.bib) 和 [experimental_methodology_additions.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/experimental_methodology_additions.bib) 中找到。最终编译前需要将两个 additions 文件中的条目合并到总 `references.bib`。

LaTeX 静态检查已经通过：12组表格环境、325组花括号、28个标签均正确，没有重复标签或缺失引用。当前机器没有安装 LaTeX 编译器，因此尚未执行完整 PDF 编译。

注意：你提供的 proposal `main.tex` 使用 `article`，不支持 `chapter`。当前版本按照最终硕士论文的 `report`/`book` 章节结构编写；如果最终模板仍使用 `article`，需要将本章的 `chapter`、`section`、`subsection` 整体分别下调一级。

用户 2026-07-15 10:20:46

结合我上面已经有的模块，我不希望总是重复和冗余的内容。
请给我提供 Results and discussion(将 result 和 discussion 合并到一起)的完整 latex。
我不想所有的对比结果都是用 table 展示，我想要一些 bar chart，PR_AUC 能不能是曲线展示？想要结果展示多样化一些。使论文更加好

助手 2026-07-15 10:21:02

我会先核对现有章节、标签和已经使用过的表格，随后把 Results 与 Discussion 重构成一个连续章节。图形上会保留少量适合查精确数值的表格，增加柱状图、误报/漏报图和性能效率图；真正的 PR 曲线只在有逐样本预测分数时绘制，避免用单个 PR-AUC 数值伪造曲线。

助手 2026-07-15 10:23:49

重构方案已经明确：保留 3 个承担“精确数值/比较边界”职责的表格，其余消融与敏感性结果改成 6 幅图。Stage 1 用模型柱状图与长度/采样敏感性图；Stage 2 用上下文关系图、窗口曲线、架构性能图和参数量-延迟散点图，同时把每幅图后的文字直接写成讨论，而不是在章节末再次复述。

助手 2026-07-15 10:34:03

图形已经生成并逐张检查过，接下来我会替换当前表格密集的章节正文。新的版本会保留所有关键数值和比较限制，但把超参数扫描、上下文消融和架构对比交给图形表达；表格只保留测试集边界、Stage 1 融合结果、主要层级增益和训练成本这几类必须精确查阅的信息。

助手 2026-07-15 10:37:11

章节正文已经完成重构，当前版本是 6 幅图配 4 张必要表格，讨论紧跟对应实验。现在进行编译前检查，并确认所有图都能被 LaTeX 找到；随后我会再核对 PR 曲线工具是否与代码输出的 `flow\_id,label,prob\_label\_1,pred` 格式一致。

助手 2026-07-15 10:40:13

已完成合并后的完整 **Results and Discussion** 章节：

[results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:11)

新版从原来的 12 张表格调整为 **4 张必要表格 + 6 幅图**，并把 discussion 直接放在每组结果后面，避免结果与讨论重复。

图形设计

- Stage 1 模型与编码方式：横向分组柱状图
- 序列长度与 packet-selection：折线图 + 柱状图
- Stage 2 context method：性能柱状图 + Recall/FPR 散点图
- Stage 2 window size：指标曲线 + FP/FN 曲线
- Stage 2 模型比较：性能柱状图 + FP/FN 横向柱状图

- 参数量与推理时间：对数坐标效率散点图

![Stage 2 architecture comparison](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/stage2_architecture_comparison.png)

所有 PDF 矢量图和 PNG 预览都位于：

[doc/figures](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures)

图形可通过 [generate_results_figures.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/generate_results_figures.py:1) 重新生成。

****关于 PR-AUC 曲线****

PR-AUC 是一个标量面积，不能仅根据 `PR-AUC=0.9345` 重建曲线。真正可以展示的是 ****Precision-Recall curve****，它需要每个测试样本的：

```
flow_id,label,prob_label_1
```

你的 Stage 2 代码已经在

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:747) 输出 `stage2\_predictions\_test.csv`。我添加了 [plot_pr_curves.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/plot_pr_curves.py:1)，等六个模型的预测 CSV 从 Colab 保存回来后即可生成真实 PR 曲线。LaTeX 中已加入条件代码：只要生成 `doc/figures/stage2\_pr\_curves.pdf`，曲线就会自动出现在章节中。

****验证情况****

- 4 个 table、7 个 figure 环境全部配对
- 花括号数量匹配
- 没有重复标签
- 9 个引用键均存在
- 6 个 PDF 图像均有效
- PR 曲线脚本已用现有预测 CSV 测试通过

当前机器没有 LaTeX 编译器，因此没有完成整篇 PDF 编译。另需注意，你现有 `main.tex` 使用 `article`，不支持 `chapter`；最终论文应切换到 `report`、`book` 或学校 thesis class，或者将章节首行改成 `section{Results and Discussion}`。

用户 2026-07-15 11:12:58

S1 ABC三种方案

方案A运行结果: "test_metrics": {
"accuracy": 0.9817283253318272,
"macro_precision": 0.891903312957051,
"macro_recall": 0.8600128012451596,
"macro_f1": 0.8752142703953976,
"weighted_precision": 0.9810772212036308,
"weighted_recall": 0.9817283253318272,
"weighted_f1": 0.9813408831023472,
"precision_label1": 0.7950310559006211,
"recall_label1": 0.7277896233120114,
"f1_label1": 0.759925788497217,
"auc": 0.9800628507610464,


```

"pr_auc": 0.8326108220868012,
"per_class_f1": {
"0": 0.990502752293578,
"1": 0.759925788497217
},
"confusion_matrix": [
[
33739,
264
],
[
383,
1024
]
],
"num_classes": 2,
"num_samples": 35410,
"loss": 0.11201799211734469,
"threshold": 0.7150000000000005
}

```

方案B运行结果: Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.4550000000000003}

```

方案C运行结果: "test_metrics": {
"accuracy": 0.9834202417634511,
"macro_precision": 0.8989505209974933,
"macro_recall": 0.8918983147078002,
"macro_f1": 0.8953896222507791,
"weighted_precision": 0.9832784957495156,
"weighted_recall": 0.9834202417634511,
"weighted_f1": 0.9833463190159758,
"precision_label1": 0.8069464544138929,
"recall_label1": 0.7920454545454545,
"f1_label1": 0.7994265232974911,
"auc": 0.9890371580790593,
"pr_auc": 0.8762558340285889,
"per_class_f1": {
"0": 0.9913527212040671,
"1": 0.7994265232974911
},
"confusion_matrix": [
[33736, 267],
[ 293, 1114]
]
}

```

```

],
"num_classes": 2,
"num_samples": 35410,
"loss": 0.06821018314045486,
"threshold": 0.6840000000000005
}

```

不同配置下的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-15 11:13:40

我先做一致性校验，因为方案 C 的这组数据里出现了一个会直接影响论文结论的问题：`num\_samples=35,410` 和新混淆矩阵一致，但 accuracy、P1、R1、F1、AUC/PR-AUC 等数值仍精确对应之前那次 84,380 样本运行。需要先区分“真正的同测试集 C 结果”和把旧指标配上新矩阵的情况，否则 A/B/C 不能作为受控对比写进论文。

助手 2026-07-15 11:16:12

****先说关键问题****

方案 A、B 的指标与混淆矩阵完全一致；方案 C 当前混合了两次不同测试运行的结果，暂时不能直接写成 A/B/C 的公平对比。

方案 C 的新混淆矩阵：

```

\[
\begin{bmatrix}
33736 & 267 \\
293 & 1114
\end{bmatrix}
\]

```

样本数确实为：

```

\[
33736+267+293+1114=35410.
\]

```

但由该矩阵计算的 Accuracy 是：

```

\[
\frac{33736+1114}{35410}=0.984185,
\]

```

并不是提供的 (0.983420) 。

更关键的是，你提供的方案 C Accuracy、Macro-Precision、Macro-Recall、Macro-F1、Class-1 Precision/Recall/F1，精确对应之前的 84,380 样本混淆矩阵：

```

\[
\begin{bmatrix}
80193 & 667 \\
732 & 2788
\end{bmatrix}

```


$\end{bmatrix}$.

$\backslash$

例如：

$\backslash$

$\frac{80193+2788}{84380}=0.983420,$

$\backslash$

$\backslash$

$\frac{2788}{2788+667}=0.806946,$

$\quad\quad\quad$

$\frac{2788}{2788+732}=0.792045.$

$\backslash$

所以目前方案 C 是：

- `num\_samples` 和新 confusion matrix：来自 35,410 样本运行；
- 其他 metrics、loss、threshold：来自之前 84,380 样本运行。

****一致性对照****

| Metric | A | B | C 提供值 | C 由新 CM 计算 |

|---|---|---|---|---|

| Accuracy | 0.981728 | 0.982999 | 0.983420 | **0.984185** |

| Macro-Precision | 0.891903 | 0.889560 | 0.898951 | **0.899026** |

| Macro-Recall | 0.860013 | 0.886905 | 0.891898 | **0.891952** |

| Macro-F1 | 0.875214 | 0.888228 | 0.895390 | **0.895454** |

| Class-1 Precision | 0.795031 | 0.788117 | 0.806946 | **0.806662** |

| Class-1 Recall | 0.727790 | 0.782516 | 0.792045 | **0.791756** |

| Class-1 F1 | 0.759926 | 0.785307 | 0.799427 | **0.799139** |

| FPR | 0.7764% | 0.8705% | 0.8249% (旧 CM) | **0.7852%** |

AUC 和 PR-AUC 无法从混淆矩阵重新计算，必须使用该次运行的逐样本概率。因此目前不能确认 C 的 `AUC=0.9890` 和 `PR-AUC=0.8763` 是否属于新的 35,410 样本运行。

A 与 B 的可靠对比

A、B 使用相同测试规模、类别分布和测试样本，因此可以直接比较。

Macro-Precision

方案 A：

$\backslash$

$P_{\mathrm{macro}}=0.8919$

$\backslash$

方案 B：

$\backslash$

$P_{\mathrm{macro}}=0.8896$

$\backslash$

A 高出 0.0023, 即约 **0.23 个百分点**。这说明方案 A 的预测略微保守, 预测为某一类别时平均更准确。但这个精度优势很小, 需要结合 Recall 和混淆矩阵理解。

Macro-Recall

方案 B 的 Macro-Recall 从 A 的 0.8600 提高到 0.8869:

$$\begin{aligned} & \Delta R_{\mathrm{macro}} = +0.0269. \end{aligned}$$

Class-1 Recall 从 0.7278 提高至 0.7825:

$$\Delta R_1 = +0.0547.$$

也就是说, B 将攻击检出率提高约 **5.47 个百分点**。

对应混淆矩阵:

- A: 383 FN
- B: 306 FN

B 少漏掉了 **77 个攻击 flow**。

Macro-F1

$$\begin{aligned} & 0.8882 - 0.8752 = 0.0130. \end{aligned}$$

方案 B 的 Macro-F1 提高 **1.30 个百分点**。Class-1 F1 同样从 0.7599 提高到 0.7853, 提升 **2.54 个百分点**。

这说明 B 虽然牺牲了少量 Precision, 但显著提高了攻击覆盖率, 最终获得更好的综合平衡。

ROC-AUC 和 PR-AUC

方案 B 相对 A:

$$\begin{aligned} & \Delta \mathrm{ROC\mbox{-}AUC} \\ & = 0.9860 - 0.9801 \\ & = 0.0059, \end{aligned}$$

$$\begin{aligned} & \Delta \mathrm{PR\mbox{-}AUC} \\ & = 0.8564 - 0.8326 \\ & = 0.0238. \end{aligned}$$

PR-AUC 提升约 **2.38 个百分点**, 明显大于 ROC-AUC 的变化。这说明 B 对少数类攻击样本的排序能力确实更强, 而不仅仅是在测试阈值上选择了更激进的分类点。

A 和 B 的阈值分别为 0.715 和 0.455。虽然 B 的较低阈值会提高 Recall, 但 AUC 和 PR-AUC 都是阈值无关指标, 因此 B 的优势不能完全归因于阈值差异。

FPR 与混淆矩阵

$$\backslash[\mathrm{FPR}_A=\frac{264}{33739+264}=0.7764\%,\backslash]$$

$$\backslash[\mathrm{FPR}_B=\frac{296}{33707+296}=0.8705\%.\backslash]$$

B 比 A 增加了 32 个 FP，FPR 上升约 0.0941 个百分点，但减少了 77 个 FN。

因此：

- A 更偏向减少误报；
- B 更偏向减少漏报；
- 按 Macro-F1、Class-1 F1 和 PR-AUC，B 是更合理的默认方案。

方案 C 的暂定分析

下面只能作为“新混淆矩阵为真实结果”的暂定分析，不能与提供的 AUC/PR-AUC 混在一起发表。

基于新矩阵，C 的 FPR 为：

$$\backslash[\mathrm{FPR}_C=\frac{267}{33736+267}=0.7852\%.\backslash]$$

FNR 为：

$$\backslash[\mathrm{FNR}_C=\frac{293}{293+1114}=20.8244\%.\backslash]$$

相比 B：

- FP：296 降至 267，减少 29；
- FN：306 降至 293，减少 13；
- FPR：0.8705% 降至 0.7852%；
- Macro-F1：由矩阵计算约从 0.8882 提高到 0.8955；
- Class-1 F1：约从 0.7853 提高到 0.7991。

如果方案 C 的真实 AUC 和 PR-AUC 也确实是 0.9890 和 0.8763，那么 C 将是三种方案中最均衡的：

- Macro-Precision 最高；
- Macro-Recall 最高；
- Macro-F1 最高；
- Class-1 F1 最高；
- AUC 和 PR-AUC 最高；
- 相比 B，同时减少 FP 和 FN；
- FPR 接近 A，但攻击检出率明显高于 A。

C 相比 B 的提供指标差异

假设这些指标最终确认来自同一个 35,410 测试集：

```
| Metric | C 相比 B |
|---|---:|
| Macro-Precision | +0.0094 |
| Macro-Recall | +0.0050 |
| Macro-F1 | +0.0072 |
| Class-1 Precision | +0.0188 |
| Class-1 Recall | +0.0095 |
| Class-1 F1 | +0.0141 |
| ROC-AUC | +0.0031 |
| PR-AUC | +0.0199 |
| FP | -29 |
| FN | -13 |
```

这会构成很有说服力的 late-fusion 改进证据，因为 C 不是单纯通过提高 Recall 来牺牲 Precision，而是同时改善两者。

论文应该怎样表述

在修正 C 之前，论文中只能可靠地写：

> Scheme B provides a better precision--recall balance than Scheme A. It improves macro-F1 by 1.30 percentage points and PR-AUC by 2.38 percentage points, while reducing false negatives by 77 at the cost of 32 additional false positives.

暂时不能正式写：

> Scheme C significantly outperforms Schemes A and B on the same test set.

方案 C 需要找回同一次运行的：

```
test_metrics.json
test_predictions.csv
test_split flow_id
config_used.yaml
selected threshold
```

并确认：

```
\[
TN+FP+FN+TP=N,
\]

\[
Accuracy=\frac{TN+TP}{N},
\]

\[
P_1=\frac{TP}{TP+FP},
\quad
R_1=\frac{TP}{TP+FN}.
\]
```

AUC 和 PR-AUC必须由同一个`test\_predictions.csv`中的`label`与预测概率重新计算。

当前 [results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:151) 仍保留 C 为 84,380 样本的版本, 因为该版本与提供的完整 metrics 内部一致。我没有用不一致的新矩阵覆盖它。等方案 C 的同一次运行文件确认后, 才能安全地把 A/B/C 更新为共同 35,410-flow 的完整受控比较。

用户 2026-07-15 11:16:29

S1 各种模型的对比:

Model	Params	Time (s)	Accuracy	F1	Macro F1	Class1 Precision	Class1 Recall	Class1 ROC-AUC	PR-AUC
threshold	precision_macro	recall_macro	precision_weighted	recall_weighted	f1_weighted				
confusion_matrix									
Ours_BothEncoding	344963	4337.6	0.9830	0.8882	0.7853	0.7881	0.7825	0.9860	0.8564
	0.4550	0.8896	0.9830	0.9830	0.9830	[[33707, 296], [306, 1101]]			
Ours_TimeOnly	344963	2250.5	0.9822	0.8818	0.7728	0.7818	0.764	0.9846	0.851
	0.4785	0.886	0.8776	0.982	0.9822	0.9821	[[33703, 300], [332, 1075]]		
Ours_PositionOnly	344963	1994.7	0.9812	0.8739	0.7575	0.7791	0.737	0.9846	0.8482
	0.4703	0.8841	0.8642	0.9808	0.9812	0.981	[[33709, 294], [370, 1037]]		
LSTM_with_time	217346	3556.3	0.9806	0.8677	0.7455	0.7786	0.715	0.9801	0.8077
	0.4638	0.8834	0.8533	0.9799	0.9806	0.9802	[[33717, 286], [401, 1006]]		
CNN1D_with_time	70274	1162.4	0.9803	0.867	0.7443	0.768	0.7221	0.9793	0.7821
	0.4822	0.8782	0.8565	0.9798	0.9803	0.98	[[33696, 307], [391, 1016]]		
GRU	162690	3684.4	0.9797	0.8595	0.7296	0.7748	0.6894	0.9799	0.8009
	0.4565	0.881	0.8406	0.9788	0.9797	0.9791	[[33721, 282], [437, 970]]		
LSTM	216834	3381.2	0.9792	0.8582	0.7273	0.7575	0.6994	0.9796	0.7976
	0.4272	0.8726	0.845	0.9785	0.9792	0.9788	[[33688, 315], [423, 984]]		
CNN1D	68354	1591	0.9779	0.8579	0.7274	0.712	0.7434	0.9809	0.7975
	0.4203	0.8507	0.8655	0.9783	0.9779	0.9781	[[33580, 423], [361, 1046]]		
BiLSTM	564738	2625.7	0.9778	0.8522	0.716	0.7273	0.705	0.9797	0.7903
	0.4408	0.8575	0.8471	0.9775	0.9778	0.9776	[[33631, 372], [415, 992]]		
Ours_NoEncoding	344963	2213.9	0.977	0.8507	0.7134	0.7069	0.72	0.9816	0.8003
	0.4335	0.8477	0.8538	0.9772	0.977	0.9771	[[33583, 420], [394, 1013]]		

Full metrics:

```
{'accuracy': 0.9626659135837334, 'macro_precision': 0.74145487832394, 'macro_recall': 0.809354661189905, 'macro_f1': 0.7706508462917445, 'weighted_precision': 0.9680282703050128, 'weighted_recall': 0.9626659135837334, 'weighted_f1': 0.9649653532784647, 'precision_label1': 0.4967628016480283, 'recall_label1': 0.6437833714721587, 'f1_label1': 0.5607973421926911, 'auc': 0.9552327656163553, 'pr_auc': 0.5604476128764463, 'per_class_f1': {'0': 0.9805043503907979, '1': 0.5607973421926911}, 'confusion_matrix': [[33244, 855], [467, 844]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.031577753180528016, 'threshold': 0.42999999999999977}
```

这是stage1中, MLP的运行结果 (flow-level aggregation statistical features)

方案B, max_seq_len=128 and strategy = head

不同模型的运行结果。详细解析结果 (主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics, 并从这些数据进行分析)

助手 2026-07-15 11:17:13

我会把这组结果分成两个证据层次: 10 个序列模型共享`34,003/1,407`的测试类别构成, 可以直接排名; MLP 虽然也是 35,410 个样本, 但类别构成为`34,099/1,311`, 只能作为 flow-statistics 参考, 不能用原始 FP/FN

与前者做严格受控差值。接下来重点核算 FPR、FNR、主要模型增益和性能成本权衡。

助手 2026-07-15 11:18:43

1. 可比性检查

除 MLP 外，10 个序列模型的混淆矩阵都满足：

```
\[
N_0=34{,}003,\qquad N_1=1{,}407,\qquad N=35{,}410.
\]
```

因此这些模型使用相同类别构成，可以直接比较。

MLP 的测试集则为：

```
\[
N_0=34{,}099,\qquad N_1=1{,}311.
\]
```

虽然总样本数同样是 35,410，但测试样本或切分版本不同。因此 MLP 只能作为补充性的 flow-statistics reference，不能直接计算严格的 FP/FN 改进幅度。

2. 主要结果概览

Model	Macro-P	Macro-R	Macro-F1	Class-1 F1	ROC-AUC	PR-AUC	FPR	FN
BothEncoding	0.8896	0.8869	0.8882	0.7853	0.9860	0.8564	0.8705%	306
TimeOnly	0.8860	0.8776	0.8818	0.7728	0.9846	0.8510	0.8823%	332
PositionOnly	0.8841	0.8642	0.8739	0.7575	0.9846	0.8482	0.8646%	370
LSTM+time	0.8834	0.8533	0.8677	0.7455	0.9801	0.8077	0.8411%	401
CNN1D+time	0.8782	0.8565	0.8670	0.7443	0.9793	0.7821	0.9029%	391
GRU	0.8810	0.8406	0.8595	0.7296	0.9799	0.8009	0.8293%	437
LSTM	0.8726	0.8450	0.8582	0.7273	0.9796	0.7976	0.9264%	423
CNN1D	0.8507	0.8655	0.8579	0.7274	0.9809	0.7975	1.2440%	361
BiLSTM	0.8575	0.8471	0.8522	0.7160	0.9797	0.7903	1.0940%	415
NoEncoding	0.8477	0.8538	0.8507	0.7134	0.9816	0.8003	1.2352%	394
MLP reference	0.7415	0.8094	0.7707	0.5608	0.9552	0.5604	2.5074%	467

3. Overall best model

`Ours\_BothEncoding` 在共同测试集上取得：

- 最高 Macro-Precision: 0.8896
- 最高 Macro-Recall: 0.8869
- 最高 Macro-F1: 0.8882
- 最高 Class-1 F1: 0.7853
- 最高 ROC-AUC: 0.9860
- 最高 PR-AUC: 0.8564
- 最少 FN: 306

其混淆矩阵为：

```
\[
\begin{bmatrix}
33707 & 296 \\
306 & 1101
\end{bmatrix}.
\]
```

对应：

```
\[
\mathrm{FPR}=\frac{296}{34003}=0.8705\%,
\]
```

```
\[
\mathrm{FNR}=\frac{306}{1407}=21.7484\%.
\]
```

它不是 FPR 最低的模型，但获得了最低 FN 和最强整体 F1。因此，它不是单纯依靠保守预测减少误报，而是在攻击覆盖率与误报之间取得了最好的综合平衡。

4. 时间与位置编码消融

BothEncoding vs TimeOnly

变化 BothEncoding 相对 TimeOnly
--- ---:
Macro-F1 +0.0064
Macro-Precision +0.0036
Macro-Recall +0.0093
Class-1 F1 +0.0125
PR-AUC +0.0054
ROC-AUC +0.0014
FP -4
FN -26

BothEncoding 同时减少 FP 和 FN，说明位置编码提供了时间编码之外的补充信息。

BothEncoding vs PositionOnly

变化 BothEncoding 相对 PositionOnly
--- ---:
Macro-F1 +0.0143
Macro-Precision +0.0055
Macro-Recall +0.0227
Class-1 F1 +0.0278
PR-AUC +0.0082
ROC-AUC +0.0014
FP +2
FN -64

加入真实时间信息后，仅增加2个 FP，却减少64个 FN。说明对当前网络流量而言，真实 elapsed time 比单纯 packet order 提供了更强的攻击识别信号。

BothEncoding vs NoEncoding

BothEncoding 相对 NoEncoding:

- Macro-F1: 提高 0.0375
- Macro-Precision: 提高 0.0419
- Macro-Recall: 提高 0.0331
- Class-1 F1: 提高 0.0719
- PR-AUC: 提高 0.0561
- FP: 减少124
- FN: 减少88
- FPR: 从1.2352%降至0.8705%

这是 Stage 1 最重要的消融证据。时间和位置编码不仅提高分数，而且同时降低两类错误。

因此论文可以得出:

> Packet order and elapsed time provide complementary information. Removing temporal encoding substantially degrades both minority-class ranking and threshold-dependent detection performance.

5. 与传统序列模型比较

最强非 Transformer 基线

以 Macro-F1 和 PR-AUC 衡量，最强非 Transformer 基线是 LSTM+time:

```
\[
F_{1,\mathrm{macro}}=0.8677,\quad
\mathrm{PR\mbox{-}AUC}=0.8077.
\]
```

BothEncoding 相对 LSTM+time:

- Macro-F1: +0.0205
- Macro-Recall: +0.0336
- Class-1 F1: +0.0398
- ROC-AUC: +0.0059
- PR-AUC: +0.0487
- FN: 减少95
- FP: 增加10

FPR只增加约0.0294个百分点，但攻击漏检率从28.50%降至21.75%。这个权衡明显有利于 BothEncoding。

GRU 的低 FPR 并不代表总体最好

GRU 的 FPR 最低:

```
\[
\mathrm{FPR}_{\mathrm{GRU}}=0.8293\%.
\]
```

但它有437个 FN，FNR 达31.06%，也是所有主要序列模型中漏检最严重的模型之一。

BothEncoding 相比 GRU:

- 多14个 FP;
- 少131个 FN;
- Macro-F1提高0.0287;
- PR-AUC提高0.0555。

所以 GRU 的低 FPR主要来自更保守的预测，而不是更强的综合检测能力。

CNN1D 的 Recall/FPR 权衡

CNN1D 的 Class-1 Recall 为0.7434，在基础模型中较高，但 Class-1 Precision只有0.7120，产生423个 FP，FPR达到1.2440%。

BothEncoding 相比 CNN1D同时减少：

- 127个 FP；
- 55个 FN。

这属于明显的双向改善。

6. 时间特征并非对所有架构同样有效

LSTM 加入时间后：

- Macro-F1: 0.8582 → 0.8677
- Class-1 F1: 0.7273 → 0.7455
- PR-AUC: 0.7976 → 0.8077
- FP: 315 → 286
- FN: 423 → 401

时间特征对 LSTM 有稳定帮助。

但 CNN1D 加入时间后：

- Macro-F1: 0.8579 → 0.8670
- FP: 423 → 307
- FN: 361 → 391
- PR-AUC: 0.7975 → 0.7821

CNN1D+time 在选定阈值上更精确、更保守，但其阈值无关排序能力反而下降。说明时间特征是否有效与编码和聚合方式有关，不能笼统写成“加入时间后所有模型都改善”。

真正稳定的结果是：Transformer 中 position 与 time 的联合编码在 Macro-F1、PR-AUC 和混淆矩阵上都得到改善。

7. MLP flow-statistics reference

MLP 的混淆矩阵为：

```
\[
\begin{bmatrix}
33244 & 855 \\
467 & 844
\end{bmatrix}.
\]
```

对应：

```
\[
\mathrm{FPR}_{\text{MLP}}
=\frac{855}{34099}
=2.5074\%,
\]
```

$$\frac{\mathrm{FNR}_{\mathrm{MLP}}}{\mathrm{FNR}_{\mathrm{BothEncoding}}} = \frac{467}{1311} = 35.6217\%$$

它的主要问题包括：

- Macro-Precision只有0.7415；
- Class-1 Precision只有0.4968；
- 每两个攻击告警中大约只有一个真正正类；
- PR-AUC只有0.5604；
- FPR约为BothEncoding的2.88倍；
- 攻击漏检率超过35%。

相对于BothEncoding，描述性差距为：

- Macro-F1：-0.1175
- Class-1 F1：-0.2245
- PR-AUC：-0.2960
- ROC-AUC：-0.0308

这些结果支持 packet-sequence representation 比单纯 flow aggregation 包含更多有效信息。但是由于MLP测试类别构成不同，论文中应写成 “supplementary reference”，不能写成严格受控的性能提升。最理想的做法是使用完全相同的 `test\_flow\_id` 重新评估MLP。

另外，MLP loss 为0.0316，虽然低于BothEncoding的0.0576，但不同模型可能使用不同损失缩放、类别权重或训练目标，因此不能根据loss跨模型判断优劣。

8. 参数量与训练时间

BothEncoding有344,963个参数，训练时间4,337.6秒，是本组中最慢的运行。

值得注意的是：

- BiLSTM参数最多：564,738，但性能较弱；
- CNN1D+time最快：1,162.4秒，但PR-AUC仅0.7821；
- BothEncoding比CNN1D+time慢约3.73倍，但PR-AUC高0.0743；
- BothEncoding、TimeOnly、PositionOnly参数完全相同，但训练时间差距很大。

最后一点不能简单归因于编码计算开销。还需要核对：

- 实际训练epoch数；
- early stopping停止位置；
- T4或L4 GPU类型；
- 数据加载时间；
- 是否包含embedding预计算；
- Colab运行负载。

因此训练时间目前适合描述，不适合做严格效率排序。Stage 1 还缺少相同硬件上的推理时间，暂时无法得出部署速度结论。

9. 论文可支持的核心结论

论文中可以写：

> The combined positional and cumulative-time encoding achieved the best overall Stage 1 performance, with a macro-F1 of 0.8882 and a PR-AUC of 0.8564. Compared with the strongest recurrent baseline, LSTM+time, it improved macro-F1 by 2.05 percentage points and PR-AUC by 4.87 percentage points, while reducing false negatives from 401 to 306. The encoding ablation further showed that packet order and elapsed time were complementary: removing both encodings reduced macro-F1 by 3.75 percentage points and PR-AUC by 5.61 percentage points.

不要使用 “statistically significant” 或 “显著优于” 这种统计意义表述，因为目前仍是单随机种子结果。

这些结果已经写入

[results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:74)，模型柱状图位于 [stage1_model_comparison.pdf](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/stage1_model_comparison.pdf)。当前把MLP单独讨论而不放进共同排名，是正确且更严谨的展示方式。

用户 2026-07-15 11:37:55

S1 ABC三种方案

```
方案A运行结果: "test_metrics": {
  "accuracy": 0.9817283253318272,
  "macro_precision": 0.891903312957051,
  "macro_recall": 0.8600128012451596,
  "macro_f1": 0.8752142703953976,
  "weighted_precision": 0.9810772212036308,
  "weighted_recall": 0.9817283253318272,
  "weighted_f1": 0.9813408831023472,
  "precision_label1": 0.7950310559006211,
  "recall_label1": 0.7277896233120114,
  "f1_label1": 0.759925788497217,
  "auc": 0.9800628507610464,
  "pr_auc": 0.8326108220868012,
  "per_class_f1": {
    "0": 0.990502752293578,
    "1": 0.759925788497217
  },
  "confusion_matrix": [
    [
      33739,
      264
    ],
    [
      383,
      1024
    ]
  ],
  "num_classes": 2,
  "num_samples": 35410,
  "loss": 0.11201799211734469,
  "threshold": 0.7150000000000005
}
```

方案B运行结果: Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.4550000000000003}

方案C运行结果: "test_metrics": {
"accuracy": 0.9841852417634511,
"macro_precision": 0.8990265209974933,
"macro_recall": 0.8919523147078002,
"macro_f1": 0.8954546222507791,
"weighted_precision": 0.9832784957495156,
"weighted_recall": 0.9834202417634511,
"weighted_f1": 0.9833463190159758,
"precision_label1": 0.8066624544138929,
"recall_label1": 0.7917564545454545,
"f1_label1": 0.7991395232974911,
"auc": 0.9890371580790593,
"pr_auc": 0.8762558340285889,
"per_class_f1": {
"0": 0.9913527212040671,
"1": 0.7994265232974911
},
"confusion_matrix": [
[33736, 267],
[293, 1114]
],
"num_classes": 2,
"num_samples": 35410,
"loss": 0.06821018314045486,
"threshold": 0.6840000000000005
}

不同配置下的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-15 11:38:14

这次方案 C 的主要阈值指标已经与 35,410 样本混淆矩阵基本一致，可以分析 A/B/C 的 Precision、Recall、F1 和 FPR。不过我还会把剩余的数据来源问题单独标出：C 的 weighted metrics、`per\_class\_f1` 以及完全未变化的 AUC/PR-AUC 仍像是上一轮 84,380 样本结果，论文中不能把它们当成已经全部验证完毕。

用户 2026-07-15 11:38:48

<turn_aborted>

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

</turn_aborted>

用户 2026-07-15 11:40:31

S1 ABC三种方案

方案A运行结果: "test_metrics": {

```
"accuracy": 0.9817283253318272,
"macro_precision": 0.891903312957051,
"macro_recall": 0.8600128012451596,
"macro_f1": 0.8752142703953976,
"weighted_precision": 0.9810772212036308,
"weighted_recall": 0.9817283253318272,
"weighted_f1": 0.9813408831023472,
"precision_label1": 0.7950310559006211,
"recall_label1": 0.7277896233120114,
"f1_label1": 0.759925788497217,
"auc": 0.9800628507610464,
"pr_auc": 0.8326108220868012,
"per_class_f1": {
"0": 0.990502752293578,
"1": 0.759925788497217
```

},

"confusion_matrix": [

[

33739,

264

],

[

383,

1024

]

],

"num_classes": 2,

"num_samples": 35410,

"loss": 0.11201799211734469,

"threshold": 0.7150000000000005

}

方案B运行结果: Full metrics: {'accuracy': 0.9829991527817001, 'macro_precision': 0.8895604171389182, 'macro_recall': 0.8869054386082953, 'macro_f1': 0.8882279230430299, 'weighted_precision': 0.9829418565248673, 'weighted_recall': 0.9829991527817001, 'weighted_f1': 0.9829700872018255, 'precision_label1': 0.788117394416607, 'recall_label1': 0.7825159914712153, 'f1_label1': 0.7853067047075606, 'auc': 0.9859677814706804, 'pr_auc': 0.8564023558456322, 'per_class_f1': {'0': 0.9911491413784992, '1': 0.7853067047075606}, 'confusion_matrix': [[33707, 296], [306, 1101]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.05759464621893551, 'threshold': 0.4550000000000003}

方案C运行结果: "test_metrics": {

"accuracy": 0.9841852417634511,

"macro_precision": 0.8990265209974933,

```

"macro_recall": 0.8919523147078002,
"macro_f1": 0.8954546222507791,
"weighted_precision": 0.9832784957495156,
"weighted_recall": 0.9834202417634511,
"weighted_f1": 0.9833463190159758,
"precision_label1": 0.8066624544138929,
"recall_label1": 0.7917564545454545,
"f1_label1": 0.7991395232974911,
"auc": 0.9890372639790593,
"pr_auc": 0.8762558340286597,
"per_class_f1": {
"0": 0.9913527212040671,
"1": 0.7994265232974911
},
"confusion_matrix": [
[33736, 267],
[ 293, 1114]
],
"num_classes": 2,
"num_samples": 35410,
"loss": 0.06836018314045486,
"threshold": 0.6830000000000005
}

```

不同配置下的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）

助手 2026-07-15 11:41:29

这次方案 C 的主要指标已经与 35,410 样本混淆矩阵基本一致，可以进行 A/B/C 的同测试集比较。我会先计算三组 FPR、FNR 和逐项增益，再同步修正 Results 中此前将 C 视为 84,380 样本的比较边界与层级基线。

助手 2026-07-15 11:42:18

方案定义也已从配置核对：A 将 flow statistics 平铺到每个 packet token；B 完全不用 flow statistics；C 保持 packet 与 flow 两条分支，并以受限 `token\_film` 进行后融合。接下来更新图与正文时，我会把“35,410 样本的受控 C 对比运行”和“供 Stage 2 使用的 84,380 样本 full-data C 运行”明确区分，避免再次混用。

助手 2026-07-15 11:46:02

这次方案 C 的核心指标已经与 35,410 样本混淆矩阵基本一致，因此 A/B/C 可以作为同一测试集上的受控对比。

****整体对比****

```

\[
\mathrm{FPR}=\frac{FP}{TN+FP},\quad
\mathrm{FNR}=\frac{FN}{FN+TP}.
\]

```

方案	Macro-P	Macro-R	Macro-F1	Class-1 F1	ROC-AUC	PR-AUC	FPR	FNR
A	0.8919	0.8600	0.8752	0.7599	0.9801	0.8326	**0.7764%**	27.2210%
B	0.8896	0.8869	0.8882	0.7853	0.9860	0.8564	0.8705%	21.7484%
C	**0.8990**	**0.8920**	**0.8955**	**0.7991**	**0.9890**	**0.8763**	0.7852%	**20.8244%**

总体排序非常清楚：

```
\[
C > B > A
\]
```

适用于 Macro-F1、Macro-Recall、Class-1 F1、ROC-AUC 和 PR-AUC。Macro-Precision 也是 C 最高，但 A 略高于 B。

A 与 B

方案 A 将 flow statistical features 平铺到每一个 packet token；方案 B 只使用 packet sequence。

B 相对 A：

- Macro-F1：提高 **0.0130**
- Macro-Recall：提高 **0.0269**
- Class-1 F1：提高 **0.0254**
- ROC-AUC：提高 **0.0059**
- PR-AUC：提高 **0.0238**
- FN：383 降至 306，减少 **77**
- FP：264 增至 296，增加 **32**

A 的 FPR 最低，但攻击漏检率高达 27.22%。B 牺牲了约 0.094 个百分点的 FPR，换取了明显更高的攻击检出率。

这说明将同一个 flow 向量重复复制到每个 packet token，可能会放大静态统计信息，使 packet ordering 和 timing information 的作用被削弱。纯 packet branch 反而获得了更好的少数类识别能力。

C 与 B

方案 C 保留 packet 与 flow 两个独立分支，在 packet sequence 编码完成后，通过受限 `token\_film` 融合 flow statistics。

C 相对 B：

- Macro-Precision：提高 **0.0095**
- Macro-Recall：提高 **0.0050**
- Macro-F1：提高 **0.0072**
- Class-1 F1：提高 **0.0138**
- ROC-AUC：提高 **0.0031**
- PR-AUC：提高 **0.0199**
- FP：296 降至 267，减少 **29**
- FN：306 降至 293，减少 **13**
- FPR：0.8705% 降至 0.7852%，相对降低约 **9.8%**

C 同时减少 FP 和 FN，因此提升不是由简单调整分类阈值造成的精度与召回率交换。它证明 flow statistics 在与 packet embedding 分支化建模后，能够提供 packet sequence 没有完全包含的补充信息。

C 与 A

C 相对 A:

- Macro-F1: 提高 **0.0202**
- Class-1 F1: 提高约 **0.0392**
- PR-AUC: 提高 **0.0436**
- FN: 383 降至 293, 减少 **90**
- FP: 264 增至 267, 仅增加 **3**
- FPR: 0.7764% 与 0.7852%, 几乎相同

这是三组结果中最有说服力的比较。C 在几乎不增加正常流误报的情况下, 显著减少攻击漏检。相对于在每个 packet 上重复注入 flow statistics, 独立编码后再融合明显更加有效。

PR-AUC 与 ROC-AUC

三个方案的 ROC-AUC 都较高, 但差异有限:

```
\[
0.9801 \rightarrow 0.9860 \rightarrow 0.9890.
\]
```

PR-AUC 的差异更加明显:

```
\[
0.8326 \rightarrow 0.8564 \rightarrow 0.8763.
\]
```

由于正类只占 $\backslash(1407/35410\approx3.97\%\backslash)$, PR-AUC 更能反映攻击样本的排序和检索能力。因此, C 的 PR-AUC 提升是比 Accuracy 提升更重要的结果。

阈值与 Loss

三个阈值分别为:

- A: 0.715
- B: 0.455
- C: 0.683

C 的阈值明显高于 B, 但 Recall 仍然更高, 同时 ROC-AUC 和 PR-AUC 也是最高。这支持 C 学到了更好的类别分离, 而不是依赖较低阈值扩大正类预测。

B 的 loss 最低, 为 0.0576, C 为 0.0684。不能仅凭 loss 选择 B, 因为 loss 还受到概率校准和训练目标影响; 论文的主要模型选择标准是验证集 Macro-F1, 测试集上应综合 Macro-F1、PR-AUC、FPR 和 FN。

剩余数据一致性问题

方案 C 的主要指标已正确, 但以下字段仍是旧运行残留:

- `weighted\_recall` 应为 **0.9841853**, 不是 0.9834202
- `weighted\_precision` 应约为 **0.9840496**
- `weighted\_f1` 应约为 **0.9841145**
- `per\_class\_f1['0']` 应约为 **0.9917686**
- `per\_class\_f1['1']` 应约为 **0.7991392**

AUC 和 PR-AUC 无法由混淆矩阵验证, 论文中可以暂时采用你提供的运行值, 但最好从同一次运行的 prediction CSV 重新计算。

我已更新 [results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:155), 将 A/B/C 改为共同 35,410-flow 的受控比较, 并增加了新的性能与 FP/FN 图:

![Stage 1 fusion comparison](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/stage1_fusion_comparison.png)

同时, 论文中已明确区分 “35,410-flow 的 A/B/C 对比运行” 和 “供 Stage 2 使用的 84,380-flow full-data C 运行”, 避免两套结果再次混用。

用户 2026-07-15 11:49:38

S1 各种模型的对比:

Model	Params	Time (s)	Accuracy	F1	Macro F1	Class1 Precision	Class1 Recall	Class1 ROC-AUC	PR-AUC							
threshold	precision_macro	recall_macro	precision_weighted	recall_weighted	f1_weighted	confusion_matrix										
Ours_BothEncoding	344963	4337.6	0.9830	0.8882	0.7853	0.7881	0.7825	0.9860	0.8564	0.4550	0.8896					
	0.8869	0.9830	0.9830	0.9830	[[33707, 296], [306, 1101]]											
Ours_TimeOnly	344963	2250.5	0.9822	0.8818	0.7728	0.7818	0.764	0.9846	0.851	0.4785	0.886	0.8776	0.982			
	0.9822	0.9821	[[33703, 300], [332, 1075]]													
Ours_PositionOnly	344963	1994.7	0.9812	0.8739	0.7575	0.7791	0.737	0.9846	0.8482	0.4703	0.8841	0.8642	0.9808	0.9812	0.981	[[33709, 294], [370, 1037]]
LSTM_with_time	217346	3556.3	0.9806	0.8677	0.7455	0.7786	0.715	0.9801	0.8077	0.4638	0.8834	0.8533	0.9799	0.9806	0.9802	[[33717, 286], [401, 1006]]
CNN1D_with_time	70274	1162.4	0.9803	0.867	0.7443	0.768	0.7221	0.9793	0.7821	0.4822	0.8782	0.8565	0.9798	0.9803	0.98	[[33696, 307], [391, 1016]]
GRU	162690	3684.4	0.9797	0.8595	0.7296	0.7748	0.6894	0.9799	0.8009	0.4565	0.881	0.8406	0.9788	0.9797	0.9791	[[33721, 282], [437, 970]]
LSTM	216834	3381.2	0.9792	0.8582	0.7273	0.7575	0.6994	0.9796	0.7976	0.4272	0.8726	0.845	0.9785	0.9792	0.9788	[[33688, 315], [423, 984]]
CNN1D	68354	1591	0.9779	0.8579	0.7274	0.712	0.7434	0.9809	0.7975	0.4203	0.8507	0.8655	0.9783	0.9779	0.9781	[[33580, 423], [361, 1046]]
BiLSTM	564738	2625.7	0.9778	0.8522	0.716	0.7273	0.705	0.9797	0.7903	0.4408	0.8575	0.8471	0.9775	0.9778	0.9776	[[33631, 372], [415, 992]]
Ours_NoEncoding	344963	2213.9	0.977	0.8507	0.7134	0.7069	0.72	0.9816	0.8003	0.4335	0.8477	0.8538	0.9772	0.977	0.9771	[[33583, 420], [394, 1013]]

Full metrics:

```
{'accuracy': 0.9617649135837334, 'macro_precision': 0.74145487832394, 'macro_recall': 0.809354661189905, 'macro_f1': 0.7761608462917445, 'weighted_precision': 0.9680282703050326, 'weighted_recall': 0.9626659135836359, 'weighted_f1': 0.9649653532785632, 'precision_label1': 0.5150728016480283, 'recall_label1': 0.6439233714721587, 'f1_label1': 0.523443421926911, 'auc': 0.9552327656164226, 'pr_auc': 0.5604476128765536, 'per_class_f1': {'0': 0.9805043503907979, '1': 0.5607973421926911}, 'confusion_matrix': [[33150, 853], [501, 906]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.031577753180528123, 'threshold': 0.42999999989999977}
```

这是stage1中, MLP的运行结果 (flow-level aggregation statistical features)

方案B, max_seq_len=128 and strategy = head

不同模型的运行结果。详细解析结果 (主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics, 并从这些数据进行分析)

助手 2026-07-15 11:49:55

我先校验 MLP 这一行，因为它现在与前面的模型使用了同一测试集（34,003 个正常、1,407 个攻击），因此终于可以做严格的 flow-statistics 对 packet-sequence 对比。不过当前 MLP 的 `macro\_precision`、`f1\_label1`、`per\_class\_f1` 和 weighted metrics 仍有互相冲突的旧值，我会以混淆矩阵重新计算阈值相关指标，并保留 AUC/PR-AUC 作为运行输出。

助手 2026-07-15 11:50:40

MLP 的混淆矩阵与其他模型现在具有相同类别规模，可以纳入主对比。它会被加入 Stage 1 模型柱状图，但参数量和训练时间仍未提供，所以不会被硬填进效率表；正文会明确把它作为同测试集的 flow-level aggregation baseline。

助手 2026-07-15 11:53:43

主要结论已经稳定：双编码 Transformer 在 Macro-F1、Macro-Recall、Macro-Precision、ROC-AUC 和 PR-AUC 上均为第一；GRU 的 FPR 略低，但它以明显更高的漏检率换取这一结果。MLP 则在同测试集上同时产生更多 FP 和 FN，能够作为 packet-sequence 建模价值的直接基线。

助手 2026-07-15 11:54:44

这组结果现在可以形成完整的 **Stage 1 模型主对比**。所有模型的测试集均为 35,410 个 flow，其中 34,003 个正常、1,407 个攻击，可以直接比较。

统一结果

\[

$\mathrm{FPR} = \frac{FP}{TN + FP}$, $\mathrm{FNR} = \frac{FN}{FN + TP}$.

\]

\]

Model	Macro-P	Macro-R	Macro-F1	ROC-AUC	PR-AUC	FPR	FP/FN
--- --- --- --- --- --- --- ---							
Ours Both	**0.8896**	**0.8869**	**0.8882**	**0.9860**	**0.8564**	0.8705%	296/306
Ours Time	0.8860	0.8776	0.8818	0.9846	0.8510	0.8823%	300/332
Ours Position	0.8841	0.8642	0.8739	0.9846	0.8482	0.8646%	294/370
LSTM+time	0.8834	0.8533	0.8677	0.9801	0.8077	0.8411%	286/401
CNN1D+time	0.8782	0.8565	0.8670	0.9793	0.7821	0.9029%	307/391
GRU	0.8810	0.8406	0.8595	0.9799	0.8009	**0.8293%**	282/437
LSTM	0.8726	0.8450	0.8582	0.9796	0.7976	0.9264%	315/423
CNN1D	0.8507	0.8655	0.8579	0.9809	0.7975	1.2440%	423/361
BiLSTM	0.8575	0.8471	0.8522	0.9797	0.7903	1.0940%	372/415
Ours No Encoding	0.8477	0.8538	0.8507	0.9816	0.8003	1.2352%	420/394
Flow-statistics MLP	0.7501	0.8094	0.7762	0.9552	0.5604	2.5086%	853/501

1. 双编码模型整体最佳

`Ours\_BothEncoding` 在五个主要综合指标上均为第一：

- Macro-Precision: 0.8896
- Macro-Recall: 0.8869
- Macro-F1: 0.8882

- ROC-AUC: 0.9860
- PR-AUC: 0.8564

其混淆矩阵为:

```
\[
\begin{bmatrix}
33707 & 296 \\
306 & 1101
\end{bmatrix}.
\]
```

虽然 GRU 的 FPR 略低, 但 GRU 漏掉437个攻击, 而双编码模型只漏掉306个。GRU 的低 FPR 是以明显降低攻击覆盖率为代价, 不能据此认为 GRU 更好。

2. Position 与 Time 的互补性

双编码相对 Time-only:

- Macro-F1: +0.0064
- Macro-Recall: +0.0093
- PR-AUC: +0.0054
- FP: 减少4个
- FN: 减少26个

双编码相对 Position-only:

- Macro-F1: +0.0143
- Macro-Recall: +0.0227
- PR-AUC: +0.0082
- FN: 减少64个, 仅增加2个FP

Time-only 明显强于 Position-only, 说明真实 packet timing 比单纯顺序更有信息。但组合后继续提升, 证明 packet order 与 elapsed time 不是重复信号。

相对 No-Encoding, 双编码:

- Macro-F1: 提高0.0375
- Macro-Precision: 提高0.0419
- Macro-Recall: 提高0.0331
- PR-AUC: 提高0.0561
- FP: 减少124
- FN: 减少88

这是支持 time-aware encoding 设计的核心消融证据。

3. 与传统序列模型比较

LSTM+time 是传统基线中 Macro-F1 最好的模型。双编码 Transformer 相比它:

- Macro-F1: +0.0205
- Class-1 F1: +0.0398
- PR-AUC: +0.0487
- ROC-AUC: +0.0059
- FN: 减少95

- FP: 增加10

也就是说，只增加10个误报便额外识别95个攻击 flow，属于明显更有利的错误交换。

CNN1D 的 Macro-Recall 相对较高，但产生423个 FP；GRU 的 FP 最少，但产生437个 FN。双编码模型在两类错误之间更均衡。

4. 与 Flow-Statistics MLP 比较

根据 MLP 混淆矩阵重新计算：

```
\[
CM_{\mathrm{MLP}}=
\begin{bmatrix}
33150 & 853 \\
501 & 906
\end{bmatrix}.
\]
```

双编码模型相对 MLP：

- Macro-F1: +0.1121
- Macro-Recall: +0.0775
- Macro-Precision: +0.1395
- Class-1 F1: +0.2130
- ROC-AUC: +0.0308
- PR-AUC: +0.2960
- FP: 减少557
- FN: 减少195
- FPR: 从2.5086%降至0.8705%

这说明 packet-sequence 模型不是单纯提高攻击召回率，而是同时减少误报和漏报。PR-AUC 提高近0.30，是 packet-level sequential representation 相对 flow aggregation 最有力的证据。

5. 性能与效率权衡

双编码模型训练时间为4,337.6秒，即72.29分钟，是该组中最慢的模型。CNN1D+time只需要1,162.4秒，但 Macro-F1低0.0212、PR-AUC低0.0743。

双编码模型有344,963个参数，并非最大模型；BiLSTM有564,738个参数但性能更差。因此提升不能简单归因于模型规模。不过训练时间需要在相同T4或L4 GPU、相同epoch和软件环境下才能形成严格效率结论。

MLP 指标需要修正

MLP提供的部分字段与混淆矩阵不一致。由矩阵计算的正确值应为：

- Accuracy: 0.961762
- Macro-Precision: 0.750089
- Macro-Recall: 0.809419
- Macro-F1: 0.776159
- Class-1 Precision: 0.515065
- Class-1 Recall: 0.643923
- Class-1 F1: 0.572331
- Weighted-F1: 0.963788

因此，原结果中的 `f1\_label1=0.523443`、`per\_class\_f1['1']=0.560797` 和 `macro\_precision=0.741455` 不应写入论文。AUC与PR-AUC无法由混淆矩阵计算，目前保留运行输出值。

我已将修正后的 MLP 加入

[results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:78) 和 Stage 1 模型图：

![Stage 1 model comparison](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/stage1_model_comparison.png)

论文可以得出：**time-aware packet-sequence Transformer 显著优于 flow-level aggregation、传统循环网络和卷积模型；位置与真实时间编码具有互补作用，但需要付出更高训练成本。**当前仍属于单随机种子结论，最终应补充多种子均值、标准差和显著性检验。

用户 2026-07-15 12:00:06

S1 各种模型的对比：

Model	Params	Time (s)	Accuracy	F1	Macro F1	Class1 Precision	Class1 Recall	Class1 ROC-AUC	PR-AUC	threshold	precision_macro	recall_macro	precision_weighted	recall_weighted	f1_weighted	confusion_matrix
Ours_BothEncoding	344963	4337.6	0.9830	0.8882	0.7853	0.7881	0.7825	0.9860	0.8564	0.4550	0.8896	0.8869	0.9830	0.9830	0.9830	[[33707, 296], [306, 1101]]
Ours_TimeOnly	344963	2250.5	0.9822	0.8818	0.7728	0.7818	0.764	0.9846	0.851	0.4785	0.886	0.8776	0.982	0.9822	0.9821	[[33703, 300], [332, 1075]]
Ours_PositionOnly	344963	1994.7	0.9812	0.8739	0.7575	0.7791	0.737	0.9846	0.8482	0.4703	0.8841	0.8642	0.9808	0.9812	0.981	[[33709, 294], [370, 1037]]
LSTM_with_time	217346	3556.3	0.9806	0.8677	0.7455	0.7786	0.715	0.9801	0.8077	0.4638	0.8834	0.8533	0.9799	0.9806	0.9802	[[33717, 286], [401, 1006]]
CNN1D_with_time	70274	1162.4	0.9803	0.867	0.7443	0.768	0.7221	0.9793	0.7821	0.4822	0.8782	0.8565	0.9798	0.9803	0.98	[[33696, 307], [391, 1016]]
GRU	162690	3684.4	0.9797	0.8595	0.7296	0.7748	0.6894	0.9799	0.8009	0.4565	0.881	0.8406	0.9788	0.9797	0.9791	[[33721, 282], [437, 970]]
LSTM	216834	3381.2	0.9792	0.8582	0.7273	0.7575	0.6994	0.9796	0.7976	0.4272	0.8726	0.845	0.9785	0.9792	0.9788	[[33688, 315], [423, 984]]
CNN1D	68354	1591	0.9779	0.8579	0.7274	0.712	0.7434	0.9809	0.7975	0.4203	0.8507	0.8655	0.9783	0.9779	0.9781	[[33580, 423], [361, 1046]]
BiLSTM	564738	2625.7	0.9778	0.8522	0.716	0.7273	0.705	0.9797	0.7903	0.4408	0.8575	0.8471	0.9775	0.9778	0.9776	[[33631, 372], [415, 992]]
Ours_NoEncoding	344963	2213.9	0.977	0.8507	0.7134	0.7069	0.72	0.9816	0.8003	0.4335	0.8477	0.8538	0.9772	0.977	0.9771	[[33583, 420], [394, 1013]]

Full metrics:

```
{'accuracy': 0.9617624135837334, 'macro_precision': 0.75008937832394, 'macro_recall': 0.809419261189905, 'macro_f1': 0.7761590462917445, 'weighted_precision': 0.9680282703050326, 'weighted_recall': 0.9626659135836359, 'weighted_f1': 0.9649653532785632, 'precision_label1': 0.5150651016480283, 'recall_label1': 0.6439233714721587, 'f1_label1': 0.572331421926911, 'auc': 0.9552327656164226, 'pr_auc': 0.5604476128765536, 'per_class_f1': {'0': 0.9805043503907979, '1': 0.5723313421926911}, 'confusion_matrix': [[33150, 853], [501, 906]], 'num_classes': 2, 'num_samples': 35410, 'loss': 0.031577753180528123, 'threshold': 0.42999999989999977}
```

这是stage1中，MLP的运行结果（flow-level aggregation statistical features）

方案B， max_seq_len=128 and strategy = head

不同模型的运行结果。详细解析结果（主要关注macro_f1、macro_recall、macro_precision、pr_auc、auc、confusion_matrix和FPR这些metrics，并从这些数据进行分析）
stage1都是在colab-T4 GPU 下运行的

助手 2026-07-15 12:00:28

我会先把所有模型的混淆矩阵统一换算成 FPR/FNR，再按“检测质量、编码消融、模型效率”三个层面分析。MLP 这组也需要先做一致性检查，因为它的新混淆矩阵已经与其他 Stage 1 模型共享同一个 35,410-flow 测试集，但部分 weighted/per-class 字段看起来仍来自旧运行。

助手 2026-07-15 12:01:29

这组结果补全后，MLP 的新混淆矩阵与其他模型拥有相同的 $\frac{34003}{1407}$ 类别分布，因此它不再只是“补充参考”，可以正式纳入受控 Stage 1 对比。我会同步把 MLP 加入模型柱状图，并把训练时间结论改为同一 Colab T4 GPU 下的可比结果。

助手 2026-07-15 12:03:18

性能柱状图已经纳入 MLP。为了让 confusion matrix 和 FPR 不只停留在文字里，我再增加一幅统一的 FP/FN 图；由于所有 Stage 1 模型共享 34,003 个负类样本，FP 条形长度与 FPR 完全成比例，可以直接看到低误报是否以更多漏检为代价。

助手 2026-07-15 12:06:09

这组实验是一个受控的 **Stage 1、方案 B、 $(L=128)$ 、head selection** 模型比较。所有模型共享：

```
\[
N=35410,\quad N_0=34003,\quad N_1=1407.
\]
```

因此 Macro-F1、PR-AUC、FPR、FP/FN 和运行时间可以直接比较。

****关键结果****

Model	Macro-P	Macro-R	Macro-F1	ROC-AUC	PR-AUC	FPR	FNR
--- ---	---	---	---	---	---	---	---
Ours Both	**0.8896**	**0.8869**	**0.8882**	**0.9860**	**0.8564**	0.8705%	**21.7484%**
Ours Time	0.8860	0.8776	0.8818	0.9846	0.8510	0.8823%	23.5963%
Ours Position	0.8841	0.8642	0.8739	0.9846	0.8482	0.8646%	26.2971%
LSTM+time	0.8834	0.8533	0.8677	0.9801	0.8077	0.8411%	28.5004%
CNN1D+time	0.8782	0.8565	0.8670	0.9793	0.7821	0.9029%	27.7896%
GRU	0.8810	0.8406	0.8595	0.9799	0.8009	**0.8293%**	31.0590%
LSTM	0.8726	0.8450	0.8582	0.9796	0.7976	0.9264%	30.0640%
CNN1D	0.8507	0.8655	0.8579	0.9809	0.7975	1.2440%	25.6574%
BiLSTM	0.8575	0.8471	0.8522	0.9797	0.7903	1.0940%	29.4954%
Ours No Encoding	0.8477	0.8538	0.8507	0.9816	0.8003	1.2352%	28.0028%
Flow-statistics MLP	0.7501	0.8094	0.7762	0.9552	0.5604	2.5086%	35.6077%

1. 所提模型的总体表现

`Ours\_BothEncoding` 同时取得最高：

- Macro-Precision: 0.8896
- Macro-Recall: 0.8869
- Macro-F1: 0.8882
- Class-1 F1: 0.7853
- ROC-AUC: 0.9860
- PR-AUC: 0.8564

其混淆矩阵为:

```
\[
\begin{bmatrix}
33707 & 296 \\
306 & 1101
\end{bmatrix}.
\]
```

在 1,407 个攻击 flow 中识别出 1,101 个, 漏检 306 个; 在 34,003 个正常 flow 中产生 296 个误报。

这说明模型优势不是单纯提高 Precision 或 Recall, 而是两个类别上的综合平衡最好。

2. 与最强传统序列基线比较

按 Macro-F1, 最强非 Transformer 基线是 LSTM+time:

```
\[
F_{1,\mathrm{macro}}=0.8677.
\]
```

Ours Both 相对 LSTM+time:

- Macro-F1: 提高 **0.0205**
- Macro-Recall: 提高约 **0.0336**
- Class-1 F1: 提高 **0.0398**
- PR-AUC: 提高 **0.0487**
- ROC-AUC: 提高约 **0.0059**
- FN: 401 降至 306, 减少 **95**
- FP: 286 增至 296, 仅增加 **10**

LSTM+time 的 FPR 略低, 但所提模型以10个额外误报换取95个更少的攻击漏检, 这是明显更有利的检测平衡。

3. Position 与 Time Encoding 消融

与 No Encoding 相比, Both Encoding:

- Macro-F1: 提高 **0.0375**
- Macro-Precision: 提高 **0.0419**
- Macro-Recall: 提高 **0.0331**
- PR-AUC: 提高 **0.0561**
- FP: 420 降至296
- FN: 394 降至306

因此性能提升不能只归因于 Transformer 主干, position/time encoding 本身提供了明显贡献。

Both 相对 Time Only:

- Macro-F1: +0.0064
- Macro-Recall: +0.0093

- PR-AUC: +0.0054
- FP: 减少4
- FN: 减少26

Both 相对 Position Only:

- Macro-F1: +0.0143
- Macro-Recall: +0.0227
- PR-AUC: +0.0082
- FP: 增加2
- FN: 减少64

时间编码单独比位置编码更有效，但最佳结果需要二者结合。这支持“packet order”和“真实时间间隔”提供互补信息的论文论点。

4. FPR 不能单独判断模型优劣

GRU 的 FPR 最低，为 0.8293%，但它有：

- Class-1 Recall: 0.6894
- FN: 437
- FNR: 31.06%

Ours Both 只比 GRU 多14个 FP，却减少了131个 FN。因此 GRU 的低 FPR主要来自更保守的预测，而不是更好的整体检测能力。

同样，CNN1D 的 Class-1 Recall 达到 0.7434，但产生423个 FP，FPR达到1.2440%。这再次说明论文应同时报告 Macro-F1、Class-1 Recall、FPR和混淆矩阵。

5. 时间信息对传统模型的影响

LSTM+time 相对普通 LSTM:

- Macro-F1: 0.8582 → 0.8677
- Class-1 F1: 0.7273 → 0.7455
- PR-AUC: 0.7976 → 0.8077
- FP: 减少29
- FN: 减少22

因此时间信息对 LSTM 有稳定正向作用。

CNN1D+time 的结果更复杂：它提高了 Macro-F1 和 Class-1 F1，并减少116个 FP，但 PR-AUC从0.7975下降到0.7821，FN增加30。这说明选定阈值下的 F1 改善，不一定意味着整个阈值范围内的排序能力改善。

6. 与 flow-statistics MLP 比较

MLP 的混淆矩阵为：

```
\[
\begin{bmatrix}
33150 & 853 \\
501 & 906
\end{bmatrix}.
\]
```


Ours Both 相对 MLP:

- Macro-F1: 提高约 **0.1121**
- Macro-Precision: 提高约 **0.1395**
- Macro-Recall: 提高约 **0.0775**
- ROC-AUC: 提高约 **0.0307**
- PR-AUC: 提高约 **0.2960**
- FP: 减少 **557**
- FN: 减少 **195**
- FPR: 2.5086% 降至0.8705%, 相对降低约 **65.3%**

这是支持 RQ1 的重要结果: packet sequence、packet order 和 timing information 相比单纯 flow aggregation, 不仅改善少数类排序能力, 也同时降低误报和漏报。

7. T4 GPU 上的效率权衡

所有 Stage 1 模型都在 Colab T4 GPU 上运行, 因此硬件层面的训练时间可以比较:

- 最快: CNN1D+time, 1,162.4秒
- Ours Both: 4,337.6秒
- Ours Both 约为 CNN1D+time 的 **3.73倍**
- Ours Both 比 LSTM+time 慢约 **22%**
- BiLSTM 参数最多, 为564,738, 但性能明显低于344,963参数的 Ours Both

因此, 所提模型的优势不是更大参数量, 而是更合适的时序建模。不过它的训练成本最高, 论文中应将其描述为准确性与训练成本之间的权衡。

MLP 的参数量和训练时间尚未提供, 最终效率表需要补上这两个值。

数据一致性提醒

MLP 的主要指标与新混淆矩阵一致, 但部分字段仍是旧值。由当前矩阵重新计算应为:

```
accuracy      = 0.9617622
weighted_precision = 0.9664348
weighted_recall   = 0.9617622
weighted_f1      = 0.9637884
f1_class0       = 0.9799864
f1_class1       = 0.5723310
```

论文中建议采用这些重新计算值; ROC-AUC和PR-AUC继续采用逐样本预测概率得到的运行结果。

相关结果、T4说明以及新增 FP/FN 图已经更新到

[results.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/results.tex:76)。图形位于:

- [模型性能图](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/stage1_model_comparison.png)
- [FP/FN与FPR图](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/stage1_error_comparison.png)

用户 2026-07-15 14:40:57

结合我上面已有的模块, 我不希望总是重复和冗余的内容。
请给我提供9Conclusion and Future Work 的完整latex。

助手 2026-07-15 14:41:09

我先核对现有章节中的研究问题、贡献表述和最终结果口径，再写结论，确保第 9 章是在“收束论证”，而不是把 Introduction 和 Results 换一种说法重复一遍。随后我会把可直接 `\input{}` 的 LaTeX 章节写入项目。

助手 2026-07-15 14:41:52

现有 Results 已经在章节末尾回答过 RQ1/RQ2，所以 Conclusion 不会再逐项复述所有消融表。我会只保留能够代表整篇论文的最终证据，并把“当前结论边界”和“下一步验证优先级”写清楚，尤其避免把不同测试清单上的结果直接作差。

助手 2026-07-15 14:42:48

我会新增 `doc/conclusion_future_work.tex`，章节结构为：总体结论、两项研究问题的最终回答、论文贡献、结论边界、分优先级的未来工作和收束段。数值只引用最能支撑主张的结果，并明确区分 35,410-flow 与 84,380-flow 两个测试清单。

助手 2026-07-15 14:44:10

章节已经写入。现在我在做最后的 LaTeX 检查，重点核对交叉引用、环境配对和指标口径；这章没有新增文献依赖，因为结论应综合本论文已经建立的证据，而不是在最后一章引入新的相关工作。

助手 2026-07-15 14:45:00

已完成第 9 章的完整 LaTeX：

[conclusion_future_work.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/conclusion_future_work.tex:7)

章节包括：

- 对整篇论文核心问题的总结
- RQ1 与 RQ2 的直接回答
- 四项主要研究贡献
- 结论适用范围与局限
- 统计可靠性和复现
- 跨数据集与跨时间验证
- 流式部署及端到端效率
- 自适应多关系上下文
- 预训练、鲁棒性和可解释性
- Closing Remarks

内容只保留了支撑最终结论的关键数字，并严格区分了 `35,410-flow` 和 `84,380-flow` 两个测试清单，避免重复 Results 中的大量表格和逐模型分析。结论同时明确说明：当前证据支持模型在公司数据上的有效性，但尚不能直接推广为跨数据集、零日攻击或所有加密流量上的普遍结论。

在论文主文件中，将它放在 Results and Discussion 后、参考文献前：

```
\input{doc/conclusion_future_work}
```

不需要手动写数字 `9`；只要它前面是第 8 章，LaTeX 会自动显示为 **Chapter 9**。本章没有新增参考文献，这是有意的：Conclusion 应综合前文已经引用和验证的证据，而不应在最后一章引入新的相关工作。

静态检查已通过：环境、花括号和交叉引用均配对正确；当前环境没有安装 LaTeX 编译器，因此未执行整篇 PDF 编译。

用户 2026-07-15 15:01:36

我能想到的future work:

- 1、首先在资源限制，我希望能pre-train，可以学习到更好的flow representation；
- 2、stage2的transformer，relation-aware transformer，或者其他的context构建方式，因为不同的attack可能有不同的context等。

助手 2026-07-15 15:02:04

这两点都很适合作为真正的“方法延伸”，而且比泛泛地写“更多数据、更多实验”更有研究价值。我会把它们提升为 Future Work 的前两项：第一项明确为资源受限条件下的自监督预训练，第二项明确为面向攻击行为的关系感知、动态上下文建模；重复性验证和部署评估则压缩为后续验证工作。

助手 2026-07-15 15:02:42

有一个需要在论文里主动说明的小点：预训练通常会增加前期计算成本，因此不能写成“预训练解决资源限制”。更严谨的逻辑是：本研究受资源限制未开展大规模预训练；未来可采用一次性、自监督且资源高效的预训练，以减少对标注数据的依赖，并通过冻结编码器或参数高效微调控制下游成本。关系感知 Stage 2 则可以直接从你目前`source\_host`最优、`endpoint`高召回的结果自然引出。

助手 2026-07-15 15:03:42

两项方向已经改成 Future Work 的主轴，并补成了可执行的研究方案：Stage 1 明确了预训练任务、资源控制和评估方式；Stage 2 明确了关系集合、relation-aware attention 的实现选择、因果约束，以及为什么必须增加 attack-family 层面的评估。现在我检查新增引用和 LaTeX 结构。

助手 2026-07-15 15:04:30

这两个方向非常合适，而且能够从当前实验结果自然推导出来。我已经将它们提升为 Future Work 的两个核心研究方向：

- [资源受限条件下的自监督预训练](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/conclusion_future_work.tex:148)
- [面向攻击类型的 Relation-Aware Context Modeling](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/conclusion_future_work.tex:184)

完整文件已更新：[conclusion_future_work.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/conclusion_future_work.tex:139)

****第一项：Stage 1 预训练****

论文中现在明确说明：

- 当前受计算资源限制，没有进行大规模预训练。
- 未来可以利用大量无标签流量进行自监督预训练。
- 可采用 masked packet-feature modelling。
- 可预测下一个 packet 的方向、长度区间和 inter-arrival time。
- 可进行 packet temporal-order verification。

- 可使用同一 flow 不同子序列之间的 contrastive learning。
- 比较随机初始化、冻结 encoder、部分微调和完整微调。
- 不只观察 macro-F1，还要验证 label efficiency、rare-attack detection 和跨数据集迁移能力。

同时避免了“预训练可以减少计算资源”的不严谨表述。预训练本身会增加前期计算成本，但可以一次训练、多次复用，并减少下游任务对标注数据的依赖。

****第二项：Relation-Aware Stage 2****

这一方向有很强的实验依据：

- `source\_host` 得到最好的整体 macro-F1 和 PR-AUC。
- `endpoint` 得到更高的 attack recall，但产生更多 FP。
- 这说明不同关系可能适合不同攻击，而不存在对所有攻击都最优的单一 context。

论文中进一步提出：

- 扫描攻击可能需要“同一 source 访问多个 destination”的关系。
- DDoS 可能需要“多个 source 指向同一 destination”的关系。
- 暴力破解可能依赖重复的 source-service 交互。
- 横向移动可能依赖多个 endpoint 之间的时间链路。
- 为不同 context token 加入 relation embedding 或 relation-specific attention bias。
- 使用不同关系的 key/value projection。
- 通过 target-conditioned gate 动态组合多种关系。
- 根据流量密度动态选择时间窗口或 flow-count window。
- 所有 context 仍严格限制为目标 flow 之前结束的 flow，避免未来信息泄漏。

此外，我保留并压缩了两类必要但不是主要创新的 Future Work：

1. 多随机种子、置信区间、PR 曲线和跨时间/跨网络验证。
2. 流式部署、端到端延迟、内存、鲁棒性和异常网络条件测试。

新增引用使用了现有 `references.bib` 中的：

```
\cite{2024MAETraffic,Dong2025PretrainingEncrypted}
```

LaTeX 环境、花括号和引用键检查均已通过。

用户 2026-07-15 15:23:59

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最终的论文的latex文件。

你是一名论文学做专家：仔细阅读和详细解析我的论文。

我现在有几个需要你帮我优化我的论文：1、Problem Formulation and System Overview 和 Proposed Methodology这两个模块是不是有很多重复？我该如何处理？

2、检查所有的模块，是不是有很多重复的表述，很冗余，帮我精简

3、检查段落直接的衔接

4、每个段落中的chapter summary是否有必要？

5、conclusion中的contribution是否有必要存在？

6、对于Result and Discussion,这个段落是不是太多的数据数字上的对比，但是discussion相对不够深入？

7、Experimental Methodology 和其他模块是否重复冗余太多？

详细检查我的论文中重复是否太多？如果有的话我该减少或者融合到一个地方？是不是描述不够精简或者精准？我该如何修正？

用户 2026-07-15 15:24:07

<turn_aborted>

The user interrupted the previous turn on purpose. Any running unified exec processes may still be running in the background. If any tools/commands were aborted, they may have partially executed.

</turn_aborted>

用户 2026-07-15 15:24:23

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最终的论文的latex文件。

你是一名NIDS方面的论文写作专家：仔细阅读和详细解析我的论文。

我现在有几个需要你帮我优化我的论文：1、Problem Formulation and System Overview 和 Proposed Methodology这两个模块是不是有很多重复？我该如何处理？

2、检查所有的模块，是不是有很多重复的表述，很冗余，帮我精简

3、检查段落直接的衔接

4、每个段落中的chapter summary是否有必要？

5、conclusion中的contribution是否有必要存在？

6、对于Result and Discussion,这个段落是不是太多的数据数字上的对比，但是discussion相对不够深入？

7、Experimental Methodology 和其他模块是否重复冗余太多？

详细检查我的论文中重复是否太多？如果有的话我该减少或者融合到一个地方？是不是描述不够精简或者精准？我该如何修正？

助手 2026-07-15 15:24:35

我会把这次当作一次论文结构与论证审稿，而不是只做语言润色。我先从`main.tex`还原实际章节顺序和被纳入编译的文件，再逐章检查“职责边界、重复观点、段落衔接、结果与讨论比例”，最后给出哪些内容应删除、迁移、合并以及建议保留的位置。

助手 2026-07-15 15:24:53

我已经确认最终论文不是通过`\input{}`拼接，而是把全部章节直接合并在一个约 3,870 行的`main.tex`中。初步就能看到几个高风险重复点：Introduction 已经详细讲了一遍完整两阶段流程和贡献；Problem Formulation 又包含 Stage 1/2 的实现性公式；Proposed Methodology 再次完整展开同一流程；Results 末尾、Conclusion 开头又各回答一次 RQ。接下来我会用段落级相似度和逐节职责检查把这些重复定位到具体行。

助手 2026-07-15 15:25:34

目前可以明确判断：两章确实重复，而且不是“必要的术语重复”，而是公式、context relation 定义、训练目标和实验控制都出现了两次。为了区分哪些是概念承接、哪些是可删的内容，我会按段落计算跨章节相似度，同时检查相同表格/公式的功能是否被后续章节再次承担。

助手 2026-07-15 15:29:02

结构审查的主结论已经很清楚：当前正文约 18,500 个英文词，其中 Problem Formulation 有 26 个公式、Proposed Methodology 又有 30

个公式。这说明前者已经越过“定义问题”的职责，提前承担了近半个方法章节；Experimental Methodology 又重复了数据划分、Stage 2 因果策略、baseline 定义和训练配置。Results

末尾则连续三次收束同一结论。接下来我会把 Results 的“数字多、讨论浅”问题和所有编译前硬错误一起核完。

助手 2026-07-15 15:32:05

除了冗余，我还发现几项必须先修的硬问题：Abstract

仍是模板占位文字；正文有一处未注释的乱码；实验环境宏被 `\providecommand` 重复定义，导致后面的 T4/L4 值实际上不会覆盖前面的占位符；当前 `sources.bib` 缺少 7 个正文正在引用的键；Stage 2 主结果表的列数和行内容也不一致。这些会直接影响提交质量，我会把它们与结构精简方案分成“必须修”和“建议修”两层记录。

助手 2026-07-15 15:34:28

审查已经完成，我把详细的逐章删除/迁移方案写成了独立报告，没有直接大规模改动 `main.tex`，因为这一步涉及章节职责重划，先保留原稿更稳妥。最后我再核对报告中的行号和关键问题清单，然后给你一个按优先级可执行的结论。

助手 2026-07-15 15:35:41

****总体判断****

是的，当前论文存在比较明显的结构性重复，但研究主线本身是清楚的。问题主要不是句子重复，而是同一项论证在多个章节被重新完成：

- Problem Formulation 有 26 组公式。
- Proposed Methodology 又有 30 组公式。
- Results 末尾、Results Summary、Conclusion 又连续回答三次 RQ1/RQ2。
- Proposed Methodology 和 Experimental Methodology 重复训练配置、baseline 和因果上下文规则。

详细逐行审查报告已经写入：

[thesis_editorial_audit.md](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/thesis_editorial_audit.md:1)

****必须先修****

在精简之前，有几项提交级问题必须处理：

1. [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:39) 的 Abstract 仍然是模板提示语。
2. [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:47) 中有 `Machine leanring` 拼写错误。
3. [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:154) 使用了无效的 `\begin{table}[t][H]`。
4. [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1574) 中 `j_k < i, quad` 缺少反斜杠。
5. 第 2331 行存在未注释乱码，可能出现在最终 PDF。
6. 第 2421 和 2440 行重复用 `\providecommand` 定义 `\ExpGPU`，后面的 T4/L4 内容不会覆盖前面的占位符。
7. Stage 2 主结果表列数不一致：[main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:3347)。
8. [sources.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/sources.bib) 缺少 7 个正在使用的引用键，包括 `Fawcett2006ROC`、`Pedregosa2011ScikitLearn` 和 `NVIDIA/Colab` 引用。

****1. Problem Formulation 与 Proposed Methodology****

确实重复较多。

Problem Formulation 应只保留：

- 检测任务与研究范围
- 设计约束
- 输入及符号定义
- 一个高层系统图
- Stage 1 抽象映射
- Stage 2 因果约束和抽象映射

建议将 [Stage 1 formulation](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:499) 压缩为：

```
\mathbf{z}^{\{1\}}_i =
F_{\{\theta_1\}}(\mathbf{x}_i, \boldsymbol{\tau}_i,
\mathbf{m}_i, \mathbf{s}_i).
```

Stage 2 只保留：

```
p_i =
G_{\{\theta_2\}}\left(
\mathbf{z}^{\{1\}}_i, \mathcal{C}_i
\right),
\quad
j \in \mathcal{C}_i \rightarrow t_j^{\{\mathrm{end}\}} < t_i^{\{\mathrm{start}\}}.
```

Attention pooling、FiLM、四种 relation、target-query attention、no-history fallback 等全部只放在 [Proposed Methodology](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1636)。

Problem Formulation 的 focal loss、threshold selection 和 RQ mapping table 也应移除。实验矩阵已经承担了 RQ mapping 的功能。

2. 其他重复内容

- Introduction 对完整 pipeline 描述过细，应压缩为一个高层方法段落。
- Related Work 每节末尾都在重新介绍“本论文如何不同”，大部分应集中到`Summary and Research Gap`。
- Proposed Methodology 的`Stage 2 Control Models`应移动到 Experimental Methodology。
- Proposed Methodology 中的学习率、batch size、patience、scheduler 应删除，只在 Experimental Methodology 出现一次。
- Introduction、Experimental Methodology 和 Results 开头都解释了一遍为什么使用 macro-F1、PR-AUC 和 FPR。完整定义只保留在 Experimental Methodology。
- Results 的`Stage 1 Answer to RQ1`、`Integrated Findings`、`Chapter Summary`和 Conclusion RQ answers 重复收束同一结论。

整体建议减少约 2,500 至 3,500 个英文词，论文会更聚焦。

3. 段落与章节衔接

局部段落衔接基本可读，但章节衔接需要调整：

- Introduction 第 95 行不应写“next section”，应使用`Chapter~\ref{...}`。
- Problem Formulation 结尾用两句话说明下一章如何从 PCAP 构造已定义变量。
- Proposed Methodology 结尾应明确：“下一章通过受控 baseline 和 ablation 验证这些组件。”
- Experimental Methodology 当前直接从环境表跳到 Results，需要增加一个结束段。
- Results 应删除 Chapter Summary，让 Cross-Stage Discussion 直接过渡到 Conclusion。

4. Chapter Summary 是否都有必要

不需要每章都有。

- Related Work 的`Summary and Research Gap`：保留。
- Problem Formulation 的 Chapter Summary：删除或缩成两句。

- Dataset: 不需要额外 Summary。
- Proposed Methodology: 保留一个短过渡段。
- Experimental Methodology: 不需要完整 Summary, 只需要结束句。
- Results: 删除当前 Chapter Summary。
- Conclusion 的 Closing Remarks: 可以保留, 但应控制在一个短段落。

****5. Conclusion 中 Contribution 是否必要****

当前独立的 [Principal Contributions](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:3679) 没有必要, 因为 Introduction 已有完整贡献列表。

建议删除四项枚举, 改成一个综合段落:

> Taken together, the thesis contributes a hierarchical formulation in which explicit intra-flow timing and causal inter-flow context provide separable sources of evidence for flow-level intrusion detection.

这样既回顾贡献, 又不会重新列一次 pipeline。

****6. Results and Discussion 是否数字太多****

是。Discussion 并非完全不足, 但大量段落先重复图中的六到十个数字, 最后才给一两句解释。

每个结果段建议采用:

1. Observation: 一个主指标和一个错误指标。
2. Mechanism: 为什么可能出现该结果。
3. Implication: 对 NIDS 的实际意义。
4. Boundary: 替代解释或缺少的控制实验。

尤其应增加:

- Stage 1 到 Stage 2 的错误迁移: 纠正多少 FN/FP, 又引入多少新错误。
- 按实际 context length 分组的性能。
- 按原始 flow packet count 分组的序列长度分析。
- `source\_host` 为什么可能优于 destination/endpoint 的行为解释。
- Scheme C 的 parameter-matched fusion control。
- 与 GTID、FlowTransformer、graph/window NIDS 的定性讨论。
- 恢复 single-seed reliability 限制, 当前它被注释掉了。

No-context 不应删除, 但应称为 lower bound。还应增加一个容量匹配的 `masked-history` control, 否则不能把全部 Stage 2 提升严格归因于 context。

****7. Experimental Methodology 是否重复****

重复较明显。它应该只负责:

- Experimental matrix
- baseline 和 ablation 定义
- 固定条件
- optimizer、batch、scheduler、seed
- checkpoint 和 threshold 规则
- metric 定义
- hardware 和 timing protocol

数据字段、split 构造过程和模型公式都应通过交叉引用处理。特别是当前章节标题包含`Thresholding`，但正文没有完整说明每组实验究竟最大化 validation class-1 F1、macro-F1 还是其他指标，这必须补充明确。

这次我没有直接重构

`main.tex`，以免在章节职责尚未确认前进行大范围删除。最合理的修订顺序是：先修硬错误和 Abstract，然后精简 Problem Formulation，再整理 Proposed/Experimental Methodology，最后重写 Results discussion 和 Conclusion。

用户 2026-07-15 15:43:02

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最终的论文的latex文件。

你是一名NIDS方面的论文写作专家：仔细阅读和详细解析我的论文。

我现在有几个需要你帮我优化我的论文：1、Problem Formulation and System Overview 和 Proposed Methodology这两个模块是不是有很多重复？我想去除 Problem Formulation and System Overview。

2、检查所有的模块，是不是有很多重复的表述，很冗余，帮我精简

3、检查段落直接的衔接

4、每个段落中的chapter summary是否有必要？

5、conclusion中的contribution是否有必要存在？

6、对于Result and Discussion,这个段落是不是太多的数据数字上的对比，但是discussion相对不够深入？

7、Experimental Methodology 和其他模块是否重复冗余太多？

详细检查我的论文中重复是否太多？如果有的话我该减少或者融合到一个地方？是不是描述不够精简或者精准？我该如何修正？

给我提供修正好的latex完整版

助手 2026-07-15 15:43:44

这次我会直接重构最终稿，而不是只给建议。核心操作是删除整个`Problem Formulation and System Overview`章节，把仍然必要的任务定义和因果约束并入`Proposed Methodology`；随后清理跨章节重复、重写 Results 的讨论结构、压缩 Conclusion，并同步修复引用、表格和实验环境问题。原始研究结果不会被改动。

助手 2026-07-15 15:46:10

我核对了实际代码中的选择规则：最终 Stage 1 Scheme C 以 validation PR-AUC 选 checkpoint，并在 validation PR 曲线上最大化 class-1 F1；最终 Stage 2 六模型同样以 validation PR-AUC 选 checkpoint，并在 0.01–0.99 的 199 个阈值上最大化 class-1 F1。论文现有 Experimental Methodology 对 threshold 写得不完整，而且 Stage 1 sampler 应为 10% 而不是 9%，我会按最终保存的 YAML 修正。

助手 2026-07-15 15:46:33

我现在开始实际重构正文。第一步会完整删除`Problem Formulation and System

Overview`，但不会丢失必要内容：任务范围已经在 Introduction 中，数据定义在 Dataset 中，模型和因果约束由 Proposed Methodology 统一承担，RQ 到实验的映射则由 Experimental Matrix 统一承担。

助手 2026-07-15 15:47:53

接下来先修正论文前端：补齐 Abstract，压缩 Introduction

中重复的两阶段流程和指标解释，保留唯一的贡献清单；Related Work

只负责评述文献与提出研究缺口，不再在每个小节末尾重复介绍本文实现。

助手 2026-07-15 15:52:00

模型章节现在已经成为唯一的公式来源：Stage 2 baseline 已移到实验设计，优化器和阈值细节从 Proposed Methodology 删除，架构表也只保留架构参数。下一步我会压缩 Experimental Methodology，并把代码中真实使用的 checkpoint/threshold 规则写清楚。

助手 2026-07-15 15:54:19

Experimental Methodology

已经只保留实验职责：固定数据条件、实验矩阵、baseline、真实训练配置、validation-only 阈值、指标和硬件测量。原来两百多行被注释的计划性文字已被一个可提交的有效章节结尾替代。现在进入 Results：我会保留图和结果值，但减少“报数”，增加机制、替代解释和证据边界。

助手 2026-07-15 15:58:35

我已经完成论文主体结构去重：删除了整个 Problem Formulation 章节，并把必要的定义与系统接口分别收回 Dataset、Methodology 和 Experimental Methodology。现在继续处理最需要“从报数转向解释”的 Results and Discussion，同时会把 Conclusion 的贡献重复压缩掉。

助手 2026-07-15 15:58:47

Results 里目前最明显的问题已经定位：context-window、模型对比和效率三个小节仍有“逐项复述表格”的痕迹，而且文中还残留了中文批注和未启用的 PR 曲线草稿。我会保留可证实的关键差异，把篇幅转向机制解释、证据边界和替代解释。

助手 2026-07-15 15:59:26

接下来我会直接改写 Results 的后半部分：删掉未使用的 PR-curve 注释和章末重复总结，新增可见的 Reliability and Validity，并明确区分“观察到的结果”“机制性解释”和“尚未被实验排除的替代解释”。这会让 discussion 更像论文论证，而不是指标清单。

助手 2026-07-15 16:01:05

Conclusion 中单独再列一次“Principal Contributions”没有必要，因为 Introduction 已经承担“论文贡献声明”，Results 又承担证据。这里我会改成简洁的结论性回答，并只保留真正限制结论外推的边界；这样不是删除贡献，而是让贡献只在最合适的位置出现一次。

助手 2026-07-15 16:01:52

论文主线现在已经更清楚：Dataset 只定义数据，Methodology 只定义模型，Experimental Methodology 只定义比较协议，Results 同时给出证据与边界，Conclusion 不再重新复制整套贡献和表格。下面我会做全篇静态审计，重点查残留引用、重复标签、未定义文献、中文批注、LaTeX 环境与表格结构。

助手 2026-07-15 16:03:04

静态审计发现两类“不会影响正文逻辑、但会降低最终稿整洁度”的问题：Methodology 里还有一整段已废弃的注释版训练流程，bibliography 也缺少实验章节实际引用的 7 个条目。我会一起清理，并修正唯一的重复标签来源。

助手 2026-07-15 16:04:36

关于 chapter summary: 独立的“本章总结”在这篇篇幅中大多会重复, 所以 Results 的总结已删除, Methodology 的总结标题也会取消, 只留下两句过渡。Related Work 的“Summary and Research Gap”则应保留, 因为它不是复述, 而是把文献缺口正式推导到研究设计。

助手 2026-07-15 16:05:44

引用与 LaTeX 结构审计现在已经通过: 没有活动状态下的未定义引用、重复标签、重复 BibTeX key 或不配对环境。编译器在本机不可用, 所以我会再做一轮“数据一致性审计”, 尤其核对表格中的 accuracy、FPR 和 confusion matrix 是否互相吻合, 避免文字优化后仍留下数字层面的答辩风险。

助手 2026-07-15 16:08:18

Experimental Methodology 审计又发现一个重要冲突: 它声称 window 实验保存了 context-length 分布, 但 Results 明确说现有汇总文件没有这些统计; 此外 no-context 被写成“测量 context 的增益”, 这在参数量不匹配时过强。我会按实际证据修正这两处, 并把所有具体训练超参数集中到 Experimental, Methodology 只保留公式与接口。

助手 2026-07-15 16:10:03

代码核对还暴露了一个答辩时很可能被问到的细节: Stage2 的“causal”实际含义是按`flow\_start\_timestamp`排序后只使用较早索引, 它没有额外强制`previous\_flow\_end <= target\_flow\_start`。我会在 Methodology 中把这个定义和边界说清楚, 并把严格的 embedding-availability 因果约束放入 Future Work, 避免论文声称超过代码实现的保证。

助手 2026-07-15 16:11:35

文件布局检查发现图路径也需要兼容处理: 本地图片实际位于`doc/figures`, 而正文写死为`MScCS/figures`。我会改成自动探测, 使它既能在当前`doc`目录编译, 也能兼容原 Overleaf 工程以`MScCS`为子目录的结构。

助手 2026-07-15 16:13:09

实验划分还有一个需要明确隔离的地方: Stage1 的 35,410-flow benchmark 与完整系统的 84,380-flow test manifest 不是同一测试集合。现在正文虽在 Results 里说明了, 但 Experimental 仍写成“所有实验使用同一 manifest”, 容易导致不合法的跨表比较。我会把两个证据块提前定义清楚, 并同步修正实验矩阵。

助手 2026-07-15 16:15:43

数字与文件审计目前没有发现新的算术矛盾: 主要 confusion matrix、FPR 和样本总数能对应, 8 个 Results 图文件也都存在。最后我会检查图的可读性和表格布局, 尤其确认坐标截断、图例与论文正文的解释一致。

助手 2026-07-15 16:16:50

图形检查整体良好, 但 efficiency 图确实存在标签相互遮挡: Vanilla 标签被 Proposed 星形覆盖, 三个 recurrent 模型的标注也过密。我会调整标注锚点并重新生成 PDF/PNG; 这比在正文里解释一个难读的图更有效。

助手 2026-07-15 16:19:13

Conclusion 的结构已经正确，但还有两处可再精简：开头重复列出全部 metrics，结论末尾还用“为了避免重复”这种编辑说明。最终稿不应向读者解释删改过程，我会改成直接、学术化的结论陈述。

助手 2026-07-15 16:20:48

最终静态结构检查已经全部通过：53 个活动引用均有 BibTeX 条目，交叉引用、标签、环境和花括号都配对，残留占位符与中文批注也已清除。由于本机既没有 LaTeX 编译器，也没有模板依赖 `mcs.sty`，我无法做最终 PDF 排版编译；我会用章节篇幅与改动范围再做最后一次平衡检查。

助手 2026-07-15 16:21:40

论文编辑已经完成，剩余只是收口说明。我保留了 Related Work 的“Summary and Research Gap”，删除了其他重复性章末总结；Results 现在约 2,900 词，数字陈述与机制讨论、证据边界基本平衡。

助手 2026-07-15 16:22:12

已完成整篇论文的重构与精简，修订后的完整版位于：

- [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex)
- [sources.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/sources.bib)

****主要修正****

1. 完整删除 `Problem Formulation and System Overview`。研究问题保留在 Introduction，数学符号和系统接口统一放入 Proposed Methodology，避免重复。
2. 明确章节职责：Introduction 负责贡献，Related Work 负责研究缺口，Dataset 负责数据，Methodology 负责模型，Experimental Methodology 负责实验协议。
3. 删除 Methodology 和 Results 中重复性的 `Chapter Summary`。只保留 Related Work 的 `Summary and Research Gap`，因为它承担文献综述到研究设计的逻辑推导。
4. 删除 Conclusion 中重复的 `Principal Contributions` 列表，改为直接回答 RQ1、RQ2，并总结整体学术贡献。
5. Results and Discussion 不再逐项复述所有数字，增加了：
 - 时间编码与位置编码的互补机制；
 - sequence length 和 context window 非单调变化的解释；
 - precision、recall、FP、FN 与 FPR 的运行意义；
 - no-context 参数量不匹配的因果解释限制；
 - Stage2 target-query 与 vanilla Transformer 的结构差异；
 - 单随机种子、缺少置信区间和外部验证的证据边界。
6. Experimental Methodology 现在集中记录优化器、采样、checkpoint、threshold 和硬件配置，不再与 Methodology 重复。
7. 明确隔离两个测试 manifest：
 - Stage1 对比：35,410 flows；
 - 完整 Stage2 对比：84,380 flows；
 - 禁止在两个 manifest 之间计算模型增益。
8. 根据代码澄清了 Stage2 的因果定义：当前实现保证按 flow start time 排序时不访问后续索引，但没有强制前序 flow 已结束。严格的 embedding-availability causality 已列为 future work。
9. 补全缺失的 7 个 BibTeX 条目，并修复错误 citation key。
10. 重新生成了 Results 图表，解决 efficiency 图中的标签遮挡；图片路径也能自动兼容本地 `figures/` 和原来的 `MScCS/figures/`。生成脚本位于 [generate_results_figures.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransf

ormer/doc/generate_results_figures.py:359)。

****检查结果****

- 未定义引用: 0
- 重复 LaTeX 标签: 0
- 未定义 citation: 0
- 重复 BibTeX key: 0
- 不匹配 LaTeX 环境: 0
- 活动引用: 53 篇
- 参考文献条目: 60 篇
- 8 个 Results 图文件均存在并完成视觉检查

当前环境没有 LaTeX 编译器, 同时缺少模板依赖 `mcs.sty`, 因此未能生成最终 PDF; 静态 LaTeX、文献、交叉引用和图文件检查均已通过。

用户 2026-07-16 09:41:11

Files mentioned by the user:

位置: \section{Alert-Derived Label Construction} These are programmatic or \emph{w...:
C:\Users\XiaoyanXiong\codex\attachments\00749836-ade1-4c16-879c-d2dfc26d5f54/pasted-text.txt

My request for Codex:

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最新的论文latex文件。

作为一名NIDS方面的专家: 仔细阅读和详细解析我的论文。

这是我看后的问题, 一一回答。

助手 2026-07-16 09:41:23

我先读取你列出的具体问题, 并对照最新版 `main.tex` 的对应段落、上下文和引用逐项核查。这里先以“论文审阅与回答”为主, 不会在未确认修改方向前直接改写文件。

助手 2026-07-16 09:41:37

问题列表已经读取到, 但附件文本存在 UTF-8 被错误解码后的乱码。我会先恢复原始中文, 再核对你最新文件中哪些段落已经被注释、删除或改动, 避免回答旧版本中已经不存在的问题。

助手 2026-07-16 09:42:16

最新版与你昨天的版本确实不同: `Sequential Stage Interface` 和 `Principal Configuration` 已被注释, 但 Experimental 仍引用 `tab:principal\_method\_config`, 所以出现 `??` 是必然的; 这不是编译偶发现象, 而是结构被注释后留下的悬空引用。接下来我会把“应保留、应精简、必须修正、可移至 limitation/future work” 分开回答。

助手 2026-07-16 09:45:36

我已经确认: 你担心的“负面表述”大多不是论文缺陷, 而是证据边界。真正需要优先处理的是三件事: 悬空引用、Stage1 两套 checkpoint 协议、以及把 Suricata 标签明确定位为 weak supervision 而非独立攻击真值。隐藏这些问题反而会让答辩风险更大。

助手 2026-07-16 09:49:00

我已仔细检查最新版 [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex) 和问题清单。先给结论：论文不存在致命的方法问题，但目前有三个必须优先处理的风险：

1. Suricata 弱标签的研究边界必须准确表述。
2. Stage 1 使用了两套 checkpoint 选择标准，严格对比性不足。
3. 注释章节后破坏了 LaTeX 层级，并产生了一个未定义引用 `\tab:principal_method_config`。

所有 citation key 都能在 `sources.bib` 中找到，目前没有未定义引用文献。

****1. Suricata 弱标签会不会使论文有问题？****

[当前表述](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:243)是必要且诚实的，不应删除。它不会使论文无效，但会限制结论：

- 模型学习的是“与 Suricata 告警相关的流量行为”，不是经过专家独立确认的真实攻击。
- 不能声称模型发现了 Suricata 漏报、实现了零日攻击检测，或优于 Suricata 本身。
- 模型预测为攻击、Suricata 未告警的样本，在当前评估中只能算 FP，不能解释为新攻击。

老师希望模型学习 Suricata 行为是可成立的研究目标。建议将任务定位为：

> Suricata-derived weak supervision for metadata-based and context-aware flow classification.

论文贡献不是“取代 Suricata”，而是研究包级时序、流统计和跨流上下文能否学习稳定的告警关联表示。由于没有外部数据，论文应明确定位为 proprietary single-capture case study。不同公开数据集分布不同，并不是隐瞒局限的理由。

最有价值的补充验证依次是：另一个时间段的 PCAP、未见主机测试、第二个 PCAP 使用相同 Suricata 配置标注，以及对模型与 Suricata 不一致样本进行小规模人工审查。

****2. 是否必须提供 Suricata 配置和规则？****

应提供可复现元数据，但不一定公开公司规则全文。至少记录：

- Suricata 版本与构建信息；
- ruleset 来源、版本或更新时间、文件哈希；
- 启用规则数量及类别/SID 分布；
- `HOME_NET`、流重组、应用层解析、checksum、threshold/suppress 配置；
- 执行命令、PCAP checksum；
- `custom-out` 插件的 Git commit 或版本。

如果规则或配置涉及公司机密，应写明“内部归档但不公开”，并在附录提供脱敏摘要。若当时没有保存快照，应如实列为 reproducibility limitation，不能事后假配置。

****3. `Validity, Privacy, and Reproducibility Considerations` 是否保留？****

建议保留，但压缩成两段。它承担四个重要功能：解释弱标签、单 PCAP 外部效度、抓包可见性限制，以及 IP/端口等敏感信息的处理。这些内容尤其适用于公司网络数据，删除后反而容易被审稿人追问。

Dataset 章节详细说明；Conclusion 中只用一句话概括，不要重复整段。

****4. 能否删除 `Sequential Stage Interface` 和 `Principal Configuration`？****

不能同时全部删除。

`Sequential Stage Interface` 说明 Stage 2 使用冻结的 Stage 1 embedding、是否端到端反向传播，这是方法定义的一部分。建议保留为一个简短段落，不必单独成为长 subsection。

`Principal Configuration` 表对复现很有帮助。当前正文仍在 [训练配置处](C:/Users/XiaoyanXiong/Desktop/NI DSTransformer/doc/main.tex:1308) 引用它，但表格被注释，因此出现 `??`。最佳处理是恢复精简版架构表；否则必须删除所有 `tab:principal\_method\_config` 引用，并把模型尺寸写入 Experimental Methodology。

另外，注释父级标题后，`Focal Objective` 被错误地归入 Stage 2；Experimental 中多个 `subsubsection` 也缺少父 `subsection`。应恢复：

```
\section{Imbalance-Aware Objective and Stage Interface}
\section{Training, Model Selection, and Thresholding}
```

****5. `Dataset Partition and Leakage Controls` 是否重复？****

不要删除，应改名为：

```
\section{Evaluation Manifests and Leakage Controls}
```

Dataset 章节负责解释数据如何构建和划分；Experimental 章节负责说明每组实验具体使用哪个 manifest，以及比较公平性。当前这里包含不可删除的信息：Stage 1 的 35,410 个测试流、Scheme C/Stage 2 的 84,380 个测试流，以及两组结果不可直接配对比较。

****6. Stage 1 是否存在“双重标准”？****

是的，当前确实存在实验协议不统一：

- 模型族 benchmark 按 validation class-1 F1 选择 checkpoint；
- principal Stage 1 按 validation PR-AUC 选择 checkpoint。

这比“checkpoint 用 PR-AUC、threshold 用 class-1 F1”更严重。后者不是双重标准：checkpoint 选择排序模型，threshold 选择工作点，二者职责不同。

理想修正是统一所有 Stage 1 模型、A/B/C 和消融实验：

```
Checkpoint selection: validation PR-AUC
Threshold selection: validation class-1 F1
Final reporting: untouched test set
```

如果无法重跑，应明确把旧模型比较称为 exploratory benchmark，把 Scheme C 称为 confirmatory configuration，并避免计算跨协议的精确提升百分比。尤其 A/B/C 三方案必须尽量使用同一 checkpoint 标准，否则不能被称为严格受控消融。

****7. `Stage1 Ablations and Sensitivity Tests` 是否重复？****

概念上不重复。Baselines 回答“比其他模型如何”，Ablations 回答“哪个组成部分起作用”。但当前文字与实验矩阵重复较多，可以合并为 `Stage 1 Comparisons`，下面用两个短段落分别介绍 baselines 和 ablations。

****8. `Validation-Only Decision Threshold` 怎么改？****

保留“只使用验证集确定阈值”，这是防止 test leakage 的重要设计。删除不同 checkpoint 规则的补丁式解释，统一写成：

```
For all confirmatory experiments, the checkpoint was selected by validation
PR-AUC. The decision threshold was then selected on the same validation split
by maximising class-1 F1, reflecting the operational importance of the minority
class. The test split was evaluated once without further tuning.
```

同时说明 macro-F1 是主要报告指标，而 class-1 F1 用于阈值选择，因为攻击类是实际工作目标。

****9. GPU 是否全部改成 L4？****

你之前明确说过 Stage 1 在 T4 上运行，这次又说全部使用 L4。不能为了论文一致性直接修改，必须以 Colab 日志、`nvidia-smi` 输出或运行记录为准。

若确认全部是 L4，需要同时修改 [GPU

宏](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1205)、Experimental efficiency 描述和 Stage 1 training-time 段落。若无法确认，应写 “Google Colab Pro+ hosted GPU runtime”，不要虚构具体 GPU。

****10. Figure 选择理由是否删除？****

“使用 figure 而不是 table” 这类写作过程说明没有学术价值，应删除。你已经将它注释，这是正确的。正文直接说明图展示什么、读者应该观察什么即可。

****11. “不能证明每类攻击都同等使用时间信息” 是什么意思？****

它表示你只有二分类标签，没有 attack-family

标签，因此只能证明时序编码在整体测试集上有效，不能证明其对扫描、DoS、C2 等攻击都同样有效。

建议保留含义，但写得更精准：

Because only binary labels are available, this ablation establishes an aggregate benefit and does not support attack-family-specific conclusions.

这是合理的研究边界，不是论文缺陷。

****12. FP/FN 是否保留精确数字？****

关键比较保留数字，次要比较改为文字。全部写成 “both lower” 证据不足；每一段都堆数字又会削弱讨论。

建议只保留三组：MLP 对 proposed、Scheme A 对 C、Stage 2 proposed 对 vanilla Transformer。其他地方写 “reduced both FP and FN” 或 “traded higher recall for more false alarms”，数字交给图表。

****13. Stage 1 training time 不够正面，是否删除？****

不要删除。效率结果不是只展示有利结果，诚实报告计算代价会增强可信度。压缩为：

The proposed encoder achieved the strongest detection performance but had the longest observed Stage-1 training time. Since wall-clock time includes model-specific early stopping and hosted-runtime variation, it is interpreted as descriptive computational cost rather than pure architectural throughput.

****14. 序列长度和截断策略结果如何讨论？****

非单调结果是有效发现。建议强调：

- `L=128` 在测试范围内达到最佳综合性能；
- `L=256` 没有进一步改善，说明更多 packet 不一定增加有效信息；
- head 与 head-tail 接近，head 因确定性和稍高攻击覆盖被选中；
- 结论仅适用于当前捕获，不能称为普遍最优。

更深入的后续分析应报告每个长度的截断比例，并按原始 flow length 分层评估。

****15. Source-host 的 “A plausible explanation” 能否删除？****

可以去掉弱化措辞，但不能把机制写成已经被证明。建议改为：

Source-host context groups repeated outbound connections from the same initiator, whereas global recency mixes unrelated flows and destination histories may combine many benign clients. This structural difference is consistent with the observed source-host advantage.

****16. Window size 和 no-context 的限制是否删除？****

都不应删除，也不应改成“完全正面”。

Window 结果建议写成：`W=128` 是当前测试范围的最佳平衡点，非单调趋势表明 context quality 比单纯扩大 context 更重要，但不能宣称它是通用最优。

No-context 必须说明其只有 514 个参数，远小于 proposed model，因此它是实际 lower bound，不是 capacity-matched ablation。最理想的补充实验是使用相同架构，屏蔽所有历史 token；这样才能更严格隔离 context 的贡献。

****17. Cross-stage discussion 放在哪里？****

拆开处理：

- 与已有研究结果一致的解释保留在 Cross-Stage Discussion；
- 二分类标签无法支持 attack-family 结论，移到 Limitations；
- error-transition 和按攻击关系分析，移到 Future Work。

****18. Single-seed 是否可以删除？****

如果确实只运行一个 seed，不能删除。建议写得简洁：

```
All neural comparisons are single-run results with fixed seed 130; small differences are therefore interpreted descriptively, and no statistical significance claim is made.
```

若想删除这一限制，需要对关键模型运行至少 3

个种子并报告均值、标准差或置信区间。对于硕士论文，资源限制下的 single-seed 可以接受，但必须公开。

****19. Conclusion 是否重复？****

Conclusion 必然会概括 Results，但不应再次复述所有数字。建议每个 RQ 只保留一句结论，最多包含一个 headline metric：

- RQ1：时间、位置和流级条件联合编码优于统计聚合与单一编码。
- RQ2：source-host context 在测试方法中取得最佳检测平衡。
- 最后总结两阶段设计带来的整体意义。

Contribution 不必在 Conclusion 再列完整清单；Introduction 已经列过。Conclusion 应保留 limitations 和 future work。

****20. 是否需要 Appendix？****

不是强制，但非常推荐，因为可以减少正文拥挤并提高可复现性。适合放入附录的内容包括：

- 脱敏后的 Suricata 版本、配置摘要、规则类别和哈希；
- 完整模型超参数及 YAML 配置；
- 全部 confusion matrices、threshold 和次要指标；
- 完整 packet/flow feature 列表；
- 代码版本和 commit 信息。

不要放公司 IP、原始 PCAP、机密规则或未经许可的内部配置。

****建议修改顺序****

第一优先级：统一 Stage 1 checkpoint 协议、核实 GPU、恢复标题层级、修复 `??`。

第二优先级：保留并精炼弱标签、single-seed、capacity mismatch 和单捕获限制。

第三优先级：减少 Results 中重复数字，将机制解释、操作意义和错误权衡写得更深入。

这轮我只完成了逐项审查，没有直接修改文件。

用户 2026-07-16 10:16:38

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最新的论文latex文件。

作为一名NIDS方面的专家：仔细阅读和详细解析我的论文。

所有实验都是使用的这个标准：

Checkpoint selection: validation PR-AUC

Threshold selection: validation class-1 F1

Final reporting: untouched test set

写作过程说明，这类描述全部去掉

这是最终要提交的论文，所以要注意措辞、严谨性等。

帮我修正，提供修正后的latex，并表明都修改了哪里，我好复查

助手 2026-07-16 10:16:49

我会直接在最新版 `doc/main.tex` 上进行终稿级修订：统一实验协议表述，删除所有“为何用图/表、本文如何组织展示”等写作过程说明，同时修复章节层级、交叉引用、重复和不够严谨的结论措辞。先做一次结构与引用审计，再分章节修改并编译验证。

助手 2026-07-16 10:17:15

初步审计已经确认：最新版仍保留了旧的“两套 checkpoint

规则”说明，与您现在确认的统一协议冲突；同时，注释掉父章节后留下了不合法的标题层级，并且

`tab:principal\_method\_config` 仍被正文引用但没有定义。我会把这些作为硬错误先修复，再精简 Results 中解释“为何使用某种图表”的元叙述。

助手 2026-07-16 10:19:13

审计已完成。除了协议冲突和未定义表格引用，我还发现两处公式排版错误：时间张量公式中的 ` $\max(\Delta \dots)$ ` 缺少数学命令，FiLM 初始化式中的 ` $\text{quad}$ ` 也缺少反斜杠；这些会一起修正。接下来我会按“方法定义只放 Methodology、实验选择只放 Experimental、证据与解释只放 Results”的边界重写重复段落。

助手 2026-07-16 10:19:19

我现在开始修改正文。第一批会处理 Abstract、Introduction、Related Work 和

Dataset：补回必要的研究边界，删除章节导读式元叙述，保留弱标签、隐私和可复现性说明，但压缩重复结论。

助手 2026-07-16 10:20:08

第二批修改集中在方法与实验协议。我会恢复必要的 Stage 1→Stage 2

冻结接口说明，但不恢复重复的架构总表；相反，直接把主配置写进唯一的实验配置表。这样既修复 `??`，也避免 Methodology 与 Experimental Methodology 重复。

助手 2026-07-16 10:21:57

现在处理 Results and Discussion 与

Conclusion。这里不会把限制“美化掉”，而是把它们移到唯一、合适的位置：Results

只保留与证据直接相关的限定，数据范围、单次运行、二分类标签和非容量匹配等统一收束到

Limitations；图表选择理由和章节导航全部删除。

助手 2026-07-16 10:24:30

正文的主要修订已完成，统一协议现在只保留一套，`\tab:principal_method_config`也已彻底移除。接下来进行机械审计：检查所有 `\ref`、`\cite`、重复 label、标题层级和残留的旧协议/T4表述，然后再做一次公式与数据一致性核对。

助手 2026-07-16 10:27:40

结构审计已经通过：当前没有未定义 `\ref`、重复 label 或缺失 citation key。复查 Results 时我还在收紧因果措辞，尤其避免把“完整 Stage 2 的整体提升”直接等同于“历史上下文的纯因果贡献”，并消除任何可能暗示根据测试集选择超参数的表述。

助手 2026-07-16 10:29:58

机械审计没有发现引用或结构错误。最后一轮语言润色会统一为英式学术英语（例如 `\modelling/behaviour/labell ed`），并修正少量过强或含混表述；文献标题和代码字段保持原样，不做拼写改写。

助手 2026-07-16 10:36:29

终稿静态验证已通过：144 个 label、33 个交叉引用、52 个实际引用的 bibliography key 均完整；没有重复 label、重复 bib key、环境不配对或缺失图文件。当前环境没有 `\latexmk/pdflatex`，而仓库中也没有模板依赖 `\mcs.sty`，所以我无法在本机生成最终 PDF；我会在交付说明中明确这一点。

助手 2026-07-16 10:37:52

已完成终稿修订。修正后的完整 LaTeX 位于：

[main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex)
[sources.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/sources.bib)
未修改，只进行了完整性检查。

****主要修改****

1. ****统一全部实验协议****

在 [Experimental Methodology](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1162) 明确统一为：

- checkpoint: validation PR-AUC;
- threshold: validation class-1 F1;
- final reporting: untouched test set.

删除了旧版 Stage 1 “validation class-1 F1 checkpoint” 及 “两套协议分别比较” 的全部内容。详细公式位于 [Checkpoint and Decision-Threshold Selection](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1284)。

2. ****修复章节层级和 `??`****

恢复了 [Imbalance-Aware Objective and Stage Interface](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1118) 和 [Training, Model Selection, and Thresholding](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1250)。将原来错误的 `\subsubsection` 调整为 `\subsection`。

删除未定义的 `\tab:principal_method_config` 引用，并将核心架构直接写入唯一的训练配置表。

3. **补回必要的 Stage 1→Stage 2 接口定义**

[Sequential Stage Interface](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1149)

现在简洁说明：

- Stage 1 使用验证集选出的 checkpoint;
- embedding 通过非采样 loader 导出;
- Stage 2 使用冻结 embedding;
- Stage 2 梯度不会更新 Stage 1。

4. **删除写作过程说明**

删除了：

- “本文其余章节组织如下” ；
- “本章将介绍.....” ；
- “使用 figure 而不是 table，因为.....” ；
- “下一章将.....” ；
- 图表展示选择的解释；
- 已注释的旧章节和旧实验协议。

5. **加强弱标签表述**

[Alert-Derived Label Construction](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:220)

保留并精炼了 Suricata 弱标签边界，明确模型学习的是 Suricata-alert-associated behaviour，不声称独立攻击真值、零日检测或优于 Suricata。

6. **精简重复并收紧因果措辞**

Results 中将 “therefore proves/validates” 等过强说法改为 “supports” 或 “is consistent with” 。
Scheme C、source-host、window size 和 target-query 的解释现在区分了观察结果、结构解释与尚未验证的机制。

7. **重新组织 Results and Discussion**

[Stage 1](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1407)

保留关键数值和错误权衡，减少次要 FP/FN 重复。

[Stage 2](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1499)

明确区分完整系统提升、上下文关系、窗口大小、架构和效率。

[Cross-Stage Discussion](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1593)

只保留跨阶段综合解释，不再夹杂 Future Work。

8. **集中所有研究限制**

[Limitations and Scope](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1617) 统一说明：

- 单一公司 PCAP；
- Suricata 弱标签；
- 固定 seed、未估计训练方差；
- 二分类标签无法支持 attack-family 结论；
- Stage 2 是 start-ordered causality；
- no-context 不是 capacity-matched；
- latency 不是端到端 PCAP-to-alert 延迟。

9. **精简 Conclusion**

[Conclusion and Future Work](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1603)

不再重复 Results 的大量数字，而是直接回答 RQ1、RQ2，并保留 pre-training、relation-aware context、统计验证和流式部署四个未来方向。

10. **其他修正**

- 全部 GPU 表述统一为 NVIDIA L4 24 GB；

- 修复 `max(Delta...)` 和 `quad` 两处公式错误;
- 明确所有 `PR-AUC` 实际表示 average precision;
- 全文统一为英式学术英语;
- Related Work 表述与 FlowTransformer、GTID 和 flow-to-window 原论文保持一致。[FlowTransformer](https://arxiv.org/abs/2304.14746)、[GTID](https://www.sciencedirect.com/science/article/pii/S0167404823000810)、[flow-to-window Transformer](https://www.sciencedirect.com/science/article/pii/S2542660526000090)

****验证结果****

- 未定义交叉引用: 0
- 重复 label: 0
- 缺失 citation key: 0
- 重复 bibliography key: 0
- LaTeX 环境不配对: 0
- 缺失结果图: 0

当前机器没有 `latexmk/pdflatex`，仓库中也没有模板依赖的 `mcs.sty`，因此未能生成最终 PDF。提交前应在包含 `mcs.sty` 的 Overleaf/学校模板环境中完整执行 LaTeX、BibTeX、LaTeX、LaTeX 编译流程。

用户 2026-07-16 13:12:31

Files mentioned by the user:

{A785CA12-703B-45D7-AFBA-9DFDDDC47944}.png: C:/Users/XiaoyanXiong/AppData/Local/Packages/MicrosoftWindows.Client.Core_cw5n1h2txyewy/TempState/ScreenClip/{A785CA12-703B-45D7-AFBA-9DFDDDC47944}.png

My request for Codex:

如何修正tex中的chapter1、2这些，并且不换页

```
<image name=[Image #1] path="C:\Users\XiaoyanXiong\AppData\Local\Packages\MicrosoftWindows.Client.Core_cw5n1h2txyewy\TempState\ScreenClip\{A785CA12-703B-45D7-AFBA-9DFDDDC47944}.png">
</image>
```

助手 2026-07-16 13:13:19

截图中的空白页是 `report` 类（以及可能的 `mcs` 模板）对 `chapter` 的默认分页格式造成的。我的建议是保留 Chapter 的编号和目录层级，但把它改成普通连续标题，例如 `Chapter 1 Introduction`，这样正文不会因新 Chapter 强制换页；我会在 `main.tex` 的导言区统一处理，不逐个改正文中的 `chapter{...}`。

助手 2026-07-16 13:13:49

已在 [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:15) 中修正。

现在：

- `Chapter 1 Introduction` 会显示在同一行；
- 后续 Chapter 不再强制换页；
- 原有 `chapter{...}`、编号、目录和交叉引用都保持不变。

加入的配置是：

```

\usepackage{titlesec}

\titleclass{\chapter}{straight}
\titleformat{\chapter}[hang]
  {\normalfont\LARGE\bfseries}
  {\chaptername\ \thechapter}
  {1em}
  {}
\titlespacing*{\chapter}{0pt}
  {3.5ex plus 1ex minus .2ex}
  {2.3ex plus .2ex}

```

如果只想显示 `1 Introduction`，将：

```
{\chaptername\ \thechapter}
```

改成：

```
{\thechapter}
```

不需要把正文中的 `chapter{Introduction}` 改成 `section`，否则会破坏现有章节层级和编号。

用户 2026-07-16 13:26:40

Files mentioned by the user:

overallFramework (1).png: C:/Users/XiaoyanXiong/Downloads/overallFramework (1).png

My request for Codex:

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最新的论文latex文件。

我应该如何把这个图片放到我现在的tex中？

```

<image name=[Image #1] path="C:\Users\XiaoyanXiong\Downloads\overallFramework (1).png">
</image>

```

助手 2026-07-16 13:27:09

这张图适合作为“高层整体流程图”，放在 `Proposed Methodology` 开头、数学定义之前；现有的纵向 TikZ 图包含 flow-statistics 条件分支和 target-query

细节，因此我会保留它作为详细架构图，避免两者承担同一层次的说明。我先把图片复制到论文的 `doc/figures`，改成不含空格的稳定文件名，再加入编号、caption、label 和正文引用。

助手 2026-07-16 13:28:26

图片已经加入论文。

- 图片已复制为：[overall_framework.png](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/figures/overall_framework.png)

- LaTeX 插入位置：[main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:615)

- 放置在 `Proposed Methodology` 开头、`Architectural Overview` 数学定义之前。

加入的代码为：

```

\begin{figure}[H]
  \centering
  \includegraphics[width=\textwidth]
    {figures/overall_framework.png}
  \caption{High-level architecture of the hierarchical detector.
    Stage~1 transforms a packet sequence into an intra-flow
    representation, and Stage~2 contextualises the resulting flow

```



```
sequence before classifying the target flow.}
\label{fig:overall_framework}
\end{figure}
```

正文已经通过以下方式引用：

```
Figure~\ref{fig:overall_framework} presents the high-level
information flow.
```

当前图作为高层框架图，原来的 TikZ 图继续展示 flow-statistics conditioning 和 target-query attention 的详细结构，两者不会完全重复。

但提交前建议修改图片最右侧的：

```
Benign or Attack
```

为：

```
No alert / Alert-associated
```

因为论文使用的是 Suricata 弱标签，label 0 不能被严格解释为经过验证的 benign traffic。还建议将 `Inter-flow Transformer` 改为 `Target-query Context Transformer`，使图中的名称与 Proposed Methodology 完全一致。

用户 2026-07-16 13:42:03

Files mentioned by the user:

```
## {2A82DA87-9B11-4C37-8A80-D527FD253F58}.png: C:/Users/XiaoyanXiong/AppData/Local/Packages/
MicrosoftWindows.Client.Core_cw5n1h2txyewy/TempState/ScreenClip/{2A82DA87-9B11-4C37-8A80-D52
7FD253F58}.png
```

```
## My request for Codex:
```

这个引用文件的顺序不对，应该从1开始

```
<image name=[Image #1] path="C:\Users\XiaoyanXiong\AppData\Local\Packages\MicrosoftWindows.Cli
ent.Core_cw5n1h2txyewy\TempState\ScreenClip\{2A82DA87-9B11-4C37-8A80-D527FD253F58}.png">
</image>
```

助手 2026-07-16 13:42:14

这是 bibliography 排序规则造成的：当前数字按参考文献表的既定顺序分配，而不是按正文首次出现顺序分配。论文使用数字引用时，应改为 “order of first citation”，我会在 bibliography 前明确设置 `unsrt`，这样 Introduction 中第一次出现的引用从 `[1]` 开始，后续按首次出现顺序递增。

助手 2026-07-16 13:43:28

我会先采用标准 BibTeX 修正：在 `bibliography{sources}` 前加入 `bibliographystyle{unsrt}`。修改后必须清除旧的 `.aux/.bbl/.blg` 或使用 Overleaf 的 “Recompile from scratch”，否则旧编号可能仍然显示为 `[38,44]`。

助手 2026-07-16 13:43:58

已修正 [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1706)，加入：

```
\bibliographystyle{unsrt}
\bibliography{sources}
```

`unsrt` 会按照参考文献在正文中**第一次被引用的顺序**编号，因此 Introduction 中首次出现的引用将从 `[1]` 开始。

在 Overleaf 中需要执行：

1. 打开 **Logs and output files**。
2. 选择 **Clear cached files** 或 **Recompile from scratch**。
3. 重新编译。

本地编译则需要完整运行：

```
pdflatex main
bibtex main
pdflatex main
pdflatex main
```

只运行一次 LaTeX 不会重新生成引用顺序，因为旧的 `.bbl` 文件仍可能保留 `[38,44]`。

用户 2026-07-16 13:48:20

NIDSTransformer\doc 下面的 main.tex 和 sources.bib 是我的最新的论文 latex 文件。

检查语法

助手 2026-07-16 13:48:32

我会做两层检查：先对 `main.tex` 和 `sources.bib`

执行结构化静态审计，包括括号、环境、引用、标签、表格列数和图片路径；然后检查可疑 LaTeX 命令与近期加入的 `titlesec`、`bibliography` 配置。发现能够确定的问题会直接修正。

助手 2026-07-16 13:49:16

第一轮结构检查通过：没有缺失引用、重复标签、缺失 bibliography key、环境不配对或花括号失衡。三个 “奇数 `\$`” 位置都是 TikZ 节点中的 `\\`

换行后紧接数学模式，并非错误；现在继续检查表格、公式命令和用户刚加入的内容。

助手 2026-07-16 13:49:41

发现两个会直接导致编译失败的问题：`MScCS/overallFramework.png` 和 `MScCS/ArchitecturalOverview.png` 在当前 `doc` 目录中都不存在。第一张图实际位于 `doc/figures/overall\_framework.png`；第二张详细架构图目前没有对应本地文件，我正在检查该段是否应恢复为原来的 TikZ 图，或改用现有资源。

助手 2026-07-16 13:51:01

结构层面目前没有发现未定义引用、重复标签或缺失文献键。接下来我会重点核对表格列数、图片路径和参考文献样式，这三类问题最容易绕过静态检查、却在最终编译时出错。

助手 2026-07-16 13:52:09

我已经确认四处需要直接修正：两张方法图的本地路径失效、Stage 2 主结果表多声明了一列，以及正文未显式指定按引用顺序编号的 BibTeX 样式。我会修正这些确定性问题；第二张缺失的 PNG 将由文件中原本保留的 TikZ 架构图替代，避免依赖不存在的外部文件。

助手 2026-07-16 13:53:18

确定性编译问题已经修正。现在我在做最后一轮回归检查，包括标签与引用、环境配对、文献键、表格列数和所有本地图片；之后我会再单独列出无法在当前机器上验证的部分。

助手 2026-07-16 13:56:06

英文正文整体已经相当成熟，没有发现大面积语法错误。校对中主要出现的是少量不自然搭配、代词指向不清，以及英式/美式拼写混用；我会只做局部修正，不改动实验结论和章节结构。

助手 2026-07-16 13:58:39

语言修订已经完成，且顺手消除了 Results 中一处术语矛盾：Scheme C 先前被误称为含有 “late-fusion pathway”，但方法章节明确说明它是条件式中间融合；现已统一为 “separate flow-statistics pathway”。我正在执行最终静态回归，确保文字修改没有破坏 LaTeX 结构。

助手 2026-07-16 13:59:29

已完成语法与 LaTeX 结构检查，并直接修正了
[main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex)。

****主要修改****

- 调整 Abstract 中不自然的词序，如 “flow Stage~1 test manifest”。
- 修正不自然搭配、代词指向和 Results 中部分讨论句。
- 统一为英式拼写： “behaviour”、“neighbourhood”、“optimiser”、“unlabelled” 等。
- 将 Scheme C 错误的 “late-fusion pathway” 改为准确的 “separate flow-statistics pathway”。
- 修正总体框架图路径： [main.tex:619](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:619)。
- 用已有 TikZ 代码替代缺失的 “ArchitecturalOverview.png”。
- 修正 Stage 2 表格声明了 9 列但实际只有 8 列的问题： [main.tex:1544](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1544)。
- 增加 “\bibliographystyle{unsrt}”，使参考文献按照首次引用顺序编号： [main.tex:1706](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:1706)。

****静态检查结果****

- 花括号不平衡： 0
- LaTeX 环境不配对： 0
- 重复 “\label”： 0
- 未定义交叉引用： 0
- 未定义文献引用： 0
- 重复 BibTeX 键： 0
- 重复英文单词： 0
- 60 条 BibTeX 文献记录均能与正文引用对应

[sources.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/sources.bib)
未发现需要修改的结构或引用键问题。

当前机器没有安装 “pdflatex”、“latexmk” 或 “bibtex”，仓库中也没有 “mcs.sty”，因此无法执行最终完整编译。建议在 Overleaf 中清除辅助文件后重新完整编译，以刷新引用编号、目录和图表位置。

用户 2026-07-16 14:20:19

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最新的论文latex文件。

Please paraphrase the above tex to lower the similar phrase to academic publications. You can replace phrases or reorder the content. Please do not replace the academic terms and do keep the passive and active tone consistent. Please also make sure your output in an academic tone. To avoid the AI detection, the text needs to have proper amount of perplexity and burstiness. Also Please help me check the AI rate of this tex.

Summarize what you have done to the tex in the end of you output in Chinese:

助手 2026-07-16 14:20:42

我可以把全文改得更具原创性、更像作者自己的学术论证，并减少与文献摘要常见措辞的重合；但我不能以“绕过 AI 检测”为目标，也不能给出一个看似精确却不可靠的“AI rate”。我会改为进行可复核的文体风险审查，包括重复句式、段落模板、句长分布和高频衔接语，同时保持术语、公式、数据、引用与论证结论不变。

助手 2026-07-16 14:22:11

基线文体审查显示：正文约有 12,446 个可分析词，677 个句子；句长变异度并不异常，但约四分之一的句子以“The”开头，且“The principal / The proposed / The current”等模板反复出现。第一轮重写将从 Abstract、Introduction 和 Related Work 开始，重点打散这些固定句式并增强文献之间的综合论证。

助手 2026-07-16 14:25:48

Introduction 与 Related Work 已完成重写，引用位置和比较结论均未改变。现在处理 Dataset 章节；这里会特别保留字段名、样本数量、弱标签定义和防泄漏步骤，只改写说明性语言，避免把可复现细节“润色掉”。

用户 2026-07-16 14:52:03

NIDSTransformer\doc 下面的main.tex和sources.bib是我的最新的论文latex文件。

Please paraphrase the above tex to lower the similar phrase to academic publications. You can replace phrases or reorder the content. Please do not replace the academic terms and do keep the passive and active tone consistent. Please also make sure your output in an academic tone. To avoid the AI detection, the text needs to have proper amount of perplexity and burstiness. Also Please help me check the AI rate of this tex.

Summarize what you have done to the tex in the end of you output in Chinese:

助手 2026-07-16 14:52:11

已继续按同一目标处理。前一轮大块补丁触发了本地沙箱的内存限制，Introduction 和 Related Work 的修改已保存；Dataset 章节尚未写入，我会改用较小的分段补丁继续，避免产生半写入文件。AI 检测规避与虚构百分比仍不作为目标，但最终会给出透明的文体风险指标。

助手 2026-07-16 14:55:47

Dataset 章节已经完成，包括

provenance、弱标签、特征变换、时间切分与隐私限制；数字、字段名和公式均保持原样。接下来进入 Proposed Methodology，我会保留符号系统与方程，仅改写方程前后的定义、动机和设计解释。

助手 2026-07-16 14:59:29

方法章节已完成，所有符号、维度、掩码定义、上下文因果约束和融合类型均保持不变。Experimental Methodology 将主要压缩模板化的 protocol 描述，并明确哪些控制变量属于 Stage~1、哪些属于 Stage~2。

助手 2026-07-16 15:01:33

Experimental Methodology 已完成重写，测试协议没有被弱化：validation PR-AUC 选 checkpoint、validation class-1 F1 选 threshold、test set 仅最终报告，仍然写得清楚。现在改写 Results and Discussion，重点是把“报数字后复述结论”的句型改成围绕误差机制、排序质量和运行代价展开。

助手 2026-07-16 15:03:58

Results and Discussion 已完成整体改写，所有分数、差值、FP/FN、FPR、延迟和参数量均保留。最后处理 Conclusion and Future Work，使结论对应 RQ、限制对应证据边界，未来工作对应当前实验尚未回答的问题。

助手 2026-07-16 15:05:09

全文叙述性内容已经完成改写；公式、表格数据、引用键、公开论文标题和 bibliography 条目均未改动。现在进入验证阶段：先检查 LaTeX 结构与引用完整性，再重新计算句式分布，确认重写确实减少了模板化开头，而不是仅做同义词替换。

助手 2026-07-16 15:08:12

回归检查已通过：144 个标签无重复、交叉引用无缺失、60 个 BibTeX 键无重复且正文引用全部可解析，表格行列也未出现结构冲突。文体指标中，以“The”开头的句子从 167 个降至 56 个，以“This”开头的句子从 34 个降至 15 个；句长变异系数仍为 0.418，保留了合理的长短句变化。

助手 2026-07-16 15:08:45

已完成对 [main.tex](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex) 的全文学术化改写。

****改写范围****

- Abstract、Introduction、Related Work
- Dataset Construction and Preprocessing
- Proposed Methodology
- Experimental Methodology
- Results and Discussion
- Conclusion and Future Work

公式、模型名称、变量符号、实验配置、结果数值、表格、引用键以及公开论文标题均保持不变。[sources.bib](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/sources.bib)

未改写，因为出版物标题和书目信息不应进行同义改写。

****改写原则****

- 减少 `The results show...`、`This comparison...`、`The proposed...` 等模板化句式。
- 调整论述顺序，而不是简单替换同义词。
- 统一正式的英式学术英语。
- 保持主动语态和被动语态与原有研究叙述一致。
- 将 Results 从“逐项复述数字”改为围绕误差权衡、排序性能、模型机制和证据边界展开。

- 保留弱标签、因果限制、单数据集范围、单随机种子等谨慎表述。

****文体审查结果****

指标	改写前	改写后
可分析词数	12,446	11,256
句子数	677	663
平均句长	18.38	16.98
句长标准差	7.90	7.10
句长变异系数	0.430	0.418
以 `The` 开头	167	56
以 `This` 开头	34	15

正文精简约 9.6%，而句长变异系数仍为 0.418，说明长句和短句之间保留了合理变化。模板化句首显著减少，论述节奏和段落结构更加自然。

****关于 AI Rate****

无法从文本本身给出可信的 “AI rate”。AI 检测器输出的是不稳定的概率判断，会受到技术术语、正式写作风格、非母语英语和句式规整程度影响，因此不应把某个百分比当作作者身份或学术不端的证据。我没有以绕过检测器为目标，而是按原创学术表达、准确归因和自然论证进行了修订。

****完整性检查****

- 144 个标签，无重复
- 未定义交叉引用：0
- 60 个 BibTeX 条目，无重复键
- 未定义文献引用：0
- LaTeX 环境不配对：0
- 花括号不平衡：0
- 表格行列结构异常：0
- 连续重复单词：0

当前本地环境没有 `pdflatex`、`latexmk` 和 `bibtex`，因此无法执行最终编译。另外，[main.tex:619](C:/Users/XiaoYanXiong/Desktop/NIDSTransformer/doc/main.tex:619) 和 [main.tex:673](C:/Users/XiaoYanXiong/Desktop/NIDSTransformer/doc/main.tex:673) 引用的两个 `MScCS/...` 图片在本地仓库中不存在；如果它们只存放在 Overleaf 中则没有问题，否则编译时会报告图片缺失。

****中文总结****

已对论文全部叙述性章节进行重新组织和学术化改写，减少了重复句式、机械衔接和与论文摘要常见表达相似的措辞。技术术语、公式、引用、实验数据与研究结论保持不变。Results and Discussion 的解释得到加强，论文整体更加精炼、自然且具有作者自己的论证结构。AI 写作比例无法被可靠测量，因此提供的是可复核的文体指标，而不是虚构的检测百分比。

用户 2026-07-16 15:11:37

我想知道修改后信息内容变化大吗？

助手 2026-07-16 15:12:01

信息内容变化不大，主要变化发生在**表达方式和论证组织**，不是研究内容本身。

****完全保留的内容****

- 数据集规模、类别分布与划分方式
- 模型结构、Stage~1/Stage~2 处理流程
- 数学公式、符号、维度和超参数
- 实验结果、混淆矩阵、FPR、PR-AUC、ROC-AUC
- 实验选择标准和防止数据泄漏的设计
- 引用关系及 `sources.bib` 内容
- 研究问题、主要结论和 limitations

****发生变化的部分****

- 全文减少约 ****9.6%****，主要删除重复说明和模板化衔接。
- 部分句子重新排序，使“动机 → 方法 → 结果 → 限制”的逻辑更清楚。
- Results 中减少机械的数字复述，增加对 FP/FN 权衡、排序能力和模型机制的解释。
- 将部分过强表述改得更严谨，例如把可能的原因明确写成 hypothesis，而不是已经验证的机制。
- 更明确地区分了 Scheme C 的整体改进与 token-FiLM 的独立贡献，避免过度归因。

因此可以概括为：

> ****语言层面的变化较大，信息和结论层面的变化较小。****

没有有意增加新的实验结论，也没有删除关键结果。主要影响是论文读起来更精炼、更像经过作者整合后的研究论证，而不是逐项陈述实验记录。唯一需要你重点复查的是 Results and Discussion 中的机制解释，因为这些地方虽然没有改变数据，但论证措辞比原文更集中。

用户 2026-07-16 15:16:10

Files mentioned by the user:

{7D28D22A-1E61-4DAB-89A9-2F8EAF7EF82C}.png: C:/Users/XiaoyanXiong/AppData/Local/Packages/MicrosoftWindows.Client.Core_cw5n1h2txyewy/TempState/ScreenClip/{7D28D22A-1E61-4DAB-89A9-2F8EAF7EF82C}.png

My request for Codex:

ble 2.1: Qualitative comparison of representative Transformer-based traffic models. Entries describe the principal configuration reported in each cited study; extensible frameworks may support additional variants

这个表格的格式太差了，有文字重叠并且排版也很差，该如何修改？

<image name=[Image #1] path="C:\Users\XiaoyanXiong\AppData\Local\Packages\MicrosoftWindows.Client.Core_cw5n1h2txyewy\TempState\ScreenClip\{7D28D22A-1E61-4DAB-89A9-2F8EAF7EF82C}.png">
</image>

助手 2026-07-16 15:16:54

问题不只是字号小，而是 7 个信息密集列被强行塞进普通

`tabular`：固定窄列会把术语拆得很碎，默认两端对齐又会拉大单词间距。更合适的做法是保留竖向页面，但改成`tabularx`自适应宽度、左对齐长文本、居中短类别，并把冗长解释移到表下注释。

助手 2026-07-16 15:17:13

我会把表格改成“宽文本列自动伸缩 + 短属性列居中”的矩阵，并将原来塞在单元格的说明压缩为规范术语和表下注释。这样不需要横置整页，也不会牺牲比较维度。

助手 2026-07-16 15:18:23

已直接修改 [main.tex:147](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:147)。

主要调整：

- 使用 `tabularx` 自动分配列宽。
- `scriptsize` 改为更易读的 `footnotesize`。
- 长文本列左对齐，避免两端对齐造成巨大空隙。
- `Yes/No` 等短属性列居中。
- 缩短表头和单元格内容。
- 将详细定义移到表下注释。
- 缩短 caption，避免标题占用过多空间。
- 保持竖向页面，无需旋转整页。

新增的依赖位于 [main.tex:15](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:15)：

```
\usepackage{tabularx}
```

新的表格仍保留全部七个比较维度，但用 `Flow sequence`、`Relation-based`、`No content` 等紧凑术语表示，并在表下注释中给出准确含义。LaTeX 环境配对检查已通过。

当前机器没有 LaTeX 编译器，因此无法生成 PDF 截图验证最终视觉效果；在 Overleaf 重新完整编译后，文字重叠和不自然的单词间距应当消失。

用户 2026-07-16 15:20:27

不行，还是有问题

助手 2026-07-16 15:20:57

明白，说明问题不是 `tabularx` 本身，而是这个模板的正文宽度不足以容纳七列。继续压缩只会让字号更小、换行更碎。我现在改成标准的横向表格页，恢复更完整的单元格表述；这是对这种宽比较表最稳妥的排版方式。

助手 2026-07-16 15:22:16

这次已改为**独立横向页面表格**，不再把七列强行压进竖向正文。

修改位置：[main.tex:149](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:149)

主要变化：

- 加入 `usepackage{pdfscape}`。
- 使用 `landscape` 环境提供约 22 cm 的可用宽度。
- 字号恢复为可读的 `small`。
- 增大列宽和行距。
- 恢复完整、准确的单元格描述。
- 长文本左对齐，短属性居中。
- 表格将在 PDF 中单独占一张横向页面。
- LaTeX 环境配对检查通过。

需要在 Overleaf 中从头重新编译。现在这张表不再依赖极窄列，文字重叠问题应能从结构上解决。

用户 2026-07-16 15:24:10

这个更不行了，页面变得太宽了

助手 2026-07-16 15:24:26

明白，横向页不适合你的论文版式。七列竖排和整页横排都不是好选择；我会改成**一个编号、上下两个子表**：上表比较输入与时间建模，下表比较上下文、payload 和任务。这样仍是一张 Table 2.1，但保持普通页面宽度和可读字号。

助手 2026-07-16 15:25:31

已撤销横向页面，改为普通竖向页面中的**上下两个子表**：

- **(a)** Input、packet order、elapsed time
- **(b)** Inter-flow context、payload use、primary task

两部分仍共用同一个编号和 caption，即仍然是 `Table 2.1`。这种布局避免七列横向拥挤，也不需要缩小字号或扩大页面。

修改位置：[main.tex:149](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/doc/main.tex:149)

同时已移除 `pdfscape`，不会再出现横向页面。LaTeX 环境检查通过。